DEVOPS ENGINEER COURSE

Networking for DevOps Engineers

From packets and subnets to cloud and Kubernetes

Beginner | Intermediate | Advanced

Concepts · hands-on labs · troubleshooting · interview Q&A · cheat sheet

Contents

Part 1 — Foundations 5

1.1 What a Network Actually Is 5
1.2 Layered Models: OSI vs TCP/IP 6
1.3 Ethernet and MAC Addresses 8
1.4 ARP: Gluing Layer 3 to Layer 2 9
1.5 Switches 11
1.6 VLANs 12
1.7 Putting It Together: A Packet’s Journey on a LAN 13
1.8 Lab: Build a Virtual LAN with Network Namespaces 13
Key takeaways 15

Part 2 — IP Addressing, Subnetting, NAT, and IPv6 16

2.1 IPv4 Addresses 16
2.2 CIDR Reference Table 17
2.3 The Fast Method: Block Sizes 18
2.4 Dividing Networks: Subnetting and VLSM 20
2.5 Route Summarization (Supernetting) 21
2.6 Subnetting Practice Problems 22
2.7 Special and Private Address Ranges 23
2.8 NAT and PAT 24
2.9 IPv6 Essentials 25
2.10 Lab: Subnet Calculator Drills and NAT in Namespaces 27
Key takeaways 29

Part 3 — Routing, Transport (TCP and UDP), Ports, Sockets, and DHCP 30

3.1 Routing Fundamentals 30
3.2 Dynamic Routing Protocols 32
3.3 The Transport Layer: Ports and Sockets 33
3.4 TCP in Depth 34
3.5 UDP 38
3.6 DHCP 39
3.7 Lab: Routing Between Networks and Watching TCP 40
Key takeaways 41

Part 4 — DNS in Depth 43

4.1 What DNS Does 43
4.2 The Resolution Path 43
4.3 Record Types 45
4.4 TTL and Caching Strategy 46
4.5 Split-Horizon DNS 47
4.6 Routing Policies (Route 53 Style) 48
4.7 DNS in Kubernetes 48
4.8 DNSSEC and Encrypted DNS (Brief) 49
4.9 Debugging with dig 50

4.10 Lab: Run Your Own Authoritative and Recursive DNS 51
Key takeaways 53

Part 5 — HTTP, TLS, Certificates, and Proxies 54
5.1 HTTP Fundamentals 54
5.2 HTTP Versions 56
5.3 Using curl as a Network Debugger 57
5.4 TLS 58
5.5 Proxies 62
5.6 Lab: Build a Private CA, Serve TLS and mTLS with Nginx 63
Key takeaways 65

Part 6 — Load Balancers, API Gateways, CDNs, Firewalls, and VPNs 66
6.1 Load Balancing Concepts 66
6.2 Nginx and HAProxy Configurations 68
6.3 Reverse Proxies and API Gateways 70
6.4 CDNs 71
6.5 Firewalls 72
6.6 VPNs 74
6.7 Lab: Load Balancing with HAProxy and Failure Injection 76
Key takeaways 77

Part 7 — Cloud Networking: AWS VPC in Depth, Azure and GCP, Terraform 79
7.1 AWS VPC Building Blocks 79
7.2 Connecting VPCs and Networks 81
7.3 Azure and GCP Equivalents 83
7.4 Terraform: A Production-Style VPC 84
7.5 Lab: Build and Verify the VPC 88
Key takeaways 89

Part 8 — Container Networking, Kubernetes Networking, and Service Mesh 91
8.1 Docker Networking 91
8.2 The Kubernetes Network Model 93
8.3 CNI Plugins 94
8.4 Services 95
8.5 Ingress and Gateway API 97
8.6 NetworkPolicy 98
8.7 CoreDNS 99
8.8 Pod Lifecycle and Networking Gotchas 100
8.9 Service Mesh 101
8.10 Lab: Kubernetes Networking Hands-On with kind 102
Key takeaways 104

Part 9 — Troubleshooting Toolkit, Methodology, and Network Security 106
9.1 The Toolkit 106

Part 10 — Real-World Scenarios, Interview Q&A, and Cheat Sheet 109
10.1 Scenario: “The Site Is Slow” 109
10.2 Scenario: “Can’t Reach the Database from a Pod” 110

10.3 Scenario: “Intermittent 502s” 111
10.4 Scenario: “DNS Works on One Host but Not Another” 112
10.5 Scenario: “New VPC Peering, Nothing Flows” 113
10.6 Scenario: “Outbound Calls Sporadically Time Out from Private Subnets” 114
10.7 Capstone Lab: Run an Incident 114
10.8 Interview Questions and Answers 114
10.9 Cheat Sheet 119
Key takeaways 123

Part 1 — Foundations

Networking is the one discipline that every DevOps task eventually touches. A failed deployment, a slow API, a pod that cannot reach its database, a certificate warning, a mysterious 502: all of them are networking problems wearing a different costume. This Part builds the mental model you will use for the rest of the course. Do not skip it even if you "know the OSI model"; the goal here is not to memorize layers but to be able to *reason* about where a packet is and what is touching it.

1.1 What a Network Actually Is

A network is a set of devices (hosts) that can exchange data using agreed-upon rules (protocols). That definition hides three separate problems that every network must solve:

1. **Physical delivery** — getting bits from one interface to another over copper, fiber, or radio.
2. **Addressing** — identifying who the data is for, both on the local segment and across the world.
3. **Reliability and meaning** — making sure data arrives intact, in order (when needed), and is delivered to the right application.

Each of those problems is solved by a different layer of protocols. Engineers split the problem into layers so that each layer can evolve independently. Your web application does not care whether the packet travelled over Wi-Fi or a 100 Gbit fiber, and your Ethernet switch does not care whether the payload is HTTP or SSH.

1.1.1 Key vocabulary

Term	Meaning
Host	Any device with a network interface: server, laptop, container, VM
Interface / NIC	The point where a host attaches to a network (`eth0` , `ens5` , `wlan0`)
Link	A direct connection between two or more interfaces at Layer 2
Segment / broadcast domain	The set of hosts that receive each other's broadcasts
Packet	A unit of data at Layer 3 (IP)
Frame	A unit of data at Layer 2 (Ethernet)
Segment (TCP) / Datagram (UDP)	A unit of data at Layer 4
Protocol	A formal set of rules for formatting and exchanging messages
Bandwidth	Maximum rate of a link (bits per second)
Throughput	Actual achieved rate
Latency	Time for a bit to travel from A to B (often measured as round-trip time, RTT)
Jitter	Variation in latency

Term	Meaning
Packet loss	Percentage of packets that never arrive

> **Note:** “Bandwidth” and “latency” are independent. A satellite link can have high bandwidth and terrible latency. A cross-region API call is usually latency-bound, not bandwidth-bound. Most “the network is slow” tickets are actually latency multiplied by the number of round trips an application makes.

1.1.2 LAN, WAN, and the Internet

- A **LAN** (Local Area Network) is a network under one administrative control in one location, typically Ethernet and Wi-Fi.
- A **WAN** (Wide Area Network) connects LANs across distance: MPLS circuits, SD-WAN, site-to-site VPNs, or the Internet itself.
- **The Internet** is a network of about 75,000+ independently operated networks (Autonomous Systems) that agree to exchange routes using BGP.
- A **cloud VPC** is a software-defined LAN that lives inside a provider’s data center. It behaves like a LAN to you but is implemented completely differently underneath (no real broadcast, no real ARP on the wire).

1.2 Layered Models: OSI vs TCP/IP

1.2.1 The OSI model

The OSI (Open Systems Interconnection) model has seven layers. Nobody implements OSI as a protocol suite, but its numbering is the industry’s shared language: “that’s an L4 load balancer”, “L7 firewall”, “L2 adjacency”.

#	OSI Layer	Job	Examples	Unit
7	Application	Application semantics	HTTP, DNS, SMTP, gRPC	Message
6	Presentation	Encoding, encryption, compression	TLS (loosely), JSON, gzip	Message
5	Session	Session setup and teardown	RPC sessions, TLS sessions (loosely)	Message
4	Transport	End-to-end delivery between processes	TCP, UDP, QUIC	Segment / Datagram
3	Network	Logical addressing and routing across networks	IPv4, IPv6, ICMP	Packet
2	Data Link	Delivery on the local link, MAC addressing	Ethernet, 802.11, ARP (L2/L3 boundary)	Frame
1	Physical	Bits on a medium	Copper, fiber, radio, optics	Bit

1.2.2 The TCP/IP model

The TCP/IP model is what the Internet actually runs. It collapses OSI 5–7 into one “Application” layer and OSI 1–2 into “Link”.

```
   OSI                       TCP/IP
+------------------+       +------------------+
| 7 Application    |       |                  |
| 6 Presentation   |  -->  |   Application    |   HTTP, DNS, TLS, SSH
| 5 Session        |       |                  |
+------------------+       +------------------+
| 4 Transport      |  -->  |   Transport      |   TCP, UDP, QUIC
+------------------+       +------------------+
| 3 Network        |  -->  |   Internet       |   IP, ICMP
+------------------+       +------------------+
| 2 Data Link      |  -->  |   Link           |   Ethernet, Wi-Fi, ARP
| 1 Physical       |       |                  |
+------------------+       +------------------+
```

> **Interview:** “Which layer does TLS live in?” The honest answer: it sits between the transport layer and the application, so people call it L6 or “L4.5”. In the TCP/IP model it is part of the application layer. Show you understand that models are approximations, not laws.

1.2.3 Encapsulation: why layers matter in practice

When your application sends data, each layer wraps the payload with its own header. On the receiving side, each layer strips its header and hands the rest upward.

```
Application data:                                [ HTTP GET /index.html ]
Transport (TCP) adds ports, seq numbers:     [TCP| HTTP GET /index.html ]
Network (IP) adds source/dest IP:         [IP|TCP| HTTP GET /index.html ]
Link (Ethernet) adds MACs + FCS:      [ETH|IP|TCP| HTTP GET /index.html |FCS]

Typical header sizes:
  Ethernet header 14 bytes (+4 FCS)   IPv4 header 20 bytes (no options)
  TCP header 20 bytes (no options; usually 32 with timestamps)
  IPv6 header 40 bytes                UDP header 8 bytes
```

This is the single most useful model for troubleshooting. Every device in the path operates on a specific layer and only looks at headers up to that layer:

Device	Highest layer it reads	What it changes
Hub / repeater (obsolete)	L1	Nothing, just repeats bits
Switch	L2	Nothing in the frame (it forwards by MAC)
Router	L3	Rewrites L2 header, decrements TTL

Device	Highest layer it reads	What it changes
NAT device	L3/L4	Rewrites IPs and ports
Stateful firewall / Security Group	L3/L4	Drops or allows; tracks connections
L4 load balancer	L4	Rewrites destination IP/port (or proxies TCP)
L7 load balancer / reverse proxy	L7	Terminates TCP and TLS, opens a new connection

> **Tip:** When debugging, ask “which layer is broken?” and test bottom-up. Is the interface up (L1/L2)? Do I have an IP and a route (L3)? Is the port open (L4)? Does the application respond correctly (L7)? Part 9 turns this into a formal methodology.

1.2.4 MTU and why headers cost you

The **MTU** (Maximum Transmission Unit) is the largest L3 packet a link can carry without fragmentation. Standard Ethernet MTU is 1500 bytes. Subtract 20 bytes IPv4 and 20 bytes TCP and you get a **MSS** (Maximum Segment Size) of 1460 bytes of application data per packet. Tunnels (VXLAN, IPsec, WireGuard, GRE) add their own headers and reduce effective MTU. That is the root cause of a whole class of “small requests work, large ones hang” bugs, which we cover in Part 9.

Network	Typical MTU
Standard Ethernet	1500
AWS within a VPC	9001 (jumbo frames)
AWS over IGW / VPN / inter-region peering	1500 (VPN lower, approx. 1436 or less)
VXLAN overlay on 1500 underlay	1450
WireGuard default	1420
PPPoE home broadband	1492

1.3 Ethernet and MAC Addresses

1.3.1 The Ethernet frame

Ethernet (IEEE 802.3) is the dominant Layer 2 technology, and Wi-Fi (802.11) is designed to look like Ethernet to upper layers.

```
+-------------+-------------+-----------+------------------+-----+
| Dest MAC    | Source MAC  | EtherType | Payload          | FCS |
| 6 bytes     | 6 bytes     | 2 bytes   | 46–1500 bytes    | 4 B |
+-------------+-------------+-----------+------------------+-----+
EtherType examples: 0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP, 0x8100 = 802.1Q VLAN tag
```

The **FCS** (Frame Check Sequence) is a CRC. A receiving NIC silently drops frames with a bad FCS, which shows up as `rx_crc_errors` in interface counters. Rising CRC errors almost always mean a physical problem: bad cable, dirty optic, duplex mismatch.

1.3.2 MAC addresses

A MAC address is a 48-bit identifier written as six hex octets: `0a:1b:2c:3d:4e:5f`.

- The first three octets are the **OUI** (Organizationally Unique Identifier) assigned to the manufacturer.
- The least significant bit of the first octet is the **I/G bit**: 0 = unicast, 1 = multicast.
- The second-least significant bit is the **U/L bit**: 1 = locally administered. Virtual NICs, containers, and phones using MAC randomization set this bit. The `02:42:...` prefix you see on Docker containers is a locally administered address.
- `ff:ff:ff:ff:ff:ff` is the broadcast address: every host on the segment receives it.

MAC addresses are only meaningful **on the local link**. Every time a packet crosses a router, the source and destination MACs are rewritten. The IP addresses stay the same end-to-end (unless NAT is involved).

```
# Show interfaces and their MAC addresses
ip link show
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP mode DEFAULT group
default qlen 1000
    link/ether 0a:3f:91:c2:7d:11 brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode
DEFAULT group default
    link/ether 02:42:5e:8a:01:9c brd ff:ff:ff:ff:ff:ff
```

Read that output carefully: `UP` means administratively up, `LOWER_UP` means the physical link (carrier) is detected. `NO-CARRIER` on `docker0` simply means no containers are attached yet.

1.4 ARP: Gluing Layer 3 to Layer 2

1.4.1 The problem ARP solves

Your application knows the destination **IP** address. The NIC can only send frames to a **MAC** address. ARP (Address Resolution Protocol, RFC 826) asks the local segment: “Who has 10.0.1.20? Tell 10.0.1.10.”

The critical decision a host makes before sending any packet:

```
Is the destination IP in one of my directly connected subnets?
   |
   +-- YES --> ARP for the destination IP itself, send frame directly to it
```

```
   |
   +-- NO  --> Look up the route table, find the next hop (usually the default gateway),
               ARP for the GATEWAY's IP, send the frame to the gateway's MAC
               (the IP header still carries the final destination IP)
```

This is why a wrong subnet mask breaks things in strange ways: the host thinks a remote IP is local (or vice versa) and ARPs for something that will never answer.

1.4.2 ARP exchange

```
Host A 10.0.1.10 (aa:aa..)                          Host B 10.0.1.20 (bb:bb..)
      |                                                      |
      |-- ARP Request (broadcast ff:ff:ff:ff:ff:ff) ------->|  "Who has 10.0.1.20?
      |   (every host on the segment sees this)             |   Tell 10.0.1.10"
      |                                                      |
      |<------------- ARP Reply (unicast to aa:aa..) ------|  "10.0.1.20 is at bb:bb.."
      |                                                      |
      |== IP packet in frame dst=bb:bb.. ================>|
```

Both hosts cache the result. On Linux, the cache is the **neighbor table**:

```
ip neigh show
```

```
10.0.1.1 dev ens5 lladdr 0a:12:7f:00:00:01 REACHABLE
10.0.1.20 dev ens5 lladdr 0a:9c:44:31:be:02 STALE
10.0.1.33 dev ens5  FAILED
```

- `REACHABLE`: recently confirmed.
- `STALE`: valid but will be re-verified on next use.
- `FAILED`: ARP got no answer. If you see `FAILED` for your default gateway, nothing off-subnet will work. This is a strong L2/L3 clue.

1.4.3 Gratuitous ARP and why HA setups depend on it

A **gratuitous ARP** is an unsolicited ARP reply a host sends to announce “IP X is now at MAC Y”. Keepalived/VRRP uses it when a virtual IP fails over from one server to another so that switches and neighbors update their caches immediately. In public clouds you usually cannot do this (the provider’s virtual network ignores it), which is why cloud HA uses API calls to move Elastic IPs or secondary IPs instead.

> **Warning:** ARP has no authentication. Any host on a segment can claim any IP (ARP spoofing), which enables man-in-the-middle attacks. Cloud providers block this at the hypervisor (source/destination check). On-prem, switches use Dynamic ARP Inspection. This is one of many reasons “being on the same LAN” should not imply trust.

1.4.4 IPv6 equivalent

IPv6 does not use ARP. It uses **NDP** (Neighbor Discovery Protocol), which runs over ICMPv6 and uses multicast instead of broadcast. `ip -6 neigh` shows the IPv6 neighbor cache. Part 2 covers this.

1.5 Switches

1.5.1 How a switch learns

A switch is a multi-port Layer 2 device. It builds a **MAC address table** (also called a CAM table) by watching source MACs of incoming frames:

```
Step 1: A (port 1) sends a frame to B. Switch learns "A is on port 1".
        Switch does not know B yet -> FLOODS frame out all ports except port 1.
Step 2: B (port 3) replies to A. Switch learns "B is on port 3".
        Switch knows A is on port 1 -> forwards ONLY to port 1.
Step 3: All further A<->B traffic is switched directly between ports 1 and 3.

MAC table:
  MAC                 Port   VLAN   Age
  aa:aa:aa:aa:aa:aa   1      10     12s
  bb:bb:bb:bb:bb:bb   3      10     12s
```

Entries age out (commonly 300 seconds). Broadcasts (ARP, DHCP Discover) are always flooded within the VLAN.

1.5.2 Loops and Spanning Tree

Ethernet frames have no TTL. If you connect two switches with two cables, a broadcast will loop forever and multiply, saturating the network in seconds (a **broadcast storm**). **STP** (Spanning Tree Protocol, 802.1D, and its faster successor RSTP 802.1w) detects redundant paths and blocks some ports to make the topology loop-free. Modern data centers avoid STP's limitations by using routed leaf-spine fabrics (every link is L3, using ECMP) with overlays like VXLAN/EVPN on top.

1.5.3 Linux bridges are switches

On Linux, a **bridge** is a software switch. Docker's `docker0`, libvirt's `virbr0`, and many Kubernetes CNIs use bridges. You will build one in this Part's lab.

```
ip link add br0 type bridge
ip link set br0 up
bridge fdb show br br0    # the bridge's MAC table
```

1.6 VLANs

1.6.1 Why VLANs exist

A **VLAN** (Virtual LAN, IEEE 802.1Q) splits one physical switch into multiple isolated broadcast domains. Hosts in VLAN 10 cannot talk to hosts in VLAN 20 at Layer 2; traffic between them must go through a router (or L3 switch), where it can be filtered. Reasons to use VLANs:

- **Security segmentation**: separate management, production, storage, guest Wi-Fi.
- **Smaller broadcast domains**: ARP and other broadcasts do not reach everyone.
- **Flexibility**: group hosts logically, not by which switch they are plugged into.

1.6.2 Access ports, trunk ports, and tagging

```
                Trunk (802.1Q tagged: VLAN 10, 20, 30)
   +----------+ ======================================== +----------+
   | Switch A |                                          | Switch B |
   +----------+                                          +----------+
    |access    |access                                    |access
    |VLAN 10   |VLAN 20                                   |VLAN 10
   [Web-1]   [DB-1]                                     [Web-2]

802.1Q tag (4 bytes) inserted after the source MAC:
+---------+---------+--------------------------+-----------+---------+-----+
| Dst MAC | Src MAC | 802.1Q tag               | EtherType | Payload | FCS |
|         |         | TPID 0x8100 | PRI|DEI|VID |           |         |     |
+---------+---------+--------------------------+-----------+---------+-----+
VID = 12 bits -> 4094 usable VLAN IDs (0 and 4095 reserved)
```

- **Access port**: belongs to one VLAN, frames are untagged. Hosts are unaware of VLANs.
- **Trunk port**: carries multiple VLANs; frames are tagged with the VLAN ID. Used between switches, and to hypervisors.
- **Native VLAN**: the one VLAN on a trunk that is sent untagged. Mismatched native VLANs are a classic misconfiguration (and a VLAN-hopping attack vector).

On Linux you can create a VLAN sub-interface directly:

```
ip link add link eth0 name eth0.20 type vlan id 20
ip addr add 192.168.20.5/24 dev eth0.20
ip link set eth0.20 up
```

1.6.3 VLANs vs VXLAN

The 4094 VLAN limit is far too small for multi-tenant clouds. **VXLAN** encapsulates Layer 2 frames in UDP (port 4789) with a 24-bit identifier (VNI), giving about 16 million segments, and lets L2 segments stretch over an L3 network. Kubernetes CNIs such as Flannel and Calico (in VXLAN mode) use it. The cost is 50 bytes of overhead, which is why VXLAN interfaces default to MTU 1450.

```
[Outer Eth 14][Outer IP 20][Outer UDP 8][VXLAN 8][ Inner Eth frame ... ]
              \_____________ 50 bytes of overhead _____________/
```

1.7 Putting It Together: A Packet's Journey on a LAN

Laptop `192.168.1.50/24` opens `http://192.168.1.10` (a local NAS), then `https://example.com` (remote).

```
Case 1: Local destination 192.168.1.10
  1. 192.168.1.10 is in 192.168.1.0/24 -> on-link.
  2. ARP "who has 192.168.1.10?" -> reply with NAS MAC.
  3. Frame: src=laptop MAC, dst=NAS MAC; IP: src=.50 dst=.10.
  4. Switch forwards by MAC. No router involved.

Case 2: Remote destination 93.184.215.14
  1. Not in 192.168.1.0/24 -> use default route via 192.168.1.1.
  2. ARP for 192.168.1.1 (the gateway), NOT for the remote IP.
  3. Frame: src=laptop MAC, dst=ROUTER MAC; IP: src=.50 dst=93.184.215.14.
  4. Router strips L2 header, decrements TTL, applies NAT (Part 2),
     builds a new L2 header for its upstream link, forwards.
```

> **Interview:** "What happens when you type a URL into a browser?" is the most common networking interview question. A strong answer walks the layers: DNS resolution (cache, resolver, recursion), route lookup, ARP for the gateway, TCP handshake, TLS handshake, HTTP request, response, and rendering. You will be able to answer every step in depth by the end of this course.

1.8 Lab: Build a Virtual LAN with Network Namespaces

You need a Linux machine (a VM or cloud instance is fine) with root access. Network namespaces give each "host" its own interfaces, routes, ARP table, and firewall, exactly like a container (containers *are* namespaces). This lab builds two hosts connected through a bridge.

```
   ns: red                              ns: blue
  [veth-red 10.10.0.1/24]            [veth-blue 10.10.0.2/24]
          |                                  |
     veth-red-br                        veth-blue-br
          \___________ br-lab (bridge) _______/
```

Step 1 — Create the namespaces.

```
sudo ip netns add red
sudo ip netns add blue
ip netns list
```

Step 2 — Create the bridge (our virtual switch).

```
sudo ip link add br-lab type bridge
sudo ip link set br-lab up
```

Step 3 — Create veth pairs. A veth pair is a virtual cable: whatever goes in one end comes out the other.

```
sudo ip link add veth-red type veth peer name veth-red-br
sudo ip link add veth-blue type veth peer name veth-blue-br
```

Step 4 — Plug one end into each namespace, the other into the bridge.

```
sudo ip link set veth-red netns red
sudo ip link set veth-blue netns blue
sudo ip link set veth-red-br master br-lab
sudo ip link set veth-blue-br master br-lab
sudo ip link set veth-red-br up
sudo ip link set veth-blue-br up
```

Step 5 — Assign IP addresses and bring up interfaces inside each namespace.

```
sudo ip -n red addr add 10.10.0.1/24 dev veth-red
sudo ip -n red link set veth-red up
sudo ip -n red link set lo up
sudo ip -n blue addr add 10.10.0.2/24 dev veth-blue
sudo ip -n blue link set veth-blue up
sudo ip -n blue link set lo up
```

Step 6 — Test and observe ARP.

```
sudo ip netns exec red ping -c 3 10.10.0.2
sudo ip netns exec red ip neigh
bridge fdb show br br-lab | grep -v permanent
```

```
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=0.071 ms
...
10.10.0.2 dev veth-red lladdr 6e:21:9a:4c:10:f3 REACHABLE
```

> **Note:** If ping fails, the most common cause is the host's firewall filtering bridged traffic (`br_netfilter` loaded with Docker installed sets `net.bridge.bridge-nf-call-iptables=1` and Docker's FORWARD policy is DROP). Run `sudo iptables -I FORWARD -i br-lab -j ACCEPT` for the lab.

Step 7 — Watch ARP on the wire. In one terminal run a capture, in another flush the cache and ping again.

```
sudo ip netns exec blue tcpdump -n -e -i veth-blue arp or icmp
sudo ip netns exec red ip neigh flush all
sudo ip netns exec red ping -c 1 10.10.0.2
```

```
12:01:44.101 6e:5b:..:a1 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-
has 10.10.0.2 tell 10.10.0.1, length 28
12:01:44.101 6e:21:..:f3 > 6e:5b:..:a1, ethertype ARP (0x0806), length 42: Reply 10.10.0.2
is-at 6e:21:9a:4c:10:f3, length 28
12:01:44.101 6e:5b:..:a1 > 6e:21:..:f3, ethertype IPv4 (0x0800), length 98: 10.10.0.1 >
10.10.0.2: ICMP echo request
```

Step 8 — Break it on purpose. Change blue's address to `10.10.1.2/24` and ping again from red. Observe the error (`Network is unreachable` from red, because red has no route to 10.10.1.0/24). Then set red's mask to `/16` so it thinks blue is local: now red ARPs, but blue (still /24) has no route back. This asymmetry is exactly what wrong masks do in production.

Step 9 — Clean up.

```
sudo ip netns del red
sudo ip netns del blue
sudo ip link del br-lab
```

Key takeaways

- Layers let you isolate problems: test bottom-up (link, IP, port, application).
- Each device reads headers only up to its layer; routers rewrite L2 headers at every hop, while IPs stay constant end-to-end unless NAT intervenes.
- MAC addresses are local to a link; ARP maps IPs to MACs, and for off-subnet destinations a host ARPs for its gateway, not the destination.
- Switches learn MACs from source addresses and flood unknown destinations; Linux bridges are software switches.
- VLANs split broadcast domains (12-bit ID, 4094 usable); VXLAN extends L2 over L3 with a 24-bit VNI and 50 bytes of overhead.
- MTU (1500 by default) minus headers determines MSS (1460); tunnels shrink it and cause hard-to-diagnose bugs.
- Network namespaces plus veth pairs and bridges are the building blocks of container networking.

Part 2 — IP Addressing, Subnetting, NAT, and IPv6

If there is one skill in this course you must be able to do quickly and correctly, it is subnetting. You will use it when designing a VPC, sizing a Kubernetes pod CIDR, writing a security group rule, debugging an overlapping-CIDR peering failure, or reading a route table. This Part goes from binary basics to VLSM and route summarization, with worked examples and practice problems you should actually do on paper.

2.1 IPv4 Addresses

An IPv4 address is a 32-bit number, written as four 8-bit **octets** in dotted decimal: `192.168.10.77`. Each octet ranges from 0 to 255.

```
192      .168      .10       .77
11000000  10101000  00001010  01001101
```

2.1.1 Binary in two minutes

Each bit position in an octet has a value:

```
Bit position:  128  64  32  16   8   4   2   1
Example 77:      0   1   0   0   1   1   0   1   = 64 + 8 + 4 + 1 = 77
Example 200:     1   1   0   0   1   0   0   0   = 128 + 64 + 8   = 200
Example 255:     1   1   1   1   1   1   1   1   = 255
```

Memorize these “mask octet” values; they are the only values a subnet mask octet can take:

Bits set	Binary	Decimal	Block size (256 - value)
0	00000000	0	256
1	10000000	128	128
2	11000000	192	64
3	11100000	224	32
4	11110000	240	16
5	11111000	248	8
6	11111100	252	4
7	11111110	254	2
8	11111111	255	1

2.1.2 Network part and host part

Every IP address is split into a **network portion** (identifies the subnet) and a **host portion** (identifies the device within the subnet). The **subnet mask** or **prefix length** says where the split is.

- `/24` means the first 24 bits are network bits. Mask `255.255.255.0`.
- `/26` means 26 network bits. Mask `255.255.255.192`.

In every subnet, two addresses are special:

- **Network address**: all host bits = 0. Identifies the subnet itself; not assignable.
- **Broadcast address**: all host bits = 1. Sent to every host in the subnet; not assignable.

So usable hosts = 2^(host bits) - 2. (Exceptions: `/31` for point-to-point links per RFC 3021, where both addresses are usable, and `/32` for a single host route.)

2.1.3 Classful addressing (history only)

Before 1993, addresses were divided into Class A (`/8`, first octet 1–126), B (`/16`, 128–191), C (`/24`, 192–223), D (multicast, 224–239), and E (reserved, 240–255). Classful addressing wasted enormous numbers of addresses and was replaced by **CIDR** (Classless Inter-Domain Routing), where the prefix can be any length. You will still hear “a class C” used to mean “a /24”. Know the history, but always think in CIDR.

2.2 CIDR Reference Table

Prefix	Subnet mask	Total addresses	Usable hosts	Wildcard
/8	255.0.0.0	16,777,216	16,777,214	0.255.255.255
/12	255.240.0.0	1,048,576	1,048,574	0.15.255.255
/16	255.255.0.0	65,536	65,534	0.0.255.255
/17	255.255.128.0	32,768	32,766	0.0.127.255
/18	255.255.192.0	16,384	16,382	0.0.63.255
/19	255.255.224.0	8,192	8,190	0.0.31.255
/20	255.255.240.0	4,096	4,094	0.0.15.255
/21	255.255.248.0	2,048	2,046	0.0.7.255
/22	255.255.252.0	1,024	1,022	0.0.3.255
/23	255.255.254.0	512	510	0.0.1.255
/24	255.255.255.0	256	254	0.0.0.255
/25	255.255.255.128	128	126	0.0.0.127
/26	255.255.255.192	64	62	0.0.0.63

Prefix	Subnet mask	Total addresses	Usable hosts	Wildcard
/27	255.255.255.224	32	30	0.0.0.31
/28	255.255.255.240	16	14	0.0.0.15
/29	255.255.255.248	8	6	0.0.0.7
/30	255.255.255.252	4	2	0.0.0.3
/31	255.255.255.254	2	2 (point-to-point)	0.0.0.1
/32	255.255.255.255	1	1 (host route)	0.0.0.0

> **Tip:** Each step of one bit in the prefix halves (or doubles) the size. A /24 is two /25s, four /26s, eight /27s. A /16 is 256 /24s. Anchor on "/24 = 256" and count bits from there.

> **Note:** AWS reserves **5** addresses in every subnet (network, +1 VPC router, +2 DNS, +3 reserved for future use, and the last address). A /24 subnet in AWS gives 251 usable IPs, a /28 (the smallest allowed) gives 11. Azure also reserves 5. GCP reserves 4.

2.3 The Fast Method: Block Sizes

You do not need to convert everything to binary. Use the **block size** (also called the "magic number") method:

1. Find the **interesting octet**: the octet where the mask is neither 255 nor 0.
2. Block size = 256 - (mask value in that octet).
3. Subnets start at multiples of the block size in that octet.
4. The network address is the largest multiple of the block size that is less than or equal to the IP's value in that octet.
5. Broadcast = next network address - 1.

2.3.1 Worked example 1: 192.168.10.77/26

```
Mask /26 = 255.255.255.192         interesting octet = 4th, mask value = 192
Block size = 256 − 192 = 64        subnets: .0, .64, .128, .192
77 falls in the block starting at 64

Network:     192.168.10.64
First host:  192.168.10.65
Last host:   192.168.10.126
Broadcast:   192.168.10.127
Usable:      2^6 − 2 = 62
```

Verify in binary:

```
IP      77  = 01001101
Mask   192  = 11000000
AND         = 01000000 = 64   -> network .64
Host bits all 1: 01111111 = 127 -> broadcast .127
```

2.3.2 Worked example 2: 10.20.30.77/27

```
Mask /27 = 255.255.255.224         block = 256 - 224 = 32
Blocks: 0, 32, 64, 96, ...         77 is in [64, 95]

Network:    10.20.30.64
Hosts:      10.20.30.65 - 10.20.30.94
Broadcast:  10.20.30.95
Usable:     30
```

2.3.3 Worked example 3: 172.16.45.200/20

Here the interesting octet is the **third**.

```
Mask /20 = 255.255.240.0           interesting octet = 3rd, value 240
Block = 256 - 240 = 16             third-octet blocks: 0,16,32,48,...
45 falls in [32, 47]

Network:    172.16.32.0
First host: 172.16.32.1
Last host:  172.16.47.254
Broadcast:  172.16.47.255
Usable:     2^12 - 2 = 4,094
```

Note that `172.16.40.0` and `172.16.45.255` are perfectly normal host addresses inside this /20. Only the very first and very last addresses of the whole block are special.

2.3.4 Worked example 4: 10.200.37.5/13

```
Mask /13 = 255.248.0.0             interesting octet = 2nd, value 248
Block = 256 - 248 = 8              second-octet blocks: 0,8,...,192,200,208
200 is itself a multiple of 8 -> block [200, 207]

Network:    10.200.0.0
Broadcast:  10.207.255.255
Usable:     2^19 - 2 = 524,286
```

2.3.5 Worked example 5: Is 10.1.5.130 inside 10.1.4.0/23?

```
/23 = 255.255.254.0, block 2 in the 3rd octet: 0,2,4,6,...
10.1.4.0/23 spans 10.1.4.0 - 10.1.5.255
```

```
10.1.5.130 is within that range -> YES
```

This is exactly the question you answer when reading a security group rule like “allow 10.1.4.0/23”.

2.4 Dividing Networks: Subnetting and VLSM

2.4.1 Equal-size subnets

To split a network into 2^n equal subnets, borrow n bits.

Example: split 10.0.0.0/16 into /20s. Borrowing 4 bits gives 2^4 = 16 subnets of 4,096 addresses each. The third octet increments by 16:

```
10.0.0.0/20     10.0.64.0/20    10.0.128.0/20   10.0.192.0/20
10.0.16.0/20    10.0.80.0/20    10.0.144.0/20   10.0.208.0/20
10.0.32.0/20    10.0.96.0/20    10.0.160.0/20   10.0.224.0/20
10.0.48.0/20    10.0.112.0/20   10.0.176.0/20   10.0.240.0/20
```

Example: split 192.168.0.0/24 into at least 6 subnets. 2^2 = 4 is too few, 2^3 = 8 is enough, so borrow 3 bits: /27, eight subnets of 30 usable hosts: `.0, .32, .64, .96, .128, .160, .192, .224`.

2.4.2 VLSM: different sizes from one block

VLSM (Variable Length Subnet Masking) carves different-sized subnets from one block. The rule: **allocate the largest subnet first**, each aligned on its own block boundary. This prevents fragmentation and overlap.

Requirement: from 192.168.1.0/24 allocate a 100-host LAN, a 50-host LAN, a 20-host LAN, and two point-to-point router links.

Need	Smallest fit	Subnet	Range	Usable
100 hosts	/25 (126)	192.168.1.0/25	.0 – .127	126
50 hosts	/26 (62)	192.168.1.128/26	.128 – .191	62
20 hosts	/27 (30)	192.168.1.192/27	.192 – .223	30
Link 1	/30 (2)	192.168.1.224/30	.224 – .227	2
Link 2	/30 (2)	192.168.1.228/30	.228 – .231	2
Free		192.168.1.232/29, .240/28	.232 – .255	spare

> **Warning:** A subnet must start on a multiple of its own size. `192.168.1.100/26` is not a valid network address (100 is not a multiple of 64); the network that contains .100 is `192.168.1.64/26`. Tools like Terraform will error on misaligned CIDRs, and some will silently “correct” them, which is worse.

2.4.3 A realistic VPC plan

You are given `10.0.0.0/16` for a production VPC spanning three availability zones. Plan for growth: Kubernetes nodes in private subnets need many IPs (with AWS VPC CNI, every pod gets a VPC IP).

```
VPC 10.0.0.0/16 (65,536 addresses)

Private (app/EKS)  10.0.0.0/20   AZ-a   4,096 addresses
                   10.0.16.0/20  AZ-b
                   10.0.32.0/20  AZ-c
Public (ALB/NAT)   10.0.48.0/24  AZ-a     256 addresses
                   10.0.49.0/24  AZ-b
                   10.0.50.0/24  AZ-c
Database           10.0.64.0/24  AZ-a
                   10.0.65.0/24  AZ-b
                   10.0.66.0/24  AZ-c
Reserved for future 10.0.128.0/17 (half the VPC kept free)
```

Why this shape: public subnets only hold load balancers and NAT gateways (small), private subnets hold workloads (large), and leaving half the space free means you can add a fourth AZ or a new tier without re-IPing.

2.5 Route Summarization (Supernetting)

Summarization combines contiguous networks into one shorter prefix, which keeps route tables small and lets one firewall rule cover many subnets.

Example: summarize 192.168.16.0/24 through 192.168.19.0/24.

```
16 = 000100 00
17 = 000100 01
18 = 000100 10
19 = 000100 11
     ^^^^^^ first 6 bits of the 3rd octet are common
16 + 6 = 22 common bits -> 192.168.16.0/22
Check: /22 block is 4 in the 3rd octet; 16 is a multiple of 4. Range 16.0 - 19.255.
```

Example: summarize 10.4.0.0/16 through 10.7.0.0/16. Four /16s, block 4 in the second octet, 4 is a multiple of 4: `10.4.0.0/14`.

Counter-example: 192.168.17.0/24 through 192.168.20.0/24 cannot be summarized into a single /22, because 17 is not a multiple of 4. The smallest single prefix covering them is 192.168.16.0/21, which also includes 16 and 21–23. That over-coverage might be fine for a route but dangerous for a firewall “allow” rule.

2.6 Subnetting Practice Problems

Work these on paper before checking the answers.

1. Find network, broadcast, host range, and usable count for `192.168.5.130/26`.
2. Same for `10.10.10.10/29`.
3. Same for `172.31.100.5/18`.
4. How many /26 subnets fit in a /22?
5. What is the smallest prefix that holds 500 hosts (non-cloud)?
6. What is the dotted-decimal mask for /21?
7. Is `10.0.15.255` a usable host address in `10.0.8.0/21`?
8. Summarize `10.4.0.0/16`, `10.5.0.0/16`, `10.6.0.0/16`, `10.7.0.0/16`.
9. Do `10.0.0.0/16` and `10.0.128.0/17` overlap?
10. Find network, broadcast and host range for `192.168.100.33/28`.
11. What is the wildcard mask for /20?
12. How many /64 subnets are in an IPv6 /48?
13. From `10.50.0.0/16` you need subnets with at least 1,000 hosts each. What prefix, and how many subnets?
14. What is the 5th /26 subnet in `172.20.0.0/22` (counting the first as the 1st)?
15. How many usable IPs does an AWS subnet of /27 provide?
16. A host is `10.8.3.7` with mask `255.255.252.0`. What is its network in CIDR notation?

2.6.1 Answers

#	Answer	Working
1	Net 192.168.5.128, bcast .191, hosts .129–.190, 62	Block 64; 130 in [128,191]
2	Net 10.10.10.8, bcast .15, hosts .9–.14, 6	Block 8; 10 in [8,15]
3	Net 172.31.64.0, bcast 172.31.127.255, hosts 172.31.64.1–172.31.127.254, 16,382	/18 = 255.255.192.0, block 64 in 3rd octet; 100 in [64,127]
4	16	2^(26-22) = 16
5	/23	/23 = 510 usable; /24 = 254 is too small
6	255.255.248.0	5 bits in 3rd octet = 248
7	No, it is the broadcast	/21 block 8: 10.0.8.0–10.0.15.255
8	10.4.0.0/14	4 contiguous /16s, aligned on 4
9	Yes	10.0.128.0/17 is the upper half of 10.0.0.0/16
10	Net .32, bcast .47, hosts .33–.46 (14)	Block 16; 33 in [32,47]
11	0.0.15.255	255.255.255.255 - 255.255.240.0
12	65,536	2^(64-48) = 2^16
13	/22, 64 subnets	/22 = 1,022 usable; 2^(22-16) = 64

#	Answer	Working
14	172.20.1.0/26	.0.0, .0.64, .0.128, .0.192, then .1.0
15	27	32 - 5 reserved
16	10.8.0.0/22	/22 block 4 in 3rd octet; 3 in [0,3]

2.6.2 Let a tool check you, but know the math

```
python3 -c "import ipaddress as i; n=i.ip_interface('172.31.100.5/18').network; print(n,
n.broadcast_address, n.num_addresses)"
```

```
172.31.64.0/18 172.31.127.255 16384
```

```
# Overlap check, useful before VPC peering
python3 -c "import ipaddress as i;
print(i.ip_network('10.0.0.0/16').overlaps(i.ip_network('10.0.128.0/17')))"
# Split into subnets
python3 -c "import ipaddress as i; [print(s) for s in
i.ip_network('10.0.0.0/22').subnets(new_prefix=24)]"
```

```
True
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
```

Terraform has the same capability built in, which you will use in Part 7:

```
# cidrsubnet(prefix, newbits, netnum)
cidrsubnet("10.0.0.0/16", 4, 2)   # => "10.0.32.0/20"
cidrsubnet("10.0.0.0/16", 8, 48)  # => "10.0.48.0/24"
```

2.7 Special and Private Address Ranges

Range	Purpose	Notes
10.0.0.0/8	Private (RFC 1918)	16.7M addresses, typical for VPCs
172.16.0.0/12	Private (RFC 1918)	172.16.0.0 – 172.31.255.255
192.168.0.0/16	Private (RFC 1918)	Home and small office
100.64.0.0/10	Carrier-grade NAT (RFC 6598)	Also used by Tailscale and as EKS secondary CIDR

Range	Purpose	Notes
127.0.0.0/8	Loopback	Never leaves the host
169.254.0.0/16	Link-local	Cloud metadata at 169.254.169.254
0.0.0.0/8	“This network”	0.0.0.0 as bind address means “all interfaces”
224.0.0.0/4	Multicast	
240.0.0.0/4	Reserved	255.255.255.255 is limited broadcast
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation (TEST-NET)	Safe to use in examples
198.18.0.0/15	Benchmarking	

Warning: Docker’s default bridge uses `172.17.0.0/16` and new networks take further ranges from 172.17–172.31 and 192.168.x. If your corporate VPN or a peered VPC uses 172.16.0.0/12, containers may become unable to reach those networks because the host now has a more specific local route. Set `default-address-pools` in `/etc/docker/daemon.json` to a range you know is unused.

Tip: Plan non-overlapping CIDRs across *all* environments and accounts from day one (an IPAM spreadsheet or AWS VPC IPAM). Overlapping VPC CIDRs cannot be peered or attached to the same Transit Gateway route table without NAT tricks. Re-IPing a production VPC is one of the most expensive mistakes in cloud networking.

2.8 NAT and PAT

2.8.1 Why NAT exists

There are only about 4.3 billion IPv4 addresses, and they ran out at the regional registries between 2011 and 2019. **NAT** (Network Address Translation) lets many private hosts share one or a few public addresses. It is also used to connect networks with overlapping address spaces.

2.8.2 Types of NAT

Type	What it rewrites	Example
SNAT (Source NAT)	Source IP on outbound	Private instance reaches the Internet via NAT gateway
PAT / NAPT / Masquerade	Source IP **and** source port, many-to-one	Home router, AWS NAT gateway, Docker MASQUERADE
DNAT (Destination NAT)	Destination IP/port on inbound	Port forwarding, `docker run -p 8080:80`, Kubernetes Services

Type	What it rewrites	Example
1:1 static NAT	One public IP maps to one private IP	AWS Elastic IP on an instance (done by the IGW)

2.8.3 How PAT works

```
Private side                    NAT device                      Internet
10.0.1.10:51000 --+         public IP 203.0.113.5
                  |---> [translation table] ----> 203.0.113.5:40001 -> 93.184.215.14:443
10.0.1.11:51000 --+                         ----> 203.0.113.5:40002 -> 93.184.215.14:443

Translation table (conntrack):
inside src          outside src          destination
10.0.1.10:51000  ->  203.0.113.5:40001    93.184.215.14:443
10.0.1.11:51000  ->  203.0.113.5:40002    93.184.215.14:443
```

The reply comes back to `203.0.113.5:40001`; the NAT device looks up the table and rewrites the destination back to `10.0.1.10:51000`. Key consequences:

- **Inbound connections cannot be initiated** to hosts behind PAT unless you configure DNAT. That is an accidental security property, not a firewall.
- **State is required.** If the NAT device restarts or the entry times out, long-idle connections silently die. This is why idle database connections through a NAT gateway break after 350 seconds on AWS (the NAT gateway idle timeout). Use TCP keepalives shorter than that.
- **Port exhaustion.** Each public IP has about 64,000 source ports per destination IP and port. An AWS NAT gateway supports about 55,000 simultaneous connections to each unique destination per public IP; services that hammer one external API can hit `ErrorPortAllocation`. Fix by adding IPs to the NAT gateway, using connection pooling, or using VPC endpoints for AWS services.

On Linux, NAT is implemented by netfilter and tracked by conntrack:

```
# Masquerade traffic leaving eth0 from the 10.10.0.0/24 network
sudo iptables -t nat -A POSTROUTING -s 10.10.0.0/24 -o eth0 -j MASQUERADE
# Port-forward host port 8080 to an internal host
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 10.10.0.2:80
# Inspect tracked connections
sudo conntrack -L -p tcp --dport 443 | head
```

2.9 IPv6 Essentials

2.9.1 Why you need to know it

IPv6 is no longer optional: mobile networks are majority IPv6, AWS charges for every public IPv4 address (since February 2024), and IPv6-only subnets and EKS IPv6 clusters remove IP exhaustion problems for pods. Every DevOps engineer should be able to read and configure it.

2.9.2 Address format

An IPv6 address is 128 bits written as eight groups of four hex digits:

```
Full:        2001:0db8:0000:0000:0000:ff00:0042:8329
Rule 1 – drop leading zeros in each group:   2001:db8:0:0:0:ff00:42:8329
Rule 2 – replace ONE run of all-zero groups with "::"  2001:db8::ff00:42:8329

Loopback:    ::1
Unspecified: ::
```

`::` can appear only once in an address; otherwise it would be ambiguous. In URLs, wrap IPv6 in brackets: `http://[2001:db8::1]:8080/`.

2.9.3 Address types

Prefix	Type	Analogy in IPv4
2000::/3	Global unicast (GUA)	Public addresses
fc00::/7 (fd00::/8 in practice)	Unique local (ULA)	RFC 1918 private
fe80::/10	Link-local, every interface has one	169.254.0.0/16
ff00::/8	Multicast (no broadcast in IPv6)	224.0.0.0/4
::1/128	Loopback	127.0.0.1
64:ff9b::/96	NAT64 well-known prefix	—

2.9.4 Subnetting IPv6

IPv6 subnetting is simpler because you never run out of host bits:

- A **/64** is the standard subnet size for any LAN (required by SLAAC). It holds 2^64 addresses.
- Sites typically receive a **/48** (65,536 /64s) or **/56** (256 /64s).
- AWS assigns a VPC a /56 and each subnet a /64.

Subnetting happens on nibble (4-bit, one hex digit) boundaries for readability: from `2001:db8:abcd::/48`, subnets are `2001:db8:abcd:0000::/64`, `2001:db8:abcd:0001::/64`, … `2001:db8:abcd:ffff::/64`.

2.9.5 NDP, SLAAC, and DHCPv6

IPv6 replaces ARP and broadcast with **Neighbor Discovery Protocol** (ICMPv6):

Message	Purpose
Router Solicitation (RS)	Host asks “any routers here?”
Router Advertisement (RA)	Router announces prefix, default gateway, flags

Message	Purpose
Neighbor Solicitation (NS)	Equivalent of ARP request (sent to a solicited-node multicast address)
Neighbor Advertisement (NA)	Equivalent of ARP reply
Redirect	Better next hop available

- **SLAAC** (Stateless Address Autoconfiguration): the host takes the /64 prefix from the RA and generates its own interface ID (randomized for privacy on clients).
- **DHCPv6**: stateful assignment, used when you need central control. RA flags tell hosts which method to use.

> **Warning:** Never blanket-block ICMPv6. NDP, router discovery, and Path MTU Discovery all depend on it. In IPv6, routers do not fragment packets, so a blocked "Packet Too Big" message causes connections to hang.

2.9.6 IPv6 and NAT

IPv6 is designed for end-to-end addressing, so NAT is generally not used. Security comes from stateful firewalls instead. AWS provides an **egress-only Internet gateway** that allows outbound IPv6 connections but blocks inbound, giving the same effect as a NAT gateway without translation. **NAT64 + DNS64** let IPv6-only workloads reach IPv4-only services.

```
ip -6 addr show dev ens5
ip -6 route show
ping -6 -c 2 ipv6.google.com
```

```
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 state UP qlen 1000
    inet6 2600:1f18:4b2:9e00:7c1a:3a5f:91b2:4e10/128 scope global dynamic noprefixroute
       valid_lft 447sec preferred_lft 137sec
    inet6 fe80::8ff:a1ff:fe2c:7d11/64 scope link
       valid_lft forever preferred_lft forever
```

2.10 Lab: Subnet Calculator Drills and NAT in Namespaces

Part A — Calculator drills.

Step 1. Write this small helper to `subnet.py`:

```
cat > subnet.py <<'EOF'
import ipaddress, sys
iface = ipaddress.ip_interface(sys.argv[1])
n = iface.network
hosts = list(n.hosts()) if n.num_addresses <= 65536 else None
print(f"network   {n.network_address}/{n.prefixlen}")
```

```
print(f"netmask   {n.netmask}   wildcard {n.hostmask}")
print(f"broadcast {n.broadcast_address}")
if hosts:
    print(f"hosts     {hosts[0]} - {hosts[-1]} ({len(hosts)} usable)")
else:
    print(f"usable    {n.num_addresses - 2}")
EOF
```

Step 2. Solve each practice problem by hand, then check:

```
python3 subnet.py 192.168.100.33/28
```

```
network   192.168.100.32/28
netmask   255.255.255.240   wildcard 0.0.0.15
broadcast 192.168.100.47
hosts     192.168.100.33 - 192.168.100.46 (14 usable)
```

Part B — Build a NAT router with namespaces.

```
  ns: lan (10.10.0.2/24) --veth-- ns: router (10.10.0.1 | 192.0.2.1) --veth-- ns: wan
(192.0.2.2/24)
```

Step 3. Create namespaces and links:

```
for ns in lan router wan; do sudo ip netns add $ns; done
sudo ip link add v-lan type veth peer name v-rlan
sudo ip link add v-wan type veth peer name v-rwan
sudo ip link set v-lan netns lan;  sudo ip link set v-rlan netns router
sudo ip link set v-wan netns wan;  sudo ip link set v-rwan netns router
sudo ip -n lan addr add 10.10.0.2/24 dev v-lan
sudo ip -n router addr add 10.10.0.1/24 dev v-rlan
sudo ip -n router addr add 192.0.2.1/24 dev v-rwan
sudo ip -n wan addr add 192.0.2.2/24 dev v-wan
for pair in "lan v-lan" "router v-rlan" "router v-rwan" "wan v-wan"; do
  set -- $pair; sudo ip -n $1 link set $2 up; sudo ip -n $1 link set lo up
done
```

Step 4. Give the LAN host a default route, enable forwarding on the router:

```
sudo ip -n lan route add default via 10.10.0.1
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1
```

Step 5. Ping from LAN to WAN. It fails: the `wan` namespace has no route back to 10.10.0.0/24 (just like the real Internet has no route to your private network).

```
sudo ip netns exec lan ping -c 2 -W 1 192.0.2.2
```

Step 6. Add masquerading on the router and retry:

```
sudo ip netns exec router iptables -t nat -A POSTROUTING -o v-rwan -j MASQUERADE
sudo ip netns exec lan ping -c 2 192.0.2.2
sudo ip netns exec wan tcpdump -n -i v-wan icmp -c 2
```

```
13:10:02.551 IP 192.0.2.1 > 192.0.2.2: ICMP echo request, id 4121, seq 1, length 64
13:10:02.551 IP 192.0.2.2 > 192.0.2.1: ICMP echo reply, id 4121, seq 1, length 64
```

The WAN side sees only the router's address. Inspect the translation with `sudo ip netns exec router conntrack -L` (install `conntrack-tools` if needed).

Step 7. Clean up: `for ns in lan router wan; do sudo ip netns del $ns; done`.

Key takeaways

- An IP address splits into network and host bits; the prefix length defines the split. Usable hosts = 2^host-bits - 2 (except /31 and /32).
- Use block size (256 - mask octet) to find network and broadcast addresses quickly; verify with binary when unsure.
- VLSM: allocate largest subnets first, always aligned to their own size.
- Summarization requires contiguous, aligned blocks; over-summarizing in firewall rules grants more access than intended.
- Plan non-overlapping CIDRs across every VPC, VPN, and container network before you build anything. AWS reserves 5 IPs per subnet.
- PAT lets many private hosts share one public IP via a stateful port mapping table; watch for idle timeouts and port exhaustion.
- IPv6 uses /64 subnets, NDP instead of ARP, SLAAC/DHCPv6, and firewalls instead of NAT. Never block ICMPv6 wholesale.

Part 3 — Routing, Transport (TCP and UDP), Ports, Sockets, and DHCP

Part 2 gave every host an address. This Part answers two questions: how does a packet find its way to a destination network (routing), and how does data get reliably from one *process* to another (transport)? Most production incidents you will debug, from connection timeouts to 502s to “too many open files”, live in the transport layer, so we go deep on TCP.

3.1 Routing Fundamentals

3.1.1 The route table

Every host, not just routers, has a **route table**: a list of destination prefixes and where to send packets for each. When sending a packet, the kernel finds all routes whose prefix contains the destination and picks the **longest prefix match** (the most specific route).

```
ip route show
```

```
default via 10.0.1.1 dev ens5 proto dhcp src 10.0.1.25 metric 100
10.0.1.0/24 dev ens5 proto kernel scope link src 10.0.1.25
10.100.0.0/16 via 10.0.1.250 dev ens5
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
```

Read each line:

Route	Meaning
`default via 10.0.1.1`	0.0.0.0/0: anything not matched elsewhere goes to the gateway 10.0.1.1
`10.0.1.0/24 dev ens5 scope link`	Directly connected: deliver on-link, ARP for the destination itself
`10.100.0.0/16 via 10.0.1.250`	Static route: send to a specific next-hop router (e.g., a VPN appliance)
`172.17.0.0/16 dev docker0 linkdown`	Connected to docker0, currently no containers

Ask the kernel which route it would actually use; this is far more reliable than reading the table yourself:

```
ip route get 10.100.4.7
```

```
10.100.4.7 via 10.0.1.250 dev ens5 src 10.0.1.25 uid 1000
    cache
```

3.1.2 Longest prefix match example

```
Routes:   0.0.0.0/0      -> IGW
          10.0.0.0/8     -> Transit Gateway
          10.20.0.0/16   -> VPC peering
          10.20.5.0/24   -> local

Destination 10.20.5.9  -> matches all four; /24 is longest -> local
Destination 10.20.9.1  -> matches /0, /8, /16; /16 wins     -> peering
Destination 10.99.0.1  -> matches /0, /8;  /8 wins          -> Transit Gateway
Destination 8.8.8.8    -> only /0                          -> IGW
```

Longest prefix match is the rule in Linux, in AWS route tables, in Kubernetes nodes, and on Internet routers. When two routes have the *same* prefix length, metrics (administrative distance or route priority) decide.

3.1.3 The default gateway

The default gateway is just the next hop for `0.0.0.0/0`. It must be on a directly connected subnet, because the host must be able to ARP for it. A host with a correct IP but no default route can talk to its own subnet but nothing else, a classic “I can ping my neighbor but not the Internet” symptom.

3.1.4 What a router does with a packet

```
1. Receive frame; verify FCS; strip L2 header.
2. Check IP header checksum; decrement TTL. If TTL hits 0 -> drop,
   send ICMP Time Exceeded back to the source (this is how traceroute works).
3. Longest-prefix lookup on destination IP -> next hop + egress interface.
4. ARP (or neighbor cache) for next hop MAC.
5. Build new L2 header (src = router's egress MAC, dst = next hop MAC).
6. If packet > egress MTU and DF bit set -> drop, send ICMP "Fragmentation Needed".
7. Transmit.
```

To make a Linux machine route between interfaces you must enable forwarding: `sysctl -w net.ipv4.ip_forward=1`. Kubernetes nodes and Docker hosts have this enabled.

3.1.5 Asymmetric routing and reverse path filtering

Routing decisions are made independently in each direction. If the request goes A -> R1 -> B but the reply goes B -> R2 -> A, stateful firewalls on either path will see only half the conversation and drop it. Linux’s `rp_filter` (strict mode) drops packets that arrive on an interface the kernel would not use to reach the source. Multi-homed hosts (two NICs in different subnets) commonly hit this: you need **policy routing** (`ip rule`, multiple route tables) so that replies leave via the interface they arrived on.

```
# Replies from 10.0.2.50 use table 100, which has its own default route
ip route add default via 10.0.2.1 dev ens6 table 100
ip rule add from 10.0.2.50 table 100
ip rule show
```

3.2 Dynamic Routing Protocols

Static routes do not scale and do not react to failures. Dynamic routing protocols let routers exchange reachability information automatically.

3.2.1 Interior vs exterior

Category	Protocol	Algorithm	Where you see it
IGP	OSPF	Link-state (Dijkstra shortest path)	Enterprise and data center networks
IGP	IS-IS	Link-state	Large ISPs, some DC fabrics
IGP	RIP	Distance vector (hop count)	Legacy only
EGP	BGP	Path vector	The Internet, cloud hybrid connectivity, Kubernetes (Calico, MetalLB, Cilium)

3.2.2 OSPF in one page

- Every router floods **Link State Advertisements** describing its links and costs.
- Every router builds an identical map of the topology (the LSDB) and runs Dijkstra to compute the shortest path to every prefix.
- Cost is usually derived from bandwidth. Equal-cost paths are load-balanced (**ECMP**).
- Networks are split into **areas** (area 0 is the backbone) to limit flooding.
- Converges in seconds after a link failure, faster with BFD.

3.2.3 BGP in one page

BGP (Border Gateway Protocol) is how the Internet's ~75,000+ **Autonomous Systems** (each identified by an ASN) exchange routes. Unlike OSPF, BGP does not compute shortest paths; it applies **policy**.

- BGP peers form a TCP session on **port 179** and exchange prefixes with attributes.
- **AS_PATH** lists the ASes a route has traversed. It prevents loops and serves as a basic distance metric.
- Path selection order (simplified): highest LOCAL_PREF, shortest AS_PATH, lowest origin type, lowest MED, eBGP over iBGP, lowest IGP cost to next hop, then tie-breakers.
- **eBGP** runs between different ASes; **iBGP** runs within one AS.

```
   AS 64500 (your DC)            AS 16509 (AWS)
[Router] <==== eBGP TCP/179 ====> [Direct Connect / VPN endpoint]
advertises 10.50.0.0/16             advertises 10.0.0.0/16 (VPC)
```

Where a DevOps engineer meets BGP:

- **AWS Direct Connect and Site-to-Site VPN** use BGP to exchange routes between on-prem and the VPC/Transit Gateway.
- **Kubernetes**: Calico peers with top-of-rack switches to advertise pod CIDRs; MetalLB in BGP mode and Cilium BGP Control Plane advertise LoadBalancer IPs.
- **Anycast**: CDNs and DNS providers announce the same prefix from many locations; BGP sends you to the nearest one.
- **Outages**: route leaks and hijacks (where an AS announces prefixes it does not own) have taken down major services. RPKI signs route origins to mitigate this.

> **Interview:** "What is the difference between OSPF and BGP?" OSPF is a link-state IGP that finds the shortest path inside one organization. BGP is a path-vector EGP that exchanges routes between organizations based on policy, over TCP 179. OSPF optimizes for cost; BGP optimizes for business policy and scale.

3.3 The Transport Layer: Ports and Sockets

IP delivers packets to a *host*. The transport layer delivers data to a *process*, identified by a **port** (a 16-bit number, 0–65535).

Range	Name	Notes
0–1023	Well-known / system ports	Binding requires root or `CAP_NET_BIND_SERVICE` on Linux
1024–49151	Registered ports	e.g., 3306 MySQL, 5432 PostgreSQL, 6443 Kubernetes API
49152–65535	Dynamic / ephemeral (IANA)	Linux default ephemeral range is 32768–60999

A **socket** is an endpoint: IP + port + protocol. A TCP **connection** is uniquely identified by the **5-tuple**:

```
(protocol, source IP, source port, destination IP, destination port)
(TCP,      10.0.1.25, 51544,       10.0.2.40,      5432)
```

This is why one server port (443) can serve millions of clients: each connection has a different client IP/port. It is also why a single client can open only about 28,000 connections (the Linux ephemeral range) to the *same* destination IP:port from one source IP, which matters for proxies and NAT.

```
cat /proc/sys/net/ipv4/ip_local_port_range
ss -tlnp            # listening TCP sockets with process
```

```
ss -tunap           # all TCP and UDP sockets
ss -tan state established '( dport = :5432 )'
```

```
State   Recv-Q  Send-Q   Local Address:Port    Peer Address:Port  Process
LISTEN  0       511            0.0.0.0:80           0.0.0.0:*      users:
(("nginx",pid=1201,fd=6))
LISTEN  0       4096         127.0.0.1:5432         0.0.0.0:*      users:
(("postgres",pid=900,fd=5))
LISTEN  0       4096                 *:9100               *:*      users:
(("node_exporter",pid=777,fd=3))
```

> **Warning:** `127.0.0.1:5432` means PostgreSQL only accepts connections from the same host. If a remote client gets "connection refused", the bind address is the first thing to check. `0.0.0.0` (or `*`, or `[::]`) means all interfaces. Inside containers, an app bound to 127.0.0.1 is unreachable through `docker run -p`, which is a very common bug.

For LISTEN sockets, `Recv-Q` is the current accept queue length and `Send-Q` is the backlog limit. If `Recv-Q` approaches `Send-Q`, the application is not calling `accept()` fast enough and new connections will be dropped.

3.4 TCP in Depth

TCP (Transmission Control Protocol) provides a reliable, ordered, byte-stream connection with flow control and congestion control, on top of unreliable IP.

3.4.1 The TCP header (fields that matter)

```
 0                   15 16                  31
+---------------------+----------------------+
|     Source Port     |   Destination Port   |
+---------------------+----------------------+
|               Sequence Number              |
+--------------------------------------------+
|           Acknowledgment Number            |
+------+------+--------+----------------------+
|Offset| Rsvd | Flags  |     Window Size      |
+------+------+--------+----------------------+
|      Checksum       |   Urgent Pointer     |
+---------------------+----------------------+
|          Options (MSS, SACK, WScale, TS)   |
+--------------------------------------------+
Flags: SYN, ACK, FIN, RST, PSH, URG, ECE, CWR
```

3.4.2 The three-way handshake

```
Client                                              Server (LISTEN)
  |---- SYN  seq=1000, MSS=1460, WScale=7, SACK --->|
  |                                                  |  (half-open, in SYN queue)
  |<--- SYN-ACK seq=5000 ack=1001, MSS=8961 --------|
  |                                                  |
  |---- ACK  seq=1001 ack=5001 -------------------->|  (moves to accept queue)
  |                                                  |
  ESTABLISHED                                    ESTABLISHED
```

Why three steps: both sides must choose an initial sequence number (ISN) and confirm the other side received it. Options negotiated here (MSS, window scaling, SACK, timestamps) cannot be changed later.

Cost: a new TCP connection costs one round trip before any data flows. Add TLS 1.3 and it is two; TLS 1.2 is three. Across regions with 80 ms RTT, a fresh HTTPS connection costs 160–240 ms before the first byte of the request. This is the core reason connection pooling and keep-alive matter so much.

What failure looks like:

Symptom	Meaning
SYN sent, RST received immediately	Host reachable, nothing listening on that port (“Connection refused”)
SYN sent, no reply, retransmits	Packet dropped by firewall/SG/NACL, or host down, or routing broken (“Connection timed out”)
SYN sent, ICMP unreachable received	A router or firewall actively rejecting (“No route to host”)

> **Tip:** “Refused” versus “timed out” is the most valuable single clue in network debugging. Refused proves the network path works and the host is up; the problem is the service. Timeout points to a firewall, security group, routing, or the host being down.

Linux retries SYN `net.ipv4.tcp_syn_retries` times (default 6) with exponential backoff (1, 2, 4, 8, 16, 32 s), so a connection to a black-holed port takes about 127 seconds to fail. Applications should set their own connect timeouts.

3.4.3 Connection teardown

```
Active closer (A)                                  Passive closer (B)
ESTABLISHED                                        ESTABLISHED
  |---- FIN ------------------------------------->|
FIN_WAIT_1                                       CLOSE_WAIT  (app must call close())
  |<--- ACK --------------------------------------|
FIN_WAIT_2                                         |
  |<--- FIN --------------------------------------| LAST_ACK
TIME_WAIT                                          |
```

```
  |---- ACK ----------------------------------->| CLOSED
  (wait 2*MSL; 60 s on Linux)
CLOSED
```

A connection can also be killed instantly with **RST** (reset), for example when an application crashes, a load balancer idle-times out a connection, or a firewall rejects it. “Connection reset by peer” means you received an RST.

3.4.4 TCP states you will see in `ss`

State	Meaning	When it is a problem
LISTEN	Waiting for connections	—
SYN-SENT	Client sent SYN, waiting	Many = destination unreachable or filtered
SYN-RECV	Server got SYN, sent SYN-ACK	Many = SYN flood or clients cannot complete handshake
ESTABLISHED	Data flowing	Huge counts may indicate connection leaks
FIN-WAIT-2	We closed, waiting for peer’s FIN	Many = peer not closing
CLOSE-WAIT	Peer closed, **our app has not called close()**	Almost always an application bug (leaked sockets)
TIME-WAIT	We closed first; waiting to absorb stray packets	Tens of thousands on a client may exhaust ports

```
ss -tan | awk 'NR>1 {print $1}' | sort | uniq -c | sort -rn
```

```
  18234 TIME-WAIT
   1210 ESTAB
    412 CLOSE-WAIT
      9 LISTEN
```

That output tells a story: lots of short-lived outbound connections (TIME-WAIT), and an application leaking sockets (412 CLOSE-WAIT that will never clear by themselves).

3.4.5 TIME_WAIT: why it exists and how to handle it

TIME_WAIT exists for two reasons: (1) if the final ACK is lost, the peer will resend its FIN, and we must still be able to respond; (2) delayed packets from the old connection must not be accepted by a new connection reusing the same 5-tuple. It lasts 2 x MSL; Linux hardcodes 60 seconds.

TIME_WAIT sits on the side that **closes first**. Problems appear when a client (often a proxy or a service calling an API) opens and closes thousands of connections per second to the same destination: each consumes an ephemeral port for 60 seconds. At 28,000 ports / 60 s you cap out around 470 new connections per second to one destination.

Fixes, in order of preference:

1. **Reuse connections** (HTTP keep-alive, connection pools, upstream keepalive in Nginx). This removes the problem instead of tuning around it.
2. `net.ipv4.tcp_tw_reuse=1` allows reusing TIME_WAIT sockets for new *outbound* connections when safe (uses timestamps). Safe and widely used.
3. Widen `net.ipv4.ip_local_port_range`, or add source IPs.
4. Do **not** use `tcp_tw_recycle`; it broke clients behind NAT and was removed in Linux 4.12.

3.4.6 Flow control: the receive window

Each side advertises a **receive window** (rwnd): how many bytes it can buffer. The sender must not have more unacknowledged data in flight than the window allows. The 16-bit window field maxes at 65,535 bytes, so the **window scale** option multiplies it (up to 1 GiB).

The maximum throughput of a single TCP connection is bounded by:

```
Throughput <= Window / RTT

Example: 64 KB window, 100 ms RTT  -> 65,535 B / 0.1 s = 655 KB/s  (~5.2 Mbit/s)
Example: 4 MB window,  100 ms RTT  -> 4,194,304 / 0.1  = 42 MB/s   (~335 Mbit/s)

Bandwidth-delay product (BDP) = bandwidth x RTT
1 Gbit/s x 100 ms = 125,000,000 B/s x 0.1 s = 12.5 MB of buffer needed to fill the pipe
```

This explains why cross-region transfers are slow even on fat links: buffers too small for the BDP. Linux autotunes buffers within `net.ipv4.tcp_rmem` / `tcp_wmem`. A zero window ("TCP ZeroWindow" in Wireshark) means the receiving application is not reading data fast enough.

3.4.7 Congestion control

The receive window protects the receiver; the **congestion window** (cwnd) protects the network. The sender can have at most min(rwnd, cwnd) bytes in flight.

- **Slow start**: cwnd starts small (Linux initial cwnd = 10 segments, about 14 KB) and roughly doubles every RTT until loss or a threshold. This is why short HTTP transfers never reach full bandwidth, and why reusing a warm connection is faster.
- **Congestion avoidance**: grow linearly after the threshold.
- **On loss**: reduce cwnd. Classic algorithms (Reno, **CUBIC**, the Linux default) treat loss as congestion.
- **BBR**: models bottleneck bandwidth and RTT instead of reacting to loss; often much better on lossy long-haul links. Enable with `sysctl -w net.ipv4.tcp_congestion_control=bbr`.

3.4.8 Retransmission

TCP detects loss two ways:

- **Retransmission timeout (RTO)**: no ACK within the RTO (computed from smoothed RTT; Linux minimum 200 ms, initial 1 s). Expensive: the connection stalls.

- **Fast retransmit**: three duplicate ACKs indicate a specific segment was lost while later ones arrived; retransmit immediately. **SACK** (selective acknowledgment) tells the sender exactly which ranges arrived.

Even 1% packet loss can cut throughput dramatically and adds tail latency spikes of 200 ms+ on affected requests. Check retransmissions with:

```
nstat -az | grep -E
'TcpRetransSegs|TcpExtTCPTimeouts|TcpExtListenOverflows|TcpExtListenDrops'
ss -ti dst 10.0.2.40   # per-connection rtt, cwnd, retrans
```

```
ESTAB 0 0 10.0.1.25:51544 10.0.2.40:5432
   cubic wscale:7,7 rto:204 rtt:1.27/0.4 mss:8949 cwnd:10 bytes_acked:48211 retrans:0/2
rcv_space:62727
```

3.4.9 Keepalives and idle timeouts

TCP itself sends nothing on an idle connection. Middleboxes (NAT gateways, load balancers, firewalls) drop idle state after a timeout: AWS NAT Gateway 350 s, AWS ALB default 60 s, AWS NLB 350 s for TCP. The endpoints do not know and the next send gets an RST or hangs. Solutions: enable TCP keepalive with an interval below the shortest middlebox timeout (Linux default `tcp_keepalive_time` is 7200 s, far too long), or application-level pings.

3.5 UDP

UDP (User Datagram Protocol) is an 8-byte header with ports, length, and checksum. No handshake, no ordering, no retransmission, no congestion control.

	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Acknowledged, retransmitted	Best effort
Ordering	Guaranteed	None
Flow/congestion control	Yes	No (application's job)
Header	20–60 bytes	8 bytes
Boundaries	Byte stream (no message boundaries)	Preserves datagram boundaries
Uses	HTTP/1.1, HTTP/2, SSH, databases	DNS, NTP, DHCP, syslog, VoIP, video, QUIC (HTTP/3), WireGuard, VXLAN

Why use UDP? Lower latency (no handshake), no head-of-line blocking, and the ability to build your own reliability exactly how you want it (which is what QUIC does).

Debugging UDP is harder: there is no handshake to prove reachability. A closed UDP port usually triggers an ICMP “port unreachable”, but if a firewall drops packets you just get silence. Firewalls and NATs track UDP “connections” with short timeouts (often 30–180 s).

```
# Test UDP reachability with netcat
nc -u -v -z 10.0.0.2 53
# Listen on UDP 9999 on one host, send from another
nc -u -l 9999
echo hello | nc -u -w1 10.0.0.5 9999
```

3.6 DHCP

DHCP (Dynamic Host Configuration Protocol) automatically gives hosts an IP address, mask, default gateway, DNS servers, and other options. It uses UDP: server port 67, client port 68.

3.6.1 DORA

```
Client (0.0.0.0:68)                          DHCP Server (:67)
  |--- DISCOVER (broadcast 255.255.255.255) --->|   "I need an address"
  |<-- OFFER  (10.0.1.25, lease 3600 s) --------|   "You can have 10.0.1.25"
  |--- REQUEST (broadcast, "I take .25") ------>|   (broadcast so other servers know)
  |<-- ACK (lease confirmed + options) ---------|
```

- **Lease**: the address is rented. The client renews at 50% of the lease (T1) directly with the server, and rebinds via broadcast at 87.5% (T2).
- **Options**: 1 subnet mask, 3 router, 6 DNS servers, 15 domain name, 42 NTP, 119 search domains, 121 classless static routes.
- **DHCP relay** (ip helper): broadcasts do not cross routers, so routers forward DHCP to a central server and tag the originating subnet (giaddr).

In AWS, DHCP is provided by the VPC and configured by a **DHCP options set** (domain name, DNS servers, NTP). Changing it is how you point an entire VPC at custom DNS resolvers. Kubernetes pods do not use DHCP; IPs come from the CNI’s IPAM.

```
# Inspect lease details on a systemd-networkd host
networkctl status ens5
# Or NetworkManager
nmcli device show ens5 | grep -E 'IP4|DHCP'
```

> **Note:** Rogue DHCP servers (someone plugs in a home router) hand out wrong gateways and break whole offices. Switches defend against this with DHCP snooping.

3.7 Lab: Routing Between Networks and Watching TCP

Part A — Two subnets and a router.

```
ns: a 10.1.0.2/24 ---- ns: r (10.1.0.1 | 10.2.0.1) ---- ns: b 10.2.0.2/24
```

Step 1. Build the topology:

```
for ns in a r b; do sudo ip netns add $ns; done
sudo ip link add va type veth peer name vra
sudo ip link add vb type veth peer name vrb
sudo ip link set va netns a; sudo ip link set vra netns r
sudo ip link set vb netns b; sudo ip link set vrb netns r
sudo ip -n a addr add 10.1.0.2/24 dev va
sudo ip -n r addr add 10.1.0.1/24 dev vra
sudo ip -n r addr add 10.2.0.1/24 dev vrb
sudo ip -n b addr add 10.2.0.2/24 dev vb
for p in "a va" "r vra" "r vrb" "b vb"; do set -- $p; sudo ip -n $1 link set $2 up; sudo ip -
n $1 link set lo up; done
sudo ip netns exec r sysctl -qw net.ipv4.ip_forward=1
```

Step 2. Ping `b` from `a`. It fails with “Network is unreachable”, because `a` has only a connected route:

```
sudo ip netns exec a ping -c1 10.2.0.2
sudo ip -n a route
```

Step 3. Add a route on `a` only. Ping now times out instead: requests reach `b`, but `b` has no route back. Confirm with `tcpdump` in `b`:

```
sudo ip -n a route add 10.2.0.0/24 via 10.1.0.1
sudo ip netns exec b tcpdump -ni vb icmp -c 2 &
sudo ip netns exec a ping -c2 -W1 10.2.0.2
```

Step 4. Add the return route on `b` (use a default route this time) and the ping succeeds. Note the TTL of 63: one router hop decremented it.

```
sudo ip -n b route add default via 10.2.0.1
sudo ip netns exec a ping -c2 10.2.0.2
sudo ip netns exec a ip route get 10.2.0.2
```

Part B — Watch a TCP handshake and states.

Step 5. Start a capture on the router and a TCP listener in `b`:

```
sudo ip netns exec r tcpdump -ni vrb 'tcp port 8000' &
sudo ip netns exec b python3 -m http.server 8000 --bind 10.2.0.2 &
```

Step 6. Make a request from `a`:

```
sudo ip netns exec a curl -s -o /dev/null -w '%{http_code}\n' http://10.2.0.2:8000/
```

```
IP 10.1.0.2.46302 > 10.2.0.2.8000: Flags [S], seq 3711029473, win 64240, options [mss
1460,sackOK,TS val 1 ecr 0,nop,wscale 7], length 0
IP 10.2.0.2.8000 > 10.1.0.2.46302: Flags [S.], seq 1402219917, ack 3711029474, win 65160,
options [mss 1460,...], length 0
IP 10.1.0.2.46302 > 10.2.0.2.8000: Flags [.], ack 1, win 502, length 0
IP 10.1.0.2.46302 > 10.2.0.2.8000: Flags [P.], seq 1:78, ack 1, win 502, length 77: HTTP: GET
/ HTTP/1.1
...
IP 10.2.0.2.8000 > 10.1.0.2.46302: Flags [F.], seq 924, ack 78, length 0
```

In tcpdump, `[S]` = SYN, `[S.]` = SYN-ACK, `[.]` = ACK, `[P.]` = PUSH+ACK (data), `[F.]` = FIN, `[R]` = RST.

Step 7. Observe TIME-WAIT on the side that closed first (here the server, because `http.server` closes after responding):

```
sudo ip netns exec b ss -tan state time-wait
```

Step 8. Compare refused vs timeout. Connect to a closed port (`curl http://10.2.0.2:9999`): you get an immediate “Connection refused” and tcpdump shows `[R.]`. Now drop traffic on the router and retry:

```
sudo ip netns exec r iptables -A FORWARD -p tcp --dport 8000 -j DROP
sudo ip netns exec a curl -m 5 http://10.2.0.2:8000/
```

```
curl: (28) Connection timed out after 5001 milliseconds
```

Step 9. Clean up: `for ns in a r b; do sudo ip netns del $ns; done` and `kill %1 %2` for the background jobs.

Key takeaways

- Every host has a route table; the kernel uses longest prefix match. Use `ip route get <ip>` to see the real decision.
- Routing is per direction; missing return routes and asymmetric paths are common causes of “one-way” connectivity.
- OSPF is a link-state IGP; BGP (TCP 179) is the policy-driven path-vector protocol of the Internet, hybrid cloud, and Kubernetes load balancer advertisement.
- A connection is identified by its 5-tuple; ephemeral port ranges limit connections from one source to one destination.

- TCP handshake costs one RTT; teardown leaves TIME_WAIT on the active closer for 60 s on Linux. CLOSE_WAIT buildup is an application bug.
- “Connection refused” (RST) means the path works but nothing listens; “timed out” means packets are being dropped.
- Throughput is bounded by window / RTT; loss triggers retransmission and slashes throughput. Keep idle connections alive below middlebox timeouts.
- UDP trades reliability for latency and simplicity; DHCP (UDP 67/68) uses DORA to hand out leases and options.

Part 4 — DNS in Depth

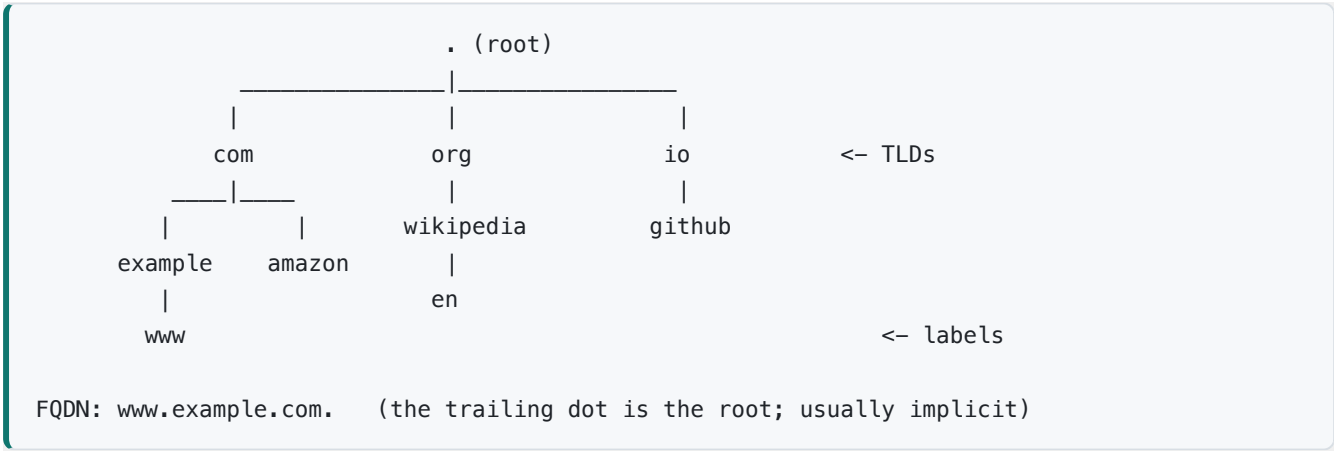
"It's always DNS" is a joke because it is so often true. DNS is a globally distributed, heavily cached database that every connection depends on, and caching means mistakes can persist for hours after they are fixed. This Part covers how resolution works, record types, TTLs and caching, split-horizon DNS, cloud routing policies, Kubernetes DNS gotchas, and how to debug all of it with `dig`.

4.1 What DNS Does

DNS (Domain Name System) maps names to data, most commonly to IP addresses. It is:

- **Hierarchical**: the namespace is a tree rooted at `.`.
- **Distributed**: each zone is served by its own authoritative servers.
- **Cached**: answers are cached at every level for the record's TTL.
- **Mostly UDP on port 53**: TCP 53 is used for responses too big for UDP (truncation), zone transfers, and increasingly for DNS over TLS (853) and DNS over HTTPS (443).

4.1.1 The namespace



A **zone** is a portion of the namespace managed as a unit. `example.com` can delegate `dev.example.com` to a different set of name servers, making it a separate zone.

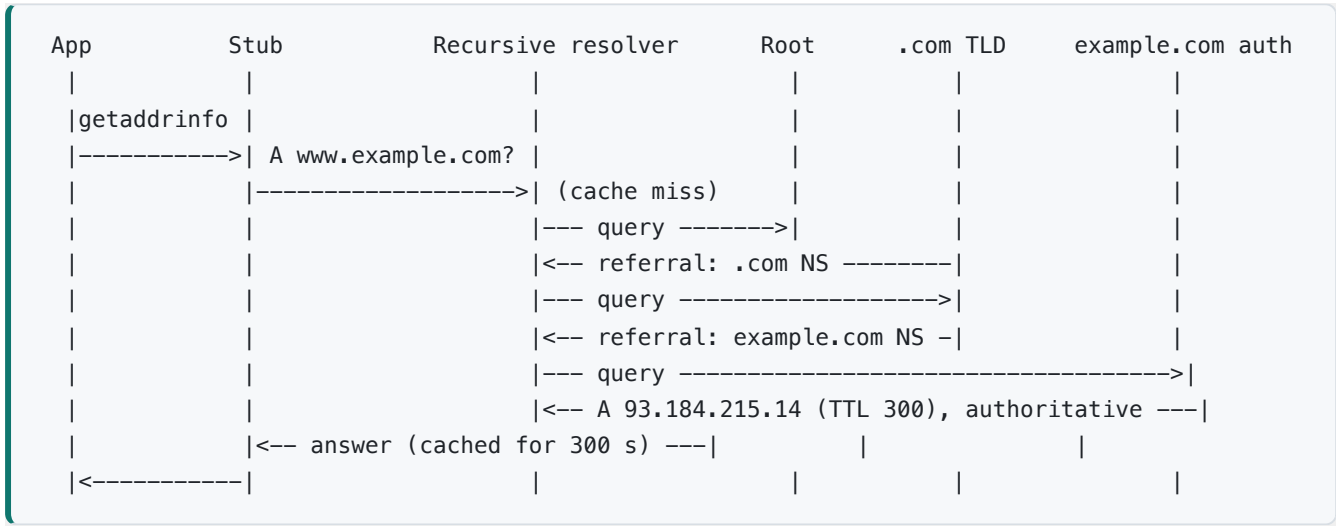
4.2 The Resolution Path

4.2.1 The players

Component	Role	Examples
Stub resolver	Library in the OS that sends queries; does not recurse	glibc, `systemd-resolved`, musl

Component	Role	Examples
Recursive resolver	Does the legwork, caches results	ISP resolver, 8.8.8.8, 1.1.1.1, AWS Route 53 Resolver (VPC+2), CoreDNS (for cluster zones and forwarding)
Root servers	Know where TLD servers are	13 named letters (a–m), hundreds of anycast instances
TLD servers	Know authoritative servers for each domain	Verisign for `.com`
Authoritative servers	Hold the actual records for a zone	Route 53, Cloudflare, BIND

4.2.2 A full recursive lookup



Key points:

- The stub sends a **recursive** query (RD flag set): “give me the final answer”.
- The recursive resolver sends **iterative** queries to root, TLD, authoritative, following referrals.
- **Glue records**: if `example.com`’s name server is `ns1.example.com`, the TLD must include its IP in the referral or you get a chicken-and-egg loop.
- The resolver caches every step (root and TLD NS records are cached for days), so most lookups only need the final hop or no hop at all.

4.2.3 What happens on the host before DNS

Linux does not go straight to DNS. `getaddrinfo()` follows `/etc/nsswitch.conf`:

```
hosts: files dns
```

So `/etc/hosts` wins first. Then the resolver configuration in `/etc/resolv.conf`:

```
nameserver 127.0.0.53
options edns0 trust-ad
```

```
search ec2.internal corp.example.com
```

On systemd-based distros, `127.0.0.53` is `systemd-resolved`, a local caching stub. The real upstream servers are shown by:

```
resolvectl status
```

```
Link 2 (ens5)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 10.0.0.2
       DNS Servers: 10.0.0.2
        DNS Domain: ec2.internal
```

> **Warning:** `dig` and `nslookup` talk to DNS servers directly and **bypass** `/etc/hosts` and nsswitch. Your application uses `getaddrinfo()`. To see what an application would get, use `getent ahosts www.example.com`. Many "DNS works with dig but the app fails" mysteries are an `/etc/hosts` entry or a search-domain issue.

4.2.4 Search domains and ndots

If a name has fewer dots than `ndots` (default 1), the resolver tries appending each search domain first. With `search corp.example.com` and a query for `db`, it tries `db.corp.example.com` and then `db.`. This is convenient and also the source of the famous Kubernetes `ndots:5` performance issue (Section 4.7).

4.3 Record Types

Type	Purpose	Example
A	Name to IPv4	`api.example.com. 300 IN A 203.0.113.10`
AAAA	Name to IPv6	`api.example.com. 300 IN AAAA 2001:db8::10`
CNAME	Alias to another name	`www.example.com. 300 IN CNAME example.com.`
ALIAS / ANAME / Route 53 Alias	Provider-specific CNAME-like at zone apex	`example.com -> my-alb-123.us-east-1.elb.amazonaws.com`
MX	Mail servers, with priority	`example.com. 3600 IN MX 10 mail.example.com.`
TXT	Arbitrary text: SPF, DKIM, DMARC, domain verification	`"v=spf1 include:_spf.google.com ~all"`

Type	Purpose	Example
NS	Authoritative name servers for a zone	`example.com. 172800 IN NS ns-1.awsdns-01.org.`
SOA	Start of authority: primary NS, serial, timers, negative TTL	
PTR	Reverse lookup: IP to name	`10.113.0.203.in-addr.arpa. PTR api.example.com.`
SRV	Service location: priority, weight, port, target	`_sip._tcp.example.com. SRV 10 60 5060 sip1.example.com.`
CAA	Which CAs may issue certificates	`example.com. CAA 0 issue "letsencrypt.org"`
HTTPS / SVCB	Service binding: ALPN (h2/h3), IP hints, ECH	`example.com. HTTPS 1 . alpn="h3,h2"`
DS / DNSKEY / RRSIG	DNSSEC chain of trust	

4.3.1 CNAME rules that bite

- A CNAME cannot coexist with any other record at the same name. Therefore you **cannot put a CNAME at the zone apex** (`example.com`), because the apex must have SOA and NS records. This is why providers invented ALIAS/flattening, and why AWS Route 53 “Alias” records exist.
- Each CNAME hop may require another lookup. Avoid long chains.
- Route 53 Alias records to AWS resources (ALB, CloudFront, S3 website, API Gateway) are free to query and track the target’s IPs automatically.

4.3.2 The SOA record and negative caching

```
example.com. 900 IN SOA ns-1.awsdns-01.org. hostmaster.example.com. (
        2026101201 ; serial
        7200       ; refresh
        900        ; retry
        1209600    ; expire
        300 )      ; minimum = negative caching TTL
```

The last field controls how long resolvers cache **NXDOMAIN** (name does not exist). If you query a name *before* creating it, resolvers will cache “does not exist” for that duration, which explains “I created the record but it still doesn’t resolve”.

4.4 TTL and Caching Strategy

TTL (time to live, in seconds) tells resolvers how long they may cache an answer.

TTL	Trade-off
30–60 s	Fast failover and migrations; more queries, more resolver load, slightly higher latency
300 s	Common default for application endpoints
3600–86400 s	Stable records (MX, NS, verification TXT); fewer queries, slow to change

Migration playbook: lower the TTL to 60 s at least one old-TTL period *before* the change (if the TTL was 86400, lower it a day ahead), make the change, verify, then raise the TTL again.

> **Note:** Not every client honors TTLs. The JVM historically cached DNS forever when a security manager was installed (`networkaddress.cache.ttl`); some HTTP clients and connection pools resolve once and keep connections for hours. After a DNS cutover, old connections keep going to old IPs until they are closed. DNS changes redirect *new* connections only.

4.5 Split-Horizon DNS

Split-horizon (split-brain) DNS returns different answers for the same name depending on who asks. The typical use: `api.example.com` resolves to a public load balancer from the Internet but to a private IP from inside the VPC or corporate network.

```
                query: api.example.com
                        |
       +----------------+-----------------+
       | from Internet                    | from inside VPC
       v                                  v
Public hosted zone                Private hosted zone (associated with VPC)
api -> 203.0.113.10 (ALB)         api -> 10.0.32.15 (internal NLB)
```

In AWS, a Route 53 **private hosted zone** is associated with one or more VPCs; queries from those VPCs to the VPC resolver use it, everyone else gets the public zone.

Hybrid DNS: on-prem and cloud usually need to resolve each other's private names. Route 53 Resolver **inbound endpoints** let on-prem DNS forward queries into the VPC; **outbound endpoints** with forwarding rules send queries for `corp.example.com` to on-prem DNS servers.

> **Warning:** With split-horizon, a private zone for `example.com` completely shadows the public zone for clients in that VPC. If a name exists only in the public zone, VPC clients get NXDOMAIN. Either create a private zone for a subdomain only (`internal.example.com`), or duplicate the needed records.

4.6 Routing Policies (Route 53 Style)

Managed DNS providers can return different answers based on policy. Route 53's policies are a useful reference set:

Policy	Behavior	Use case
Simple	One record set, possibly multiple values returned in random order	Single endpoint
Weighted	Return records in proportion to weights (e.g., 90/10)	Canary or blue/green cutover
Latency	Return the region with the lowest measured latency to the resolver	Multi-region active-active
Failover	Primary while healthy, secondary when its health check fails	Active-passive DR
Geolocation	Based on continent/country/state of the requester	Data residency, localized content
Geoproximity	Based on distance, adjustable with a bias	Shift traffic between regions gradually
Multivalue answer	Up to 8 healthy records, randomly	Simple client-side load distribution with health checks
IP-based	Based on the client's source CIDR	Route specific ISPs or networks

Health checks drive these policies: Route 53 health checkers probe an endpoint (HTTP/HTTPS/TCP) from multiple locations and mark it unhealthy after a threshold.

> **Tip:** DNS-based failover is only as fast as TTL plus health-check detection time plus client caching. Expect 1–5 minutes in practice. For sub-second failover, use anycast or a global load balancer (AWS Global Accelerator, GCP global LB) in front of regions.

> **Note:** The "location" seen by DNS is the *resolver's* location, not the user's, unless the resolver sends **EDNS Client Subnet** (ECS). Users on a corporate VPN or a far-away public resolver may be routed to a suboptimal region.

4.7 DNS in Kubernetes

Kubernetes runs **CoreDNS** as a cluster DNS service (usually `kube-dns` Service at the 10th IP of the service CIDR, e.g., `10.96.0.10`).

Name	Resolves to
`my-svc.my-ns.svc.cluster.local`	ClusterIP of the Service
`my-svc` (from a pod in `my-ns`)	Same, via search domains

Name	Resolves to
Headless Service name	A records for each ready pod IP
`pod-0.my-svc.my-ns.svc.cluster.local`	Specific StatefulSet pod
`_http._tcp.my-svc.my-ns.svc.cluster.local`	SRV record for named port

A pod's `/etc/resolv.conf`:

```
search my-ns.svc.cluster.local svc.cluster.local cluster.local ec2.internal
nameserver 10.96.0.10
options ndots:5
```

4.7.1 The ndots:5 problem

With `ndots:5`, any name with fewer than 5 dots is first tried with each search domain. A lookup of `api.stripe.com` (2 dots) generates:

```
api.stripe.com.my-ns.svc.cluster.local   A + AAAA -> NXDOMAIN
api.stripe.com.svc.cluster.local         A + AAAA -> NXDOMAIN
api.stripe.com.cluster.local             A + AAAA -> NXDOMAIN
api.stripe.com.ec2.internal              A + AAAA -> NXDOMAIN
api.stripe.com                           A + AAAA -> answer
```

Up to 10 queries for one external lookup. At scale this overloads CoreDNS and can hit the AWS VPC resolver limit (1,024 packets per second per ENI). Fixes:

- Use FQDNs with a trailing dot in configuration: `api.stripe.com.`
- Lower ndots per pod: `dnsConfig: { options: [ { name: ndots, value: "2" } ] }`
- Run **NodeLocal DNSCache** (a caching agent on every node).
- Scale CoreDNS and enable its `cache` plugin (on by default).

4.7.2 Conntrack and UDP DNS races

On older kernels, parallel A and AAAA queries from the same socket could race in conntrack and one would be dropped, causing 5-second timeouts (the resolver's retry interval). Signs: intermittent latency of exactly 5 s on lookups. Mitigations: NodeLocal DNSCache, `options single-request-reopen` (glibc), or newer kernels.

4.8 DNSSEC and Encrypted DNS (Brief)

- **DNSSEC** signs records so resolvers can verify authenticity (RRSIG, DNSKEY, DS chain from the root). It protects integrity, not privacy. Misconfigured DNSSEC (expired signatures, broken DS after a provider migration) makes the entire domain fail to resolve for validating resolvers.

- **DoT** (DNS over TLS, port 853) and **DoH** (DNS over HTTPS, port 443) encrypt the stub-to-resolver hop for privacy. Corporate networks sometimes block them to enforce internal resolvers.

4.9 Debugging with dig

4.9.1 Reading dig output

```
dig www.example.com
```

```
; <<>> DiG 9.18.28 <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41022
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        3600    IN      CNAME   example.com.
example.com.            217     IN      A       93.184.215.14

;; Query time: 3 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Oct 12 10:02:11 UTC 2026
;; MSG SIZE  rcvd: 74
```

Field	Meaning
`status: NOERROR`	Success. `NXDOMAIN` = name does not exist. `SERVFAIL` = resolver failed (upstream down, DNSSEC failure). `REFUSED` = server will not answer you
`flags: qr rd ra`	qr = response, rd = recursion desired, ra = recursion available, **aa** = authoritative answer
TTL (217)	Remaining cache time; a counting-down value means it came from cache
`SERVER`	Which server actually answered

> **Note:** `NOERROR` with zero answers ("NODATA") means the name exists but has no record of that type, e.g., you asked for AAAA and only A exists. It is different from NXDOMAIN.

4.9.2 The dig commands you will actually use

```
dig +short api.example.com                # just the answer
dig api.example.com AAAA                  # specific record type
```

```
dig @8.8.8.8 api.example.com                # ask a specific resolver
dig @ns-1.awsdns-01.org api.example.com     # ask the authoritative server directly (bypass
caches)
dig +trace api.example.com                  # follow delegation from root yourself
dig example.com NS +short                   # who is authoritative?
dig -x 203.0.113.10                         # reverse lookup (PTR)
dig +tcp api.example.com                    # force TCP (test if TCP 53 is allowed)
dig api.example.com +norecurse @10.0.0.2    # is it in this resolver's cache?
dig +dnssec example.com                     # show DNSSEC records
getent ahosts api.example.com               # what the app sees (hosts file, nsswitch)
```

4.9.3 A debugging procedure

```
1. What does the APP see?       getent ahosts name     (compare with dig)
2. What does my resolver say?   dig name               (status, TTL, server)
3. What does the AUTHORITY say? dig NS zone; dig @auth-ns name
     - Auth correct, resolver wrong -> caching; wait TTL or flush
     - Auth wrong -> fix the record / check you edited the right zone
4. Is delegation correct?       dig +trace name
     - Registrar NS records must match the hosted zone's NS records
5. Split horizon?               Same query from inside vs outside the network
6. Search domains / ndots?      Query the FQDN with trailing dot
```

> **Interview:** "You changed a DNS record but users still hit the old server. Why?" Cached answers within TTL at recursive resolvers, negative caching if the name previously did not exist, OS and browser caches, applications caching resolutions or holding long-lived connections, and possibly editing the wrong zone (private vs public, or a zone the registrar does not delegate to).

4.10 Lab: Run Your Own Authoritative and Recursive DNS

This lab uses Docker to run CoreDNS as an authoritative server for a lab zone, then queries it.

Step 1. Create a zone file and Corefile:

```
mkdir -p ~/dnslab && cd ~/dnslab
cat > db.lab.test <<'EOF'
$ORIGIN lab.test.
$TTL 60
@     IN SOA ns1.lab.test. admin.lab.test. 2026101201 7200 900 1209600 30
@     IN NS  ns1.lab.test.
ns1   IN A   127.0.0.1
web   IN A   10.0.0.10
web   IN A   10.0.0.11
api   IN CNAME web.lab.test.
_http._tcp IN SRV 10 50 8080 web.lab.test.
@     IN TXT "v=spf1 -all"
EOF
```

```
cat > Corefile <<'EOF'
lab.test:53 {
    file /etc/coredns/db.lab.test
    log
    errors
}
.:53 {
    forward . 1.1.1.1 8.8.8.8
    cache 30
    log
}
EOF
```

Step 2. Run CoreDNS:

```
docker run -d --name coredns -p 1053:53/udp -p 1053:53/tcp \
  -v ~/dnslab:/etc/coredns coredns/coredns:1.11.3 -conf /etc/coredns/Corefile
```

Step 3. Query your zone. Note the `aa` flag (authoritative):

```
dig @127.0.0.1 -p 1053 web.lab.test
dig @127.0.0.1 -p 1053 api.lab.test +short
dig @127.0.0.1 -p 1053 _http._tcp.lab.test SRV +short
```

```
web.lab.test.
10.0.0.10
10.0.0.11
10 50 8080 web.lab.test.
```

Step 4. Observe negative caching. Query a name that does not exist; note `NXDOMAIN` and the SOA in the AUTHORITY section (its last field, 30, is the negative TTL):

```
dig @127.0.0.1 -p 1053 new.lab.test
```

Step 5. Observe recursive forwarding and caching. Run twice and watch the TTL count down and query time drop:

```
dig @127.0.0.1 -p 1053 github.com | grep -E 'IN\s+A|Query time'
sleep 5; dig @127.0.0.1 -p 1053 github.com | grep -E 'IN\s+A|Query time'
```

Step 6. Trace the public delegation path for a real domain:

```
dig +trace +nodnssec github.com | grep -E 'NS|A\s'
```

Step 7. See the ndots effect. Run a second CoreDNS instance on a user-defined Docker network with a fixed IP, then query it from a client container configured like a Kubernetes pod:

```
docker network create --subnet 172.30.0.0/24 dnsnet
docker run -d --name coredns2 --network dnsnet --ip 172.30.0.53 \
  -v ~/dnslab:/etc/coredns coredns/coredns:1.11.3 -conf /etc/coredns/Corefile
docker run --rm --network dnsnet --dns 172.30.0.53 \
  --dns-search a.svc.cluster.local --dns-search svc.cluster.local \
  --dns-opt ndots:5 debian:12-slim getent ahosts github.com
docker logs coredns2 | grep github
```

```
[INFO] 172.30.0.2:41871 - 1203 "A IN github.com.a.svc.cluster.local. udp 48 false 512"
NXDOMAIN qr,rd,ra 123 0.0213s
[INFO] 172.30.0.2:41871 - 5530 "AAAA IN github.com.a.svc.cluster.local. udp 48 false 512"
NXDOMAIN qr,rd,ra 123 0.0198s
[INFO] 172.30.0.2:52210 - 7712 "A IN github.com.svc.cluster.local. udp 46 false 512" NXDOMAIN
qr,rd,ra 121 0.0201s
...
[INFO] 172.30.0.2:39101 - 1820 "A IN github.com. udp 28 false 512" NOERROR qr,rd,ra 54
0.0102s
```

Repeat with `github.com.` (trailing dot) and count how many queries disappear.

Step 8. Clean up: `docker rm -f coredns coredns2 && docker network rm dnsnet`.

Key takeaways

- DNS is hierarchical, distributed and cached: stub -> recursive resolver -> root -> TLD -> authoritative.
- Applications resolve via `getaddrinfo()` (hosts file, nsswitch, search domains); `dig` bypasses that. Use `getent ahosts` to see what apps see.
- Know your record types; CNAMEs cannot live at the apex (use ALIAS/Alias records) or coexist with other records.
- TTLs control how fast changes propagate; lower them before migrations. NXDOMAIN is cached too (SOA minimum).
- Split-horizon returns different answers inside vs outside; private zones shadow public ones.
- Weighted, latency, failover, and geo policies make DNS a traffic-management tool, but failover is bounded by TTL and client behavior.
- In Kubernetes, `ndots:5` multiplies queries for external names; use FQDNs, tune ndots, and run NodeLocal DNSCache.
- Debug systematically: app view, resolver view, authoritative view, delegation, split-horizon.

Part 5 — HTTP, TLS, Certificates, and Proxies

Almost every service you run speaks HTTP, and almost all of it is wrapped in TLS. This Part explains how HTTP evolved and why each version exists, how TLS establishes trust, how to inspect and debug certificates with `openssl` and `curl`, and how forward and reverse proxies sit in the request path.

5.1 HTTP Fundamentals

5.1.1 Anatomy of a request and response

HTTP is a request/response protocol. HTTP/1.1 is plain text, which makes it easy to read:

```
GET /api/v1/users/42 HTTP/1.1
Host: api.example.com
User-Agent: curl/8.9.1
Accept: application/json
Authorization: Bearer eyJhbGciOi...
Connection: keep-alive

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 61
Cache-Control: no-store
Date: Mon, 12 Oct 2026 10:15:02 GMT

{"id":42,"name":"Ada","email":"ada@example.com","active":true}
```

The `Host` header is mandatory in HTTP/1.1. It is what allows **virtual hosting**: many sites on one IP address, with the server (or load balancer) routing by hostname.

5.1.2 Methods and status codes

Method	Safe	Idempotent	Typical use
GET	Yes	Yes	Read
HEAD	Yes	Yes	Headers only (health checks, size checks)
POST	No	No	Create, actions
PUT	No	Yes	Replace
PATCH	No	No (usually)	Partial update
DELETE	No	Yes	Delete

Method	Safe	Idempotent	Typical use
OPTIONS	Yes	Yes	CORS preflight, capabilities

Idempotency matters to infrastructure: proxies and service meshes can safely **retry** idempotent requests on failure, but retrying a POST can create duplicate orders.

Code	Meaning	Who usually generates it in DevOps context
200/201/204	Success	Application
301/302/307/308	Redirect (308/307 preserve the method)	App or LB (HTTP to HTTPS)
304	Not modified (cache validation)	App or CDN
400	Bad request	App, or proxy rejecting malformed headers
401 / 403	Unauthenticated / forbidden	App, API gateway, WAF
404	Not found	App, or proxy with no matching route
413	Payload too large	Proxy (`client_max_body_size` in Nginx)
429	Too many requests	Rate limiter / API gateway
499	Client closed request (Nginx-specific)	Client gave up before response
500	Internal server error	Application bug
502	Bad gateway: proxy got an invalid or no response from upstream	Proxy/LB (upstream crashed, reset, refused)
503	Service unavailable	LB with no healthy targets, or app overloaded
504	Gateway timeout: upstream did not answer in time	Proxy/LB timeout

> **Tip:** 502, 503 and 504 are almost always produced by the **proxy in front of** your app, not the app itself. They tell you which hop to look at: a 502 means the proxy could talk to something but got garbage, an RST, or a refused connection; a 504 means it waited and gave up. Always check the proxy's logs for the upstream error.

5.1.3 Headers that matter to infrastructure

Header	Purpose
`Host`	Virtual host routing
`X-Forwarded-For`	Original client IP chain, appended by each proxy

Header	Purpose
`X-Forwarded-Proto`	Original scheme (`https`) when TLS terminated at the proxy
`Forwarded`	Standardized version of the above (RFC 7239)
`X-Request-ID` / `traceparent`	Correlate logs and traces across hops
`Connection` , `Keep-Alive`	Connection reuse (HTTP/1.1)
`Cache-Control` , `ETag` , `Vary`	Caching behavior at browsers and CDNs
`Strict-Transport-Security`	HSTS: force HTTPS for a period
`Upgrade`	Switch protocols, e.g., WebSocket

> **Warning:** `X-Forwarded-For` is client-controlled input. Only trust the entries added by your own proxies (for Nginx use `set_real_ip_from` plus `real_ip_header` ; for apps, configure trusted proxy counts). Otherwise anyone can spoof their IP for rate limiting or audit logs.

5.2 HTTP Versions

5.2.1 HTTP/1.1

- Text protocol, one request at a time per connection.
- **Keep-alive** reuses the TCP connection for multiple requests (default in 1.1).
- **Head-of-line (HOL) blocking**: a slow response blocks the next on that connection. Browsers open about 6 parallel connections per host to work around it.

5.2.2 HTTP/2

- **Binary framing**; each request/response is a **stream** on one TCP connection.
- **Multiplexing**: many concurrent streams, no application-level HOL blocking.
- **HPACK** header compression (big win for repetitive headers like cookies and auth tokens).
- Negotiated via TLS **ALPN** (`h2`). Cleartext HTTP/2 (`h2c`) is used internally, e.g., by gRPC.
- **gRPC runs on HTTP/2**, which has load-balancing consequences: one long-lived connection carries all requests, so an L4 load balancer sends everything from one client to one backend. You need L7 (per-request) balancing for gRPC (Part 6, Part 8).
- Still suffers **TCP-level HOL blocking**: one lost packet stalls all streams until it is retransmitted.

5.2.3 HTTP/3 and QUIC

- Runs over **QUIC**, a transport built on **UDP port 443**.
- QUIC integrates TLS 1.3 into the transport: connection setup is 1 RTT, and 0-RTT on resumption.
- Streams are independent at the transport layer: a lost packet only stalls its own stream.

- **Connection migration**: connections are identified by a connection ID, not the 5-tuple, so a phone moving from Wi-Fi to cellular keeps its connection.
- Discovered via the `Alt-Svc` response header or the DNS `HTTPS` record. Clients fall back to HTTP/2 if UDP 443 is blocked, which corporate firewalls often do.

```
             HTTP/1.1          HTTP/2            HTTP/3
Semantics    HTTP              HTTP              HTTP
Framing      text              binary frames     binary frames
Multiplexing no                streams           streams
Security     TLS (optional)    TLS (in practice) TLS 1.3 built in
Transport    TCP               TCP               QUIC over UDP
New conn RTT 1 (TCP) + TLS     1 (TCP) + TLS     1 (0 on resume)
```

```
curl -sI --http2 https://www.cloudflare.com | head -1
curl -sI --http3 https://www.cloudflare.com | head -1   # needs curl built with HTTP/3
curl -sI https://www.google.com | grep -i alt-svc
```

```
HTTP/2 200
HTTP/3 200
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

5.2.4 WebSockets

A WebSocket starts as an HTTP/1.1 request with `Upgrade: websocket` and then becomes a long-lived bidirectional TCP stream. Proxies must be configured to pass the `Upgrade` and `Connection` headers, and idle timeouts must be long enough (or the app must send pings), otherwise connections drop every 60 seconds.

5.3 Using curl as a Network Debugger

`curl -v` shows each layer of a request: DNS, TCP, TLS, HTTP.

```
curl -v https://api.example.com/health
```

```
* Host api.example.com:443 was resolved.
* IPv6: (none)
* IPv4: 203.0.113.10
*   Trying 203.0.113.10:443...
* Connected to api.example.com (203.0.113.10) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
```

```
* SSL connection using TLSv1.3 / TLS_AES_128_GCM_SHA256 / X25519 / RSASSA-PSS
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=api.example.com
*  start date: Sep  1 00:00:00 2026 GMT
*  expire date: Nov 30 23:59:59 2026 GMT
*  subjectAltName: host "api.example.com" matched cert's "api.example.com"
*  issuer: C=US; O=Let's Encrypt; CN=R11
*  SSL certificate verify ok.
> GET /health HTTP/2
> Host: api.example.com
< HTTP/2 200
< content-type: application/json
{"status":"ok"}
```

Timing breakdown, which tells you *where* the time goes:

```
curl -o /dev/null -s -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%
{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n' https://api.example.com/
```

```
dns=0.012 connect=0.045 tls=0.112 ttfb=0.890 total=0.905
```

Interpretation: DNS 12 ms, TCP 33 ms (RTT), TLS 67 ms, then 778 ms waiting for the server. The network is fine; the backend is slow.

Other essential flags:

```
curl --resolve api.example.com:443:10.0.32.15 https://api.example.com/   # bypass DNS, keep
SNI/Host
curl -H 'Host: api.example.com' http://10.0.32.15/                       # test a vhost by IP
(HTTP)
curl --connect-to api.example.com:443:new-lb.example.net:443 https://api.example.com/
curl -k https://...      # skip cert verification (diagnosis only, never in production code)
curl --cacert ca.pem --cert client.pem --key client.key https://...      # mTLS
```

5.4 TLS

5.4.1 What TLS provides

1. **Confidentiality**: traffic is encrypted.
2. **Integrity**: tampering is detected.
3. **Authentication**: the server proves its identity with a certificate (and optionally the client, in mTLS).

Use TLS 1.2 and 1.3 only. SSLv3, TLS 1.0 and 1.1 are deprecated and insecure.

5.4.2 TLS 1.3 handshake

```
Client                                                  Server
  |--- ClientHello ------------------------------------------>|
  |    supported versions, cipher suites, key_share (X25519), |
  |    SNI = api.example.com, ALPN = [h2, http/1.1]           |
  |                                                           |
  |<-- ServerHello (chosen cipher, key_share) ----------------|
  |    {EncryptedExtensions (ALPN = h2)}                       |
  |    {Certificate chain}                                     |
  |    {CertificateVerify: signature proving key ownership}    |
  |    {Finished}                                              |
  |                                                           |
  |--- {Finished} ------------------------------------------->|
  |--- {HTTP request} (application data, 1 RTT total) ------->|
       { } = encrypted with keys derived from the ECDHE exchange
```

- **Key exchange**: ephemeral Diffie-Hellman (ECDHE). Both sides derive the same session keys without ever sending them. Because keys are ephemeral, stealing the server's private key later cannot decrypt recorded traffic (**forward secrecy**).
- **Authentication**: the certificate binds a public key to a name; `CertificateVerify` proves the server holds the matching private key.
- **TLS 1.2** needs 2 RTTs and allows weaker options (RSA key exchange without forward secrecy). TLS 1.3 removed them.
- **SNI** (Server Name Indication) is sent in the ClientHello so the server knows which certificate to present. Without SNI, a server with many certificates returns the default one, the cause of many "wrong certificate" errors when testing with `openssl` without `-servername`. SNI is visible to observers (ECH, Encrypted Client Hello, addresses this).
- **ALPN** negotiates the application protocol (h2, http/1.1) inside the handshake.

5.4.3 Certificates and the chain of trust

An X.509 certificate contains a subject, a public key, a validity period, **Subject Alternative Names** (SANs; the hostnames it is valid for), key usage, and the issuer's signature.

```
  [Root CA]                  self-signed, in the OS/browser trust store
      | signs
  [Intermediate CA]          sent by the server
      | signs
  [Leaf: api.example.com]    sent by the server

Server must send: leaf + intermediate(s). Not the root.
Client verifies:  each signature up the chain, ends at a trusted root,
                  validity dates, hostname matches a SAN, not revoked.
```

> **Warning:** The most common TLS failure in production is a **missing intermediate**. Browsers often still work (they cache intermediates or fetch them via AIA), but `curl`, Java, Python, and Go clients fail with "unable to get local issuer certificate". Always deploy the **full chain** (`fullchain.pem` from Let's Encrypt).

Hostname matching uses SANs only; the Common Name is ignored by modern clients. A wildcard `*.example.com` matches `api.example.com` but **not** `example.com` or `a.b.example.com`.

5.4.4 Certificate lifetimes and automation

Publicly trusted certificate lifetimes are shrinking: CA/Browser Forum rules reduce the maximum from 398 days to 200 days (from March 2026), 100 days (2027), and 47 days (2029). Let's Encrypt issues 90-day certificates. The lesson: **automate issuance and renewal**. Manual certificate management is now an outage generator.

Let's Encrypt and ACME: the ACME protocol proves you control a domain with a challenge:

Challenge	How	When to use
HTTP-01	Serve a token at `http://domain/.well-known/acme-challenge/<token>` on port 80	Simple public web servers
DNS-01	Create a TXT record `_acme-challenge.domain`	Wildcards, internal servers, no inbound port 80
TLS-ALPN-01	Special certificate on port 443	When only 443 is open

Tools: `certbot`, `acme.sh`, Caddy (automatic), and in Kubernetes **cert-manager**, which watches `Certificate` resources or Ingress/Gateway annotations and stores certificates in Secrets.

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
      - dns01:
          route53:
            region: us-east-1
```

Use the Let's Encrypt **staging** endpoint while testing to avoid rate limits.

5.4.5 mTLS

In **mutual TLS**, the server also requests a certificate from the client and verifies it against a trusted CA. It authenticates *machines/workloads* rather than users. Uses: service-to-service authentication in a service mesh (Part 8), securing APIs between companies, IoT devices, Kubernetes components (kubelet to API server).

```
Client                                Server
  |--- ClientHello ---------------------->|
  |<-- ServerHello, Certificate,          |
  |    CertificateRequest (acceptable CAs) |
  |--- Certificate (client cert),         |
  |    CertificateVerify, Finished ------->|  server validates client chain
```

5.4.6 openssl commands you need

```
# Inspect a live server's certificate chain (always pass -servername for SNI)
openssl s_client -connect api.example.com:443 -servername api.example.com -showcerts
</dev/null

# Just the leaf's subject, issuer, dates, SANs
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null
2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName

# Days until expiry check (exit code non-zero if expiring within 14 days)
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null
2>/dev/null \
  | openssl x509 -noout -checkend $((14*86400))

# Inspect a certificate file
openssl x509 -in cert.pem -noout -text

# Verify a chain locally
openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem

# Does this key match this certificate? (hashes must be identical)
openssl x509 -noout -pubkey -in cert.pem | openssl sha256
openssl pkey -pubout -in key.pem | openssl sha256

# Generate a key and CSR with SANs
openssl req -new -newkey rsa:2048 -nodes -keyout api.key -out api.csr \
  -subj "/CN=api.example.com" -addext
"subjectAltName=DNS:api.example.com,DNS:www.example.com"

# Test specific protocol versions
openssl s_client -connect api.example.com:443 -servername api.example.com -tls1_2 </dev/null
```

```
subject=CN=api.example.com
issuer=C=US, O=Let's Encrypt, CN=R11
notBefore=Sep  1 00:00:00 2026 GMT
notAfter=Nov 30 23:59:59 2026 GMT
X509v3 Subject Alternative Name:
    DNS:api.example.com, DNS:www.example.com
```

5.4.7 Common TLS errors

Error	Likely cause
`certificate has expired`	Renewal failed; check automation
`unable to get local issuer certificate`	Missing intermediate, or private CA not in client trust store
`hostname mismatch` / `no alternative certificate subject name matches`	Wrong cert served (missing SNI, wrong vhost) or name not in SANs
`self-signed certificate in certificate chain`	Corporate TLS-inspecting proxy, or private CA
`wrong version number`	Speaking TLS to a plain HTTP port (or vice versa)
`handshake failure` / `no shared cipher`	Protocol/cipher mismatch, or mTLS client cert missing

5.5 Proxies

5.5.1 Forward proxy vs reverse proxy

```
FORWARD PROXY (acts for clients)          REVERSE PROXY (acts for servers)

 [Client]--+                                          +--[App 1]
 [Client]--+--> [Proxy] ---> Internet      Internet --> [Proxy]--+--[App 2]
 [Client]--+                                          +--[App 3]

 Clients know about it                    Clients think the proxy IS the server
 Egress control, caching, auditing        TLS termination, LB, routing, caching, WAF
 e.g., Squid, corporate proxy             e.g., Nginx, HAProxy, Envoy, ALB, Traefik
```

Forward proxies show up in DevOps as egress control: build agents or servers in a locked-down network reach package repositories only through a proxy. Tools honor `HTTP_PROXY`, `HTTPS_PROXY`, and `NO_PROXY` environment variables (inconsistently: some only read lowercase names, and `NO_PROXY` CIDR support varies by tool).

For HTTPS through a forward proxy, the client sends `CONNECT api.example.com:443` and the proxy opens a TCP tunnel; it cannot see inside unless it performs **TLS interception** (presenting its own certificates signed by a corporate CA that must be installed on clients, the source of many “self-signed certificate in

chain" errors inside corporate networks and Docker builds).

5.5.2 Transparent proxies

A transparent proxy intercepts traffic without client configuration, typically with iptables redirection (`REDIRECT` / `TPROXY`). Service mesh sidecars use exactly this to capture all pod traffic.

5.5.3 Preserving client identity

A reverse proxy opens a **new** TCP connection to the backend, so the backend sees the proxy's IP. Ways to preserve the client IP:

Method	Layer	How
`X-Forwarded-For` / `Forwarded`	L7	Proxy adds header; app reads it
PROXY protocol (v1 text, v2 binary)	L4	Proxy prepends a header with original addresses before the TCP payload; backend must be configured to expect it
Transparent / DSR	L3/L4	Preserve source IP at network level (NLB with client IP preservation, `externalTrafficPolicy: Local` in Kubernetes)

> **Warning:** If you enable PROXY protocol on the load balancer but not on the backend (or vice versa), every request fails with 400 errors or broken TLS handshakes, because one side sees unexpected bytes at the start of the stream.

5.6 Lab: Build a Private CA, Serve TLS and mTLS with Nginx

Step 1. Create a root CA:

```
mkdir -p ~/tlslab && cd ~/tlslab
openssl req -x509 -newkey rsa:4096 -nodes -keyout ca.key -out ca.pem -days 3650 \
  -subj "/CN=Lab Root CA"
```

Step 2. Create a server key and CSR, then sign it with the CA, including SANs:

```
openssl req -new -newkey rsa:2048 -nodes -keyout server.key -out server.csr -subj
"/CN=app.lab.test"
cat > server.ext <<'EOF'
subjectAltName = DNS:app.lab.test, DNS:localhost, IP:127.0.0.1
extendedKeyUsage = serverAuth
EOF
openssl x509 -req -in server.csr -CA ca.pem -CAkey ca.key -CAcreateserial \
  -out server.pem -days 30 -extfile server.ext
openssl x509 -in server.pem -noout -subject -issuer -ext subjectAltName
```

Step 3. Create a client certificate for mTLS:

```
openssl req -new -newkey rsa:2048 -nodes -keyout client.key -out client.csr -subj
"/CN=billing-service"
echo "extendedKeyUsage = clientAuth" > client.ext
openssl x509 -req -in client.csr -CA ca.pem -CAkey ca.key -CAcreateserial \
  -out client.pem -days 30 -extfile client.ext
```

Step 4. Write an Nginx config with TLS and optional client verification:

```
# ~/tlslab/default.conf
server {
    listen 443 ssl;
    http2 on;
    server_name app.lab.test;

    ssl_certificate     /etc/nginx/tls/server.pem;
    ssl_certificate_key /etc/nginx/tls/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    ssl_client_certificate /etc/nginx/tls/ca.pem;
    ssl_verify_client optional;

    location /public {
        return 200 "hello, anyone\n";
    }
    location /private {
        if ($ssl_client_verify != SUCCESS) { return 403 "client cert required\n"; }
        return 200 "hello, $ssl_client_s_dn\n";
    }
}
```

Step 5. Run it:

```
docker run -d --name tlslab -p 8443:443 \
  -v ~/tlslab:/etc/nginx/tls:ro -v ~/tlslab/default.conf:/etc/nginx/conf.d/default.conf:ro \
  nginx:1.27
```

Step 6. Test, observing each failure mode:

```
curl https://localhost:8443/public                       # fails: CA not trusted
curl --cacert ca.pem https://localhost:8443/public       # works
curl --cacert ca.pem https://127.0.0.1:8443/public       # works: IP is in SANs
curl --cacert ca.pem https://localhost:8443/private      # 403: no client cert
curl --cacert ca.pem --cert client.pem --key client.key https://localhost:8443/private
```

```
curl: (60) SSL certificate problem: unable to get local issuer certificate
hello, anyone
hello, anyone
```

```
client cert required
hello, CN=billing-service
```

Step 7. Inspect the handshake and negotiated protocol:

```
openssl s_client -connect localhost:8443 -servername app.lab.test -CAfile ca.pem -alpn h2
</dev/null 2>/dev/null \
  | grep -E 'Verify return|Protocol|ALPN'
```

Step 8. Clean up: `docker rm -f tlslab`.

Key takeaways

- HTTP/1.1 is text with keep-alive; HTTP/2 multiplexes streams over one TCP connection (and gRPC rides on it); HTTP/3 runs over QUIC/UDP 443 and removes TCP head-of-line blocking.
- 502/503/504 come from proxies; they point to the hop between proxy and upstream.
- `curl -v` and `curl -w` timing break a request into DNS, TCP, TLS, and server time.
- TLS 1.3 completes in 1 RTT with ephemeral ECDHE (forward secrecy); SNI selects the certificate; ALPN selects the protocol.
- Servers must send the full chain (leaf + intermediates); hostname validation uses SANs.
- Certificate lifetimes are shrinking toward 47 days: automate with ACME (certbot, cert-manager).
- mTLS authenticates both sides and underpins zero-trust service-to-service security.
- Reverse proxies terminate connections; preserve client IPs with X-Forwarded-For or PROXY protocol and only trust headers from your own proxies.

Part 6 — Load Balancers, API Gateways, CDNs, Firewalls, and VPNs

This Part covers the infrastructure that sits between users and your services: load balancers that spread traffic and hide failures, API gateways that enforce policy, CDNs that move content to the edge, firewalls that decide what is allowed, and VPNs that connect private networks securely.

6.1 Load Balancing Concepts

A load balancer distributes requests across a pool of backends (targets, upstreams) so you get **scalability** (add more backends), **availability** (route around failures), and **operability** (drain a node for deployment without dropping traffic).

6.1.1 L4 vs L7

```
L4 (transport) load balancer                L7 (application) load balancer

Client --TCP--> [LB] --TCP--> Backend       Client --TLS/HTTP--> [LB] --HTTP--> Backend
       sees: IPs, ports, TCP flags                 sees: Host, path, headers, cookies,
methods
       decides per CONNECTION                      decides per REQUEST
       (may pass through TLS untouched)            (terminates TLS, new upstream connection)
```

	L4	L7
Routing basis	5-tuple	Host, path, headers, cookies, gRPC method
TLS	Passthrough (or terminate in some products)	Terminates (and may re-encrypt)
Performance	Very high, low latency, millions of connections	Higher CPU per request
Protocols	Any TCP/UDP (databases, MQTT, DNS, game servers)	HTTP/1.1, HTTP/2, gRPC, WebSocket
Client IP	Often preserved	Via `X-Forwarded-For`
Features	Simple health checks, connection balancing	Retries, rewrites, redirects, auth, rate limits, WAF, canary by header
AWS	Network Load Balancer (NLB)	Application Load Balancer (ALB)
Software	HAProxy (mode tcp), Nginx `stream`, IPVS, Maglev, Cilium	Nginx, HAProxy (mode http), Envoy, Traefik

> **Interview:** "When would you use an NLB instead of an ALB?" Non-HTTP protocols, static IP addresses (NLB supports Elastic IPs per AZ, useful for customer allowlists), extreme throughput or very low latency, preserving client source IP natively, TLS passthrough so the backend holds the private key, and as a PrivateLink endpoint service (which requires an NLB or Gateway Load Balancer).

6.1.2 Balancing algorithms

Algorithm	How	Good for	Watch out
Round robin	Next backend in order	Homogeneous, short requests	Ignores load
Weighted round robin	Proportional to weight	Mixed instance sizes, canaries	Static
Least connections	Fewest active connections	Long or variable requests	Needs accurate state
Least outstanding requests	Fewest in-flight requests (ALB option)	Variable latency APIs	
Random with two choices (P2C)	Pick 2 at random, take the less loaded	Large fleets, distributed LBs (Envoy default for least request)	
IP hash / source hash	Hash of client IP	Simple stickiness	Uneven behind NAT
Consistent hashing (ring, Maglev)	Hash of a key; minimal remapping when pool changes	Caches, sharded backends	Hot keys

6.1.3 Health checks

- **Active**: the LB probes each backend (`GET /healthz` every 5–10 s). After N consecutive failures (unhealthy threshold) it stops routing; after M successes it resumes.
- **Passive** (outlier detection): the LB watches real traffic and ejects a backend that returns too many errors or resets.

Designing a good health endpoint:

- **Liveness-style** checks ("process is up and can serve") belong on the load balancer.
- Do **not** make the LB health check depend on downstream dependencies (database, other services). If the DB blips, every backend fails its check simultaneously and the LB has nothing to route to: you turned a partial outage into a total one. Many LBs "fail open" when all targets are unhealthy precisely because of this.
- Keep it cheap and fast; it is called constantly.

6.1.4 Sticky sessions

Session affinity sends a client to the same backend each time, via an LB-generated cookie, an application cookie, or source IP hash. It is needed when session state lives in backend memory. Downsides: uneven load, lost sessions when a backend dies, and slower draining. Prefer stateless backends with shared

session storage (Redis) and use stickiness only when you must.

6.1.5 Connection draining and graceful deployments

When a backend is deregistered, the LB stops sending new requests but lets in-flight requests finish for the **deregistration delay** (ALB default 300 s). Your app must also handle `SIGTERM` gracefully: stop accepting, finish in-flight work, then exit. In Kubernetes, add a short `preStop` sleep so the endpoint is removed from load balancers before the process stops (Part 8).

6.1.6 Timeouts: the source of mysterious 502s and 504s

Every hop has timeouts, and they must be **ordered**: each upstream's keep-alive idle timeout must be **longer** than the downstream proxy's.

```
Client --> ALB (idle 60 s) --> Nginx (keepalive_timeout 75 s) --> App (keep-alive 90 s)
```

If the app closes an idle keep-alive connection at 5 s (Node.js `server.keepAliveTimeout` default is 5 s) while the ALB thinks it is reusable for 60 s, the ALB occasionally sends a request on a connection the app just closed and gets a reset: an **intermittent 502**. Fix by setting the app's keep-alive timeout above the LB idle timeout.

6.2 Nginx and HAProxy Configurations

6.2.1 Nginx as an L7 load balancer

```
upstream api_backend {
    least_conn;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.13:8080 weight=2;
    server 10.0.1.14:8080 backup;
    keepalive 64;                       # pool of idle upstream connections
}

server {
    listen 443 ssl;
    http2 on;
    server_name api.example.com;
    ssl_certificate     /etc/ssl/api/fullchain.pem;
    ssl_certificate_key /etc/ssl/api/privkey.pem;

    client_max_body_size 20m;

    location / {
        proxy_pass http://api_backend;
        proxy_http_version 1.1;          # required for upstream keepalive
        proxy_set_header Connection "";  # do not forward "close"
```

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 3s;
        proxy_read_timeout 30s;
        proxy_next_upstream error timeout http_502 http_503;
        proxy_next_upstream_tries 2;
    }

    location /ws/ {
        proxy_pass http://api_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 1h;
    }
}

server {
    listen 80;
    server_name api.example.com;
    return 308 https://$host$request_uri;
}
```

Open-source Nginx performs **passive** health checks only (`max_fails` / `fail_timeout`); active checks require Nginx Plus or a module. Note `proxy_next_upstream` retries non-idempotent requests only if you add `non_idempotent`, which is correct default behavior.

6.2.2 Nginx as an L4 load balancer

```
stream {
    upstream postgres_replicas {
        server 10.0.64.10:5432;
        server 10.0.65.10:5432;
    }
    server {
        listen 5432;
        proxy_pass postgres_replicas;
        proxy_connect_timeout 2s;
        proxy_timeout 10m;
    }
}
```

6.2.3 HAProxy

```
global
    log stdout format raw local0
    maxconn 50000
```

```
defaults
    mode http
    log global
    option httplog
    option forwardfor
    timeout connect 3s
    timeout client  60s
    timeout server  30s
    timeout http-keep-alive 10s

frontend fe_https
    bind :443 ssl crt /etc/haproxy/certs/api.pem alpn h2,http/1.1
    http-request set-header X-Forwarded-Proto https
    acl is_admin path_beg /admin
    http-request deny if is_admin !{ src 10.0.0.0/8 }
    use_backend be_admin if is_admin
    default_backend be_api

backend be_api
    balance leastconn
    option httpchk
    http-check send meth GET uri /healthz ver HTTP/1.1 hdr Host api.example.com
    http-check expect status 200
    cookie SRV insert indirect nocache
    server api1 10.0.1.11:8080 check inter 5s fall 3 rise 2 cookie api1
    server api2 10.0.1.12:8080 check inter 5s fall 3 rise 2 cookie api2

backend be_admin
    server admin1 10.0.1.20:8080 check

frontend fe_stats
    bind 127.0.0.1:8404
    stats enable
    stats uri /stats
```

HAProxy is notable for its detailed logs: the **termination state** code in each log line (e.g., `sH--`, `SC--`, `cD--`) tells you which side ended the session and why, which is invaluable when chasing 502s and 504s.

6.3 Reverse Proxies and API Gateways

An **API gateway** is a reverse proxy specialized for APIs, adding cross-cutting concerns so each service does not reimplement them:

Concern	Example
Authentication	Validate JWTs, OAuth2 introspection, API keys, mTLS
Authorization	Scopes, per-route policies
Rate limiting and quotas	100 req/s per API key, burst 200

Concern	Example
Request/response transformation	Header injection, path rewrites, protocol translation (REST to gRPC)
Routing and versioning	`/v1/*` to old service, `/v2/*` to new
Observability	Uniform access logs, metrics, tracing headers
Developer portal and keys	Usage plans

Examples: AWS API Gateway, Kong, Apigee, Tyk, Envoy Gateway, and Kubernetes Gateway API implementations. The difference between an "L7 load balancer", "reverse proxy", "ingress controller", and "API gateway" is mostly feature set; underneath they are frequently the same software (Envoy and Nginx power many of them).

> **Tip:** Rate limiting at the gateway needs shared state across gateway replicas (Redis or a rate-limit service), or each replica enforces its own limit and the effective limit becomes limit x replicas.

6.4 CDNs

A **CDN** (Content Delivery Network) is a globally distributed set of reverse proxies (points of presence, PoPs) that cache content close to users and shield your origin.

```
User (Tokyo) --DNS/anycast--> Tokyo PoP --(cache HIT)--> response in 10 ms
                                  |
                                  +--(cache MISS)--> [Regional/shield cache] --> Origin (us-
east-1)
```

Benefits: lower latency (TCP and TLS terminate near the user, so the handshake RTTs are short), offloaded origin bandwidth, DDoS absorption, and edge features (WAF, bot management, edge compute).

Caching is controlled by headers:

```
Cache-Control: public, max-age=31536000, immutable   # fingerprinted static assets
(app.3f9a1c.js)
Cache-Control: public, s-maxage=60, stale-while-revalidate=30   # CDN caches 60 s, browsers
revalidate
Cache-Control: private, no-store                      # never cache (user-specific data)
Vary: Accept-Encoding                                 # separate cache entries per encoding
```

- The **cache key** is usually host + path + selected query strings/headers. Including too much (all cookies) destroys hit ratio; including too little leaks one user's content to another.
- **Invalidation** is slow and sometimes costly; prefer versioned filenames for assets.
- Restrict the origin to accept traffic only from the CDN (AWS CloudFront managed prefix list in security groups, origin access control for S3, or a secret header), otherwise attackers bypass the CDN and WAF.

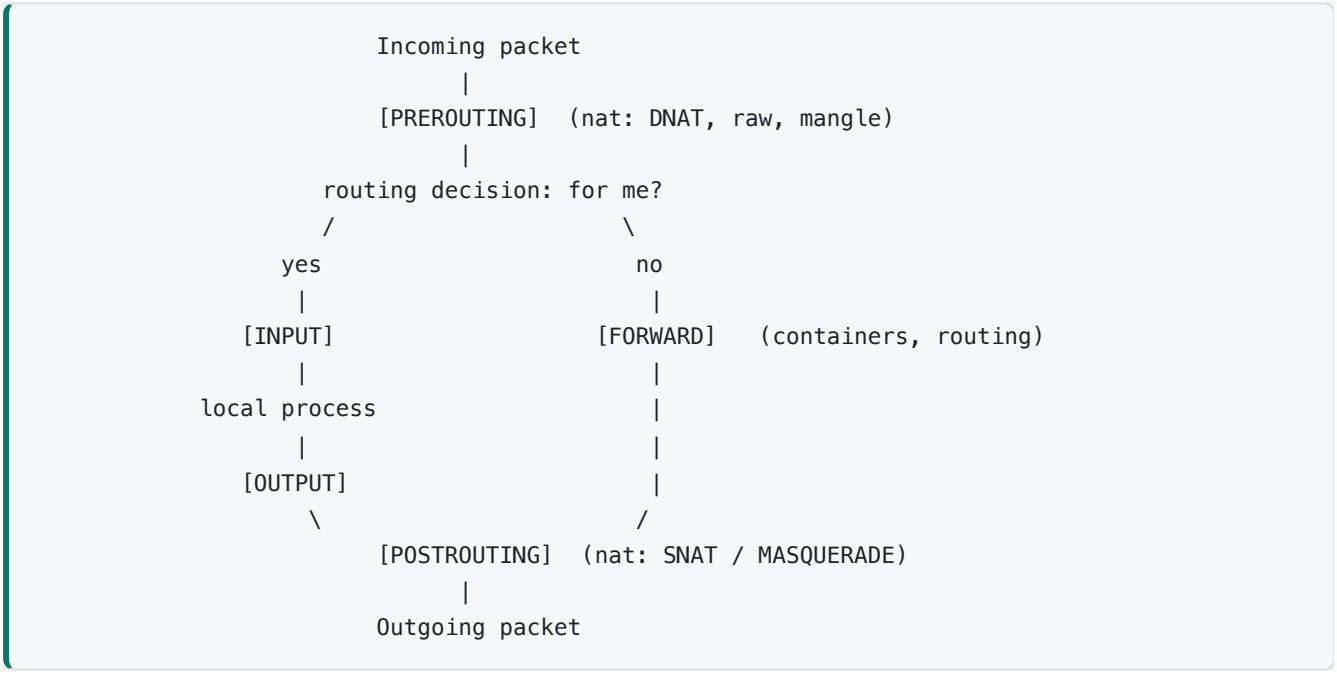
6.5 Firewalls

6.5.1 Stateless vs stateful

- **Stateless** (packet filter): each packet is evaluated alone. To allow a client to reach a server on 443, you must also allow return traffic to the client's **ephemeral port range** (1024–65535).
- **Stateful**: the firewall tracks connections (conntrack). If outbound SYN was allowed, the replies are automatically allowed. Nearly all modern firewalls are stateful.
- **Next-generation / L7 firewalls**: inspect application protocols, TLS SNI, URLs, and identity.

6.5.2 Linux: netfilter, iptables, nftables

Netfilter hooks in the Linux kernel process every packet. `iptables` is the classic frontend; `nftables` is its modern replacement (most distros now run `iptables-nft`, which translates iptables commands to nftables).



A minimal host firewall with nftables:

```
cat > /etc/nftables.conf <<'EOF'
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif lo accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept
    tcp dport 22 ip saddr 10.0.0.0/8 accept
    tcp dport { 80, 443 } accept
    counter log prefix "nft-drop: " drop
  }
```

```
}
EOF
nft -f /etc/nftables.conf
nft list ruleset
```

The equivalent iptables approach:

```
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -s 10.0.0.0/8 -j ACCEPT
iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT
iptables -P INPUT DROP
iptables -L -n -v --line-numbers     # counters show which rules match
```

> **Warning:** Set the established/related rule and your SSH allow rule **before** changing the default policy to DROP, or you will lock yourself out of a remote machine. On cloud instances, prefer security groups for perimeter control and keep host firewalls simple.

6.5.3 Security groups vs network ACLs (AWS)

	Security Group	Network ACL
Attached to	ENI (instance, pod ENI, LB, RDS)	Subnet
State	**Stateful**: return traffic automatically allowed	**Stateless**: return traffic needs explicit rules
Rules	Allow only	Allow and deny
Evaluation	All rules evaluated together	Numbered, lowest first, first match wins
Source can be	CIDR, prefix list, **another security group**	CIDR only
Default	New SG: deny all inbound, allow all outbound	Default NACL: allow all
Typical use	Primary workload firewall	Coarse subnet guardrails, explicit denies (block an IP range)

Security-group-to-security-group references are powerful: "allow 5432 from `sg-app`" means any ENI with `sg-app` attached, regardless of IP, which survives autoscaling.

NACL ephemeral port trap:

```
Inbound to web subnet:   rule 100 allow TCP 443 from 0.0.0.0/0
Outbound from subnet:    rule 100 allow TCP 1024-65535 to 0.0.0.0/0   <- replies to clients'
ephemeral ports
And for instances calling out (e.g., to an API on 443):
```

```
Outbound:                rule 110 allow TCP 443 to 0.0.0.0/0
Inbound:                 rule 110 allow TCP 1024-65535 from 0.0.0.0/0  <- replies coming back
```

Forget the ephemeral rules and connections hang (timeouts, not refusals), which is why NACL mistakes are so hard to spot.

6.5.4 Web Application Firewalls (preview)

A WAF inspects HTTP requests for attacks (SQL injection, XSS, known bad bots) and enforces rate limits. It sits at the CDN or L7 load balancer. Part 9 covers WAF and DDoS in more detail.

6.6 VPNs

A VPN builds an encrypted tunnel across an untrusted network so that two private networks (site-to-site) or a user and a network (remote access) can communicate as if directly connected.

6.6.1 IPsec

IPsec is the standard for site-to-site VPNs and cloud VPN gateways.

- **IKE** (Internet Key Exchange, UDP 500, and UDP 4500 with NAT traversal) authenticates peers (pre-shared key or certificates) and negotiates keys. IKEv2 is the current version.
- **Phase 1** builds the IKE SA (a secure control channel). **Phase 2** builds the IPsec SAs (child SAs) that carry data.
- **ESP** (IP protocol 50) encrypts the payload. **AH** (protocol 51) authenticates only and is rarely used.
- **Tunnel mode** encrypts the whole original packet and adds a new IP header (site-to-site). **Transport mode** encrypts only the payload.
- **Policy-based** VPNs define “interesting traffic” by selectors; **route-based** VPNs create a tunnel interface and use routing (static or BGP). Cloud VPNs are route-based with BGP for dynamic failover.

```
On-prem 192.168.0.0/16                                    AWS VPC 10.0.0.0/16
[Router/FW] ==== IPsec tunnel 1 (BGP) ==== [AWS VPN endpoint A] --+
     \======== IPsec tunnel 2 (BGP) ====== [AWS VPN endpoint B] --+-- VGW / Transit Gateway
Each AWS Site-to-Site VPN connection = 2 tunnels for redundancy, ~1.25 Gbit/s per tunnel
```

Common IPsec problems: mismatched proposals (encryption/hash/DH group), mismatched traffic selectors, UDP 500/4500 or ESP blocked, overlapping CIDRs, and MTU (IPsec overhead of 50–80 bytes; clamp TCP MSS to around 1360–1379).

6.6.2 WireGuard

WireGuard is a modern, minimal VPN built into the Linux kernel. It uses a fixed, modern crypto suite (Curve25519, ChaCha20-Poly1305), runs over a single UDP port (51820 by convention), and identifies peers by public key. Tailscale and many mesh VPNs are built on it.

```
# /etc/wireguard/wg0.conf on server (public IP 203.0.113.50)
[Interface]
Address = 10.200.0.1/24
ListenPort = 51820
PrivateKey = <server-private-key>

[Peer]
# laptop
PublicKey = <laptop-public-key>
AllowedIPs = 10.200.0.2/32
```

```
# /etc/wireguard/wg0.conf on laptop
[Interface]
Address = 10.200.0.2/24
PrivateKey = <laptop-private-key>
DNS = 10.0.0.2

[Peer]
PublicKey = <server-public-key>
Endpoint = 203.0.113.50:51820
AllowedIPs = 10.200.0.0/24, 10.0.0.0/16   # routes sent into the tunnel
PersistentKeepalive = 25                  # keep NAT mappings alive
```

```
wg genkey | tee privatekey | wg pubkey > publickey
sudo wg-quick up wg0
sudo wg show
```

```
interface: wg0
  public key: kV3...=
  listening port: 51820

peer: 9Hq...=
  endpoint: 198.51.100.7:41200
  allowed ips: 10.200.0.2/32
  latest handshake: 12 seconds ago
  transfer: 18.42 MiB received, 2.10 MiB sent
```

`AllowedIPs` does double duty: on send it acts as a routing table (which peer gets this destination), on receive it acts as an ACL (which source IPs are accepted from this peer). If “latest handshake” never appears, the UDP port is blocked or keys are wrong; WireGuard is silent by design and will not reply to unauthenticated packets.

	IPsec (IKEv2)	WireGuard	OpenVPN
Transport	UDP 500/4500, ESP	UDP (one port)	UDP or TCP (often 1194/443)
Complexity	High, many options	Very low	Medium

	IPsec (IKEv2)	WireGuard	OpenVPN
Performance	High (hardware offload)	Very high	Lower (user space)
Interop	Universal (routers, clouds)	Linux/BSD/mobile, few hardware appliances	Wide client support
Typical use	Site-to-site, cloud VPN	Remote access, mesh, server-to-server	Legacy remote access

6.7 Lab: Load Balancing with HAProxy and Failure Injection

Step 1. Create a Docker network and three backends that identify themselves:

```
docker network create lbnet
for i in 1 2 3; do
  docker run -d --name web$i --network lbnet hashicorp/http-echo:0.2.3 -listen=:8080 -
text="backend web$i"
done
```

Step 2. Write an HAProxy config:

```
mkdir -p ~/lblab && cat > ~/lblab/haproxy.cfg <<'EOF'
global
    log stdout format raw local0
defaults
    mode http
    log global
    option httplog
    timeout connect 2s
    timeout client 30s
    timeout server 30s
frontend fe
    bind :80
    default_backend be
backend be
    balance roundrobin
    option httpchk GET /
    server web1 web1:8080 check inter 2s fall 2 rise 2
    server web2 web2:8080 check inter 2s fall 2 rise 2
    server web3 web3:8080 check inter 2s fall 2 rise 2
frontend stats
    bind :8404
    stats enable
    stats uri /
EOF
docker run -d --name lb --network lbnet -p 8080:80 -p 8404:8404 \
  -v ~/lblab/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro haproxy:3.0
```

Step 3. Observe round robin:

```
for i in $(seq 6); do curl -s localhost:8080; done
```

```
backend web1
backend web2
backend web3
backend web1
backend web2
backend web3
```

Step 4. Kill a backend and watch the health check remove it (about 4 s with `inter 2s fall 2`):

```
docker stop web2
docker logs lb 2>&1 | grep -i 'web2'
for i in $(seq 4); do curl -s localhost:8080; done
```

```
Server be/web2 is DOWN, reason: Layer4 timeout, check duration: 2001ms. 2 active and 0 backup
servers left.
backend web1
backend web3
backend web1
backend web3
```

Open `http://localhost:8404/` to see the stats page.

Step 5. Change `balance roundrobin` to `balance source`, restart the LB (`docker restart lb`), and note that all your requests now land on the same backend (source-IP stickiness).

Step 6. Bring web2 back with `docker start web2` and watch it return after two successful checks (`rise 2`).

Step 7. Clean up: `docker rm -f lb web1 web2 web3 && docker network rm lbnet`.

Key takeaways

- L4 balancers route connections by 5-tuple (fast, protocol-agnostic); L7 balancers route requests by HTTP attributes and terminate TLS.
- Choose algorithms deliberately: least-connections or P2C for variable workloads, consistent hashing for caches.
- Health checks should reflect the instance's own ability to serve, not its dependencies; drain connections during deploys.
- Order timeouts so upstream keep-alive timeouts exceed downstream idle timeouts, or you will get intermittent 502s.
- API gateways add auth, rate limiting, and transformation on top of reverse proxying; CDNs cache at the edge and are controlled by Cache-Control and cache keys.

- Stateful firewalls track connections; stateless ones (NACLs) need explicit ephemeral-port return rules. Security groups are stateful, allow-only, and can reference other groups.
- IPsec (IKE UDP 500/4500, ESP) is the site-to-site standard; WireGuard is a simple, fast UDP VPN where AllowedIPs is both route table and ACL.

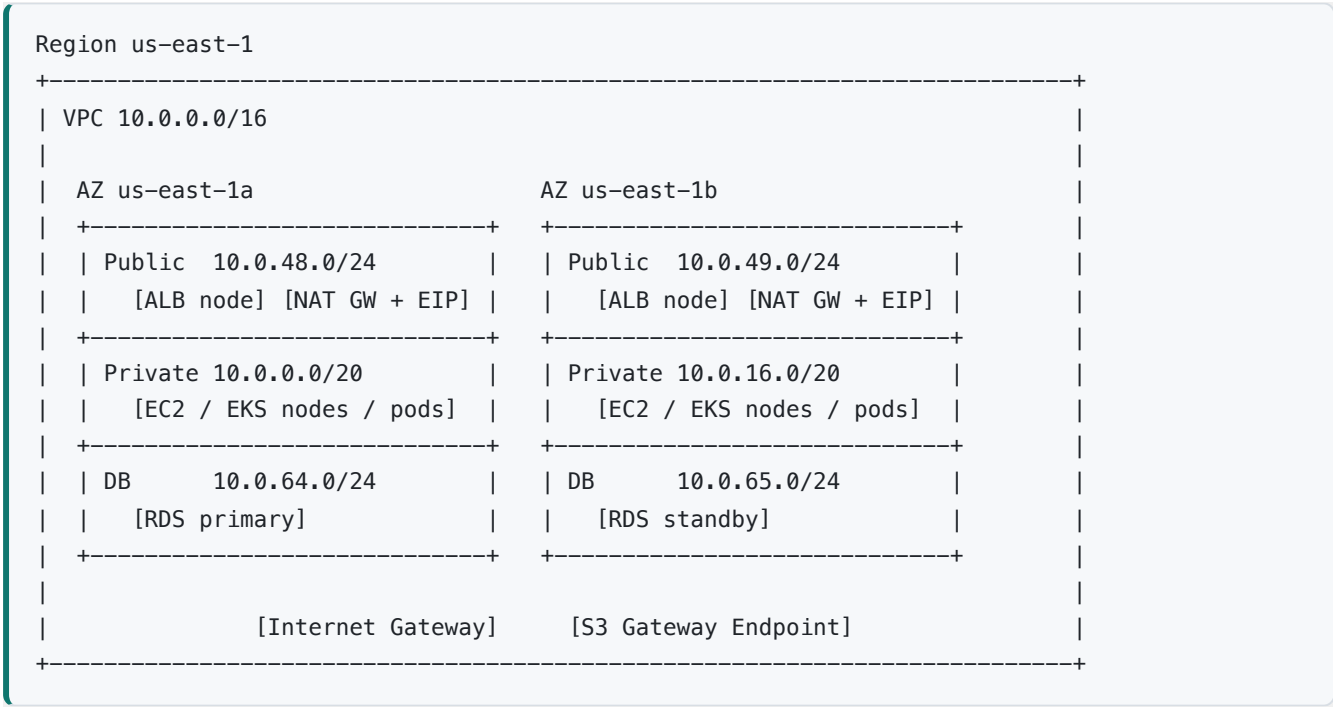
Part 7 — Cloud Networking: AWS VPC in Depth, Azure and GCP, Terraform

Cloud networks look like the networks from Parts 1–6 (subnets, routes, gateways, firewalls), but they are implemented as software in the provider's hypervisors. There is no real broadcast, ARP is answered by the hypervisor, and every "device" is an API object. This Part uses AWS as the deep example because its concepts map cleanly onto the other providers, then shows Azure and GCP equivalents and builds a production-style VPC with Terraform.

7.1 AWS VPC Building Blocks

7.1.1 VPC

A **VPC** (Virtual Private Cloud) is a logically isolated network in one **region**. You give it a primary IPv4 CIDR between /16 and /28 (from RFC 1918 or other ranges) and can add secondary CIDRs and an IPv6 /56.



7.1.2 Subnets

- A subnet lives in exactly **one Availability Zone**. High availability means at least one subnet per AZ per tier.
- AWS reserves 5 addresses per subnet (Part 2): in `10.0.48.0/24`, `.0` network, `.1` VPC router, `.2` DNS resolver, `.3` reserved, `.255` last.
- "Public" and "private" are not subnet settings. **A subnet is public if its route table has a route to an Internet Gateway.** That is the whole definition.

7.1.3 Route tables

Each subnet is associated with one route table (the **main** route table if you do not choose). Every route table has an unremovable **local** route for the VPC CIDR.

```
Public route table                      Private route table (AZ-a)
Destination      Target                 Destination      Target
10.0.0.0/16      local                  10.0.0.0/16      local
0.0.0.0/0        igw-0abc               0.0.0.0/0        nat-0aaa (NAT GW in AZ-a)
::/0             igw-0abc               pl-63a5400a (S3) vpce-0s3 (gateway endpoint)
                                        10.50.0.0/16     tgw-0123 (Transit Gateway to on-
prem)
```

Longest prefix match applies (Part 3). Per-AZ private route tables let each AZ use its own NAT gateway, avoiding cross-AZ dependency and data charges.

7.1.4 Internet Gateway (IGW)

The IGW is a horizontally scaled, highly available VPC attachment that does two things: it is the target for Internet-bound routes, and it performs **1:1 NAT** between an instance's private IP and its public IP/Elastic IP. The instance's OS never sees its public IP (`ip addr` shows only the private address); that mapping lives in the IGW. For Internet access, an instance needs all three: a public IP (or EIP), a route to the IGW, and permissive SG/NACL rules.

7.1.5 NAT Gateway

A NAT gateway lets instances in private subnets initiate outbound IPv4 connections while remaining unreachable from the Internet (PAT, Part 2).

- Deploy it in a **public** subnet; point private subnets' `0.0.0.0/0` at it.
- It is AZ-scoped: deploy **one per AZ** for resilience. A single NAT gateway is a single point of failure and incurs cross-AZ data charges.
- It is billed per hour **and per GB processed**. Large pulls through NAT (container images, S3 traffic, package mirrors) can cost more than the compute. Use VPC endpoints for AWS services.
- Idle timeout is 350 s; about 55,000 simultaneous connections per unique destination per public IP (you can add secondary IPs).
- For IPv6, use an **egress-only Internet gateway** instead.

7.1.6 Elastic Network Interfaces and Elastic IPs

An **ENI** is a virtual NIC with a primary private IP, optional secondary IPs or prefixes, a MAC, security groups, and a source/destination check flag. EKS's VPC CNI attaches ENIs and secondary IPs (or /28 prefixes with prefix delegation) to nodes and gives them to pods. To run a NAT instance, firewall appliance, or VPN router on EC2, **disable the source/destination check** or the hypervisor drops traffic not addressed to the ENI.

An **Elastic IP** is a static public IPv4 address you own until you release it. All public IPv4 addresses (including auto-assigned ones) are billed hourly.

7.1.7 Security Groups and NACLs in the VPC

Part 6 compared them in detail. A typical three-tier design:

Security group	Inbound	Source
`sg-alb`	TCP 443	0.0.0.0/0 (or CloudFront prefix list)
`sg-app`	TCP 8080	`sg-alb`
`sg-db`	TCP 5432	`sg-app`

Keep NACLs at their default allow-all unless you have a specific need (blocking a CIDR, compliance boundaries). Debugging stateless rules across many subnets is expensive.

7.1.8 VPC DNS

The **Route 53 Resolver** runs at the VPC base address +2 (`10.0.0.2`) and also at `169.254.169.253`. It resolves public names, private hosted zones associated with the VPC, and AWS internal names (`ip-10-0-1-25.ec2.internal`). Two VPC attributes must be enabled for private DNS and interface endpoints to work: `enableDnsSupport` and `enableDnsHostnames`.

7.2 Connecting VPCs and Networks

7.2.1 VPC Peering

A peering connection is a private, non-transitive link between two VPCs (same or different account/region).

```
VPC A 10.0.0.0/16 <==peering==> VPC B 10.1.0.0/16 <==peering==> VPC C 10.2.0.0/16
A cannot reach C through B. Peering is NOT transitive.
```

Rules: CIDRs must not overlap; you must add routes in **both** VPCs' route tables (`10.1.0.0/16 -> pcx-...`); security groups must allow the peer's CIDR (or reference peer SGs in the same region); no edge-to-edge routing (A cannot use B's NAT gateway, IGW, or VPN). A full mesh of N VPCs needs N(N-1)/2 peerings, which does not scale beyond a handful.

7.2.2 Transit Gateway

A **Transit Gateway** (TGW) is a regional, managed router that VPCs, VPNs, Direct Connect gateways, and other TGWs (via peering) attach to. It replaces a mesh with a hub-and-spoke.

```
            +--------------------- Transit Gateway ----------------------+
            |  RT "prod":  10.0.0.0/16 -> att-prod   10.50.0.0/16 -> vpn |
            |  RT "shared": all VPC CIDRs propagated                     |
            +-----+--------------+---------------+-------------+--------+
                  |              |               |             |
```

```
   VPC prod        VPC staging     VPC shared      Site-to-Site VPN
 10.0.0.0/16     10.10.0.0/16    10.20.0.0/16    on-prem 10.50.0.0/16
```

Concepts:

- **Attachment**: a VPC (via an ENI in a subnet per AZ), VPN, Direct Connect gateway, or peering.
- **Association**: each attachment is associated with one TGW route table, which is used to route traffic *coming from* that attachment.
- **Propagation**: an attachment can automatically install its CIDRs into one or more TGW route tables.
- Segmentation comes from multiple TGW route tables: prod and staging both reach shared services but not each other.
- VPC route tables still need routes pointing at the TGW (`10.0.0.0/8 -> tgw-...`).
- Centralized egress: route all VPCs' `0.0.0.0/0` to an egress VPC with NAT gateways or a firewall (AWS Network Firewall or Gateway Load Balancer appliances).

7.2.3 Hybrid connectivity

Option	What	Notes
Site-to-Site VPN	IPsec over the Internet, 2 tunnels, BGP or static	Quick, cheap, about 1.25 Gbit/s per tunnel, Internet latency
Direct Connect	Dedicated private fiber to an AWS location	Consistent latency, 1–100 Gbit/s, weeks to provision; not encrypted by default (use MACsec or VPN over DX)
Client VPN	Managed OpenVPN-based remote access	User access

7.2.4 VPC Endpoints and PrivateLink

Endpoints let private resources reach AWS services (or third-party services) **without** an IGW or NAT gateway, keeping traffic on the AWS network.

	Gateway endpoint	Interface endpoint (PrivateLink)
Services	S3 and DynamoDB only	Most AWS services, SaaS, your own services
How	Route table entry with a prefix list	ENI with a private IP in your subnets
DNS	Normal public names, routed privately	Private DNS overrides the service name to the ENI IPs
Security	Endpoint policy	Endpoint policy + security group on the ENI
Cost	Free	Hourly per AZ + per GB
Reach from on-prem/peered VPC	No	Yes

PrivateLink endpoint services let you expose your own service (behind an NLB or Gateway Load Balancer) to consumers in other VPCs or accounts, one direction only, even with **overlapping CIDRs**, because the consumer only sees an ENI in its own VPC.

```
Consumer VPC 10.0.0.0/16                          Provider VPC 10.0.0.0/16 (overlap is fine)
 [app] --> [Interface endpoint ENI 10.0.3.41] ===PrivateLink===> [NLB] --> [service]
```

> **Tip:** Add gateway endpoints for S3 and DynamoDB to every VPC on day one. They are free, reduce NAT gateway data charges, and are a common answer to "why is our NAT bill so high?". For EKS in private subnets, interface endpoints for ECR (api and dkr), STS, and CloudWatch Logs let nodes work without any Internet egress.

7.2.5 Observability: Flow Logs and Reachability Analyzer

VPC Flow Logs record accepted and rejected flows per ENI, subnet, or VPC:

```
version account-id   interface-id          srcaddr    dstaddr    srcport dstport proto pkts
bytes start      end        action log-status
2       123456789012 eni-0a1b2c3d4e5f67890 10.0.1.25  10.0.64.10 51544   5432    6     10
840   1760263200 1760263260 ACCEPT OK
2       123456789012 eni-0a1b2c3d4e5f67890 198.51.100.9 10.0.1.25 44211  22      6     3
180   1760263200 1760263260 REJECT OK
```

`REJECT` means a security group or NACL dropped it. If you see an `ACCEPT` inbound but a `REJECT` for the return flow, suspect a NACL (stateless). Flow logs do not show DNS queries to the Route 53 Resolver (use Resolver query logs), metadata service traffic, or packet contents.

Reachability Analyzer statically analyzes the configuration between two resources and tells you which component (route, SG, NACL) blocks a path, without sending packets. Use it before reaching for tcpdump.

7.3 Azure and GCP Equivalents

Concept	AWS	Azure	GCP
Private network	VPC (regional)	VNet (regional)	VPC (**global**; subnets are regional)
Subnet scope	AZ	Region (spans zones)	Region (spans zones)
Instance firewall	Security Group (stateful)	NSG (stateful, allow and deny, priorities) on NIC or subnet; ASGs for grouping	VPC firewall rules / policies (stateful), targeted by network tags or service accounts
Subnet stateless ACL	NACL	(none; NSG at subnet)	(none)
Custom routing	Route tables	User-Defined Routes (UDR)	Routes (with tags), Cloud Router for dynamic

Concept	AWS	Azure	GCP
Internet egress NAT	NAT Gateway	NAT Gateway	Cloud NAT (software-defined, no appliance)
Network peering	VPC Peering	VNet Peering (non-transitive)	VPC Network Peering (non-transitive)
Hub router	Transit Gateway	Virtual WAN / hub-spoke with Azure Firewall or NVA	Network Connectivity Center
Private service access	PrivateLink / interface endpoints	Private Endpoint / Private Link	Private Service Connect
Shared networking across accounts	RAM-shared subnets	(subscriptions + peering)	Shared VPC (host and service projects)
Dedicated link	Direct Connect	ExpressRoute	Cloud Interconnect
Managed DNS	Route 53	Azure DNS / Private DNS zones	Cloud DNS
L7 LB	ALB	Application Gateway, Front Door (global)	Global external Application LB
L4 LB	NLB	Azure Load Balancer	Network LB / passthrough
Flow logs	VPC Flow Logs	NSG / VNet flow logs	VPC Flow Logs

Notable differences:

- **GCP VPCs are global**: one VPC can have subnets in every region, and instances communicate across regions over Google's backbone without peering.
- **Azure reserves 5 IPs** per subnet and requires some services to have dedicated subnets (`GatewaySubnet`, `AzureFirewallSubnet`, delegated subnets).
- **GCP firewall rules** apply to the whole VPC and target instances by tag or service account, rather than being attached to interfaces.

7.4 Terraform: A Production-Style VPC

The configuration below builds the design from Section 7.1: three AZs, public/private/database subnets, an IGW, one NAT gateway per AZ, per-AZ private route tables, an S3 gateway endpoint, flow logs, and tiered security groups. It uses `cidrsubnet()` so the plan is derived from one CIDR.

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.70" }
  }
}
```

```
provider "aws" {
  region = var.region
}

variable "region"   { default = "us-east-1" }
variable "vpc_cidr" { default = "10.0.0.0/16" }
variable "name"     { default = "prod" }

data "aws_availability_zones" "available" { state = "available" }

locals {
  azs = slice(data.aws_availability_zones.available.names, 0, 3)
  # /20 private subnets: 10.0.0.0/20, 10.0.16.0/20, 10.0.32.0/20
  private_cidrs = [for i in range(3) : cidrsubnet(var.vpc_cidr, 4, i)]
  # /24 public subnets:  10.0.48.0/24, 10.0.49.0/24, 10.0.50.0/24
  public_cidrs  = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, 48 + i)]
  # /24 database subnets: 10.0.64.0/24, 10.0.65.0/24, 10.0.66.0/24
  db_cidrs      = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, 64 + i)]
}

resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = var.name }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags   = { Name = "${var.name}-igw" }
}

resource "aws_subnet" "public" {
  count                   = 3
  vpc_id                  = aws_vpc.this.id
  cidr_block              = local.public_cidrs[count.index]
  availability_zone       = local.azs[count.index]
  map_public_ip_on_launch = false
  tags = { Name = "${var.name}-public-${local.azs[count.index]}", "kubernetes.io/role/elb" =
"1" }
}

resource "aws_subnet" "private" {
  count             = 3
  vpc_id            = aws_vpc.this.id
  cidr_block        = local.private_cidrs[count.index]
  availability_zone = local.azs[count.index]
  tags = { Name = "${var.name}-private-${local.azs[count.index]}",
"kubernetes.io/role/internal-elb" = "1" }
}

resource "aws_subnet" "db" {
  count             = 3
```

```
  vpc_id            = aws_vpc.this.id
  cidr_block        = local.db_cidrs[count.index]
  availability_zone = local.azs[count.index]
  tags = { Name = "${var.name}-db-${local.azs[count.index]}" }
}

# Public routing: default route to the IGW
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = { Name = "${var.name}-public" }
}

resource "aws_route_table_association" "public" {
  count          = 3
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

# One NAT gateway per AZ
resource "aws_eip" "nat" {
  count  = 3
  domain = "vpc"
  tags   = { Name = "${var.name}-nat-${local.azs[count.index]}" }
}

resource "aws_nat_gateway" "this" {
  count         = 3
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags          = { Name = "${var.name}-nat-${local.azs[count.index]}" }
  depends_on    = [aws_internet_gateway.this]
}

# Per-AZ private route tables, each using its own AZ's NAT gateway
resource "aws_route_table" "private" {
  count  = 3
  vpc_id = aws_vpc.this.id
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.this[count.index].id
  }
  tags = { Name = "${var.name}-private-${local.azs[count.index]}" }
}

resource "aws_route_table_association" "private" {
  count          = 3
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}
```

```
# Database subnets: no Internet route at all
resource "aws_route_table" "db" {
  vpc_id = aws_vpc.this.id
  tags   = { Name = "${var.name}-db" }
}

resource "aws_route_table_association" "db" {
  count          = 3
  subnet_id      = aws_subnet.db[count.index].id
  route_table_id = aws_route_table.db.id
}

# Free S3 gateway endpoint on private and db route tables
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.this.id
  service_name      = "com.amazonaws.${var.region}.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids   = concat(aws_route_table.private[*].id, [aws_route_table.db.id])
}

# Tiered security groups
resource "aws_security_group" "alb" {
  name   = "${var.name}-alb"
  vpc_id = aws_vpc.this.id
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app" {
  name   = "${var.name}-app"
  vpc_id = aws_vpc.this.id
  ingress {
    from_port       = 8080
    to_port         = 8080
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
}

resource "aws_security_group" "db" {
  name   = "${var.name}-db"
  vpc_id = aws_vpc.this.id
  ingress {
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]
  }
}

# Flow logs to CloudWatch (IAM role omitted for brevity; see aws_flow_log docs)
# resource "aws_flow_log" "this" { ... traffic_type = "ALL" ... }

output "vpc_id"             { value = aws_vpc.this.id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }
output "public_subnet_ids"  { value = aws_subnet.public[*].id }
output "subnet_plan" {
  value = { public = local.public_cidrs, private = local.private_cidrs, db = local.db_cidrs }
}
```

> **Note:** For real projects, the community module `terraform-aws-modules/vpc/aws` implements all of this with options for single vs per-AZ NAT, IPv6, endpoints, and flow logs. Writing it by hand once is the best way to understand what that module does.

7.5 Lab: Build and Verify the VPC

You need an AWS account, credentials with VPC/EC2 permissions, and Terraform 1.6+. NAT gateways cost money per hour; destroy the lab when finished.

Step 1. Verify the subnet math without touching AWS:

```
terraform console <<'EOF'
[for i in range(3) : cidrsubnet("10.0.0.0/16", 4, i)]
[for i in range(3) : cidrsubnet("10.0.0.0/16", 8, 48 + i)]
EOF
```

```
tolist([
  "10.0.0.0/20",
  "10.0.16.0/20",
  "10.0.32.0/20",
])
tolist([
  "10.0.48.0/24",
  "10.0.49.0/24",
```

```
  "10.0.50.0/24",
])
```

Step 2. Save the configuration as `main.tf`, then:

```
terraform init
terraform plan -out tfplan
terraform apply tfplan
```

Step 3. Inspect what was created:

```
VPC=$(terraform output -raw vpc_id)
aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC \
  --query 'Subnets[].[Tags[?Key==`Name`]|
[0].Value,CidrBlock,AvailabilityZone,AvailableIpAddressCount]' --output table
aws ec2 describe-route-tables --filters Name=vpc-id,Values=$VPC \
  --query 'RouteTables[].{name:Tags[?Key==`Name`]|[0].Value,routes:Routes[].
[DestinationCidrBlock||DestinationPrefixListId,GatewayId||NatGatewayId]}'
```

Note `AvailableIpAddressCount` of 251 for each /24 and 4,091 for each /20 (5 reserved).

Step 4. Launch a test instance in a private subnet with SSM access (no SSH, no public IP) and verify egress:

```
# from an SSM session on the private instance
curl -s https://checkip.amazonaws.com      # prints the NAT gateway's EIP for that AZ
ip route                                   # default via 10.0.x.1 (the VPC router)
dig +short s3.us-east-1.amazonaws.com      # public S3 IPs, but routed through the gateway
endpoint
```

Compare the printed IP with `aws ec2 describe-nat-gateways`.

Step 5. Use Reachability Analyzer to check that the app SG can reach the DB SG on 5432 but the Internet cannot (Console: VPC > Reachability Analyzer, source = IGW, destination = an ENI in the DB subnet). The analysis should report “not reachable” and name the missing route.

Step 6. Destroy everything: `terraform destroy`.

Key takeaways

- A VPC is regional; subnets are AZ-scoped; a subnet is public only because its route table points to an IGW.
- The IGW performs 1:1 NAT for public IPs; NAT gateways provide outbound-only PAT for private subnets and should be one per AZ.
- NAT gateway data charges are often avoidable: use free S3/DynamoDB gateway endpoints and interface endpoints for other services.

- Peering is non-transitive and needs non-overlapping CIDRs and routes on both sides; Transit Gateway provides hub-and-spoke routing with route-table segmentation.
- PrivateLink exposes a service through an ENI in the consumer VPC and tolerates overlapping CIDRs.
- Flow logs show ACCEPT/REJECT per flow; Reachability Analyzer finds blocking configuration without sending packets.
- Azure and GCP have direct equivalents; GCP's VPC is global and its firewall rules target tags or service accounts.
- Derive subnet plans with `cidrsubnet()` so the whole layout follows from one CIDR.

Part 8 — Container Networking, Kubernetes Networking, and Service Mesh

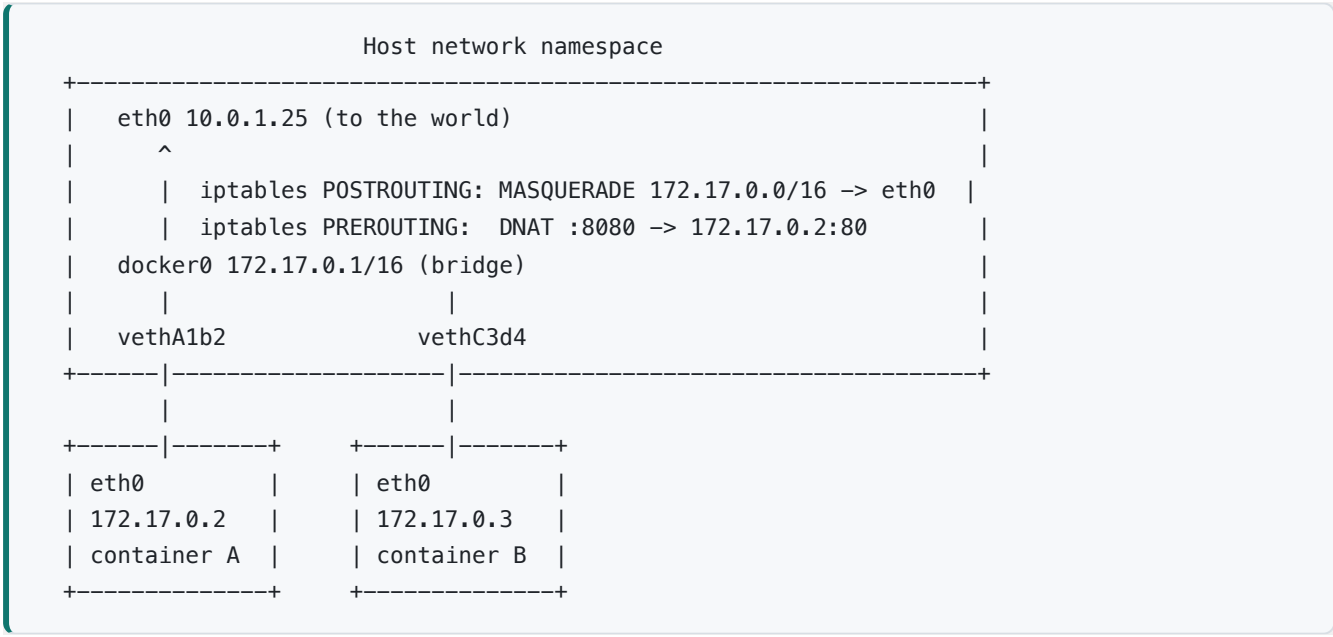
Container networking is not new networking; it is the namespaces, veth pairs, bridges, routes, and NAT rules from Parts 1–3, automated. Once you see that, Kubernetes networking stops being magic. This Part goes from a single Docker host to Kubernetes Services, Ingress and Gateway API, NetworkPolicy, CoreDNS, and service meshes.

8.1 Docker Networking

8.1.1 The default bridge

When Docker starts, it creates a Linux bridge `docker0` (default `172.17.0.1/16`). For each container it:

1. Creates a network namespace for the container.
2. Creates a **veth pair**; one end becomes `eth0` inside the container, the other is attached to `docker0`.
3. Assigns an IP from the bridge subnet and sets the container's default route to `172.17.0.1`.
4. Adds iptables rules for NAT.



8.1.2 Seeing it on the host

```
docker run -d --name web -p 8080:80 nginx:1.27
ip link show master docker0
docker inspect -f '{{.NetworkSettings.IPAddress}}' web
sudo iptables -t nat -S | grep -E 'MASQUERADE|DNAT'
```

```
7: veth4f1c2a9@if6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0
state UP
172.17.0.2
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER ! -i docker0 -p tcp -m tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:80
```

The `@if6` suffix means the veth's peer is interface index 6 in another namespace (the container's `eth0`). Read the two rules:

- **MASQUERADE**: traffic from containers leaving via anything other than docker0 gets the host's IP as source (outbound PAT).
- **DNAT**: traffic arriving on host port 8080 has its destination rewritten to the container. This is all `-p 8080:80` does.

> **Warning:** Docker's published ports are inserted into the `nat` table and the `DOCKER` chain, before your `INPUT` rules are consulted. A host firewall like UFW that only filters `INPUT` does **not** block published container ports. Use the `DOCKER-USER` chain for filtering, or publish on a specific address: `-p 127.0.0.1:8080:80`.

8.1.3 Network drivers

Driver	Behavior	Use
`bridge` (default)	Private bridge, NAT to outside	Single-host apps
User-defined bridge	Same, plus **embedded DNS** (127.0.0.11) for container-name resolution and isolation between networks	Compose projects (the default there)
`host`	No network namespace; shares host stack	Max performance, network tools; port conflicts possible
`none`	Loopback only	Isolation
`macvlan` / `ipvlan`	Container gets an address on the physical LAN	Legacy apps expecting L2 presence
`overlay`	VXLAN across hosts (Swarm)	Multi-host

On the default bridge, containers cannot resolve each other by name; on a user-defined network they can. That is why `docker compose` services find each other by service name.

8.1.4 Common Docker networking bugs

Symptom	Cause
`curl localhost:8080` from host fails, but container is running	App inside listens on `127.0.0.1` instead of `0.0.0.0`

Symptom	Cause
Container cannot reach a corporate network	Docker subnet overlaps with VPN/VPC CIDR; change `default-address-pools`
Container cannot reach the Internet	`net.ipv4.ip_forward=0`, or FORWARD chain policy/rules blocking
Large downloads hang inside containers on a VPN or overlay	MTU mismatch; set Docker's `mtu` option or clamp MSS
`host.docker.internal` does not resolve on Linux	Add `--add-host=host.docker.internal:host-gateway`

8.2 The Kubernetes Network Model

Kubernetes defines requirements, not an implementation:

1. Every **pod gets its own IP** address.
2. Pods can communicate with all other pods on any node **without NAT**.
3. Agents on a node (kubelet) can communicate with all pods on that node.
4. Containers in a pod share one network namespace (they reach each other via `localhost`).

There are four distinct communication problems, solved by different components:

Problem	Solved by
Container to container in a pod	Shared network namespace (the `pause` container holds it)
Pod to pod	**CNI plugin**
Pod to Service	**kube-proxy** (or eBPF replacement) and CoreDNS
External to Service	NodePort / LoadBalancer / Ingress / Gateway

Three separate CIDRs are configured in every cluster and must not overlap with each other or with networks the cluster must reach:

```
Node network      10.0.0.0/16     (VPC subnets; node IPs)
Pod network       10.244.0.0/16   (per-node slices, e.g., node1 10.244.1.0/24)
Service network   10.96.0.0/12    (virtual IPs; exist only in kube-proxy rules)
```

On AWS EKS with the VPC CNI, pods get IPs **from the VPC subnets** directly, so pod CIDR = node subnets. That is simple and fast but consumes VPC addresses quickly; mitigate with prefix delegation, secondary CIDRs in `100.64.0.0/10` (custom networking), or IPv6.

8.3 CNI Plugins

CNI (Container Network Interface) is a spec: when a pod is created, the container runtime calls a CNI plugin binary with `ADD`, and the plugin creates interfaces, assigns an IP (via IPAM), and sets routes. On `DEL`, it cleans up. Configuration lives in `/etc/cni/net.d/` and binaries in `/opt/cni/bin/`.

8.3.1 How pod-to-pod across nodes works

Two general approaches:

```
1) Overlay (encapsulation): VXLAN / IP-in-IP / Geneve

 Node1 10.0.1.10                                  Node2 10.0.2.20
 [pod A 10.244.1.5]                               [pod B 10.244.2.7]
       |                                                 ^
   inner: 10.244.1.5 -> 10.244.2.7                       |
   outer: 10.0.1.10 -> 10.0.2.20 UDP 4789 =========> decapsulate
 Works on any underlay; costs MTU (50 bytes for VXLAN) and some CPU.

2) Native routing (no encapsulation)

 Node routes: 10.244.2.0/24 via 10.0.2.20 (installed by BGP or the cloud route table)
 Packets travel unmodified. Faster, but the underlay must know pod routes,
 or pod IPs must be native underlay IPs (AWS VPC CNI, Azure CNI, GKE VPC-native).
```

8.3.2 Popular CNIs

CNI	Data plane	Highlights
Flannel	VXLAN (or host-gw)	Simple, no NetworkPolicy by itself
Calico	iptables/nftables or eBPF; BGP native routing, IP-in-IP or VXLAN	Mature NetworkPolicy, BGP peering with physical network, global policies
Cilium	eBPF	kube-proxy replacement, L3–L7 policy, Hubble flow observability, transparent encryption (WireGuard/IPsec), BGP, Cluster Mesh, Gateway API
AWS VPC CNI	Native VPC IPs on ENIs	Pods are first-class VPC citizens; security groups for pods
Azure CNI (Overlay / Powered by Cilium)	VNet or overlay	Azure native
GKE Dataplane V2	Cilium-based eBPF	Default on GKE

> **Note:** **eBPF** lets programs run safely inside the kernel at hooks such as the network driver (XDP) and the traffic control layer. Cilium uses it to implement Service load balancing, policy, and observability without long iptables chains, which matters at scale: iptables rules are evaluated linearly, and clusters with thousands of Services have tens of thousands of rules.

8.4 Services

Pods are ephemeral; their IPs change. A **Service** provides a stable virtual IP and DNS name in front of a set of pods selected by labels. The control plane tracks ready pod IPs in **EndpointSlices**.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: shop
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - name: http
      port: 80          # Service port
      targetPort: 8080  # container port
```

8.4.1 Service types

Type	Reachable from	How
ClusterIP (default)	Inside the cluster	Virtual IP from the service CIDR
NodePort	Any node IP on a port in 30000–32767	ClusterIP + a port opened on every node
LoadBalancer	External, via a cloud or MetalLB LB	NodePort + ClusterIP + provisioned LB (or direct pod-IP targets with AWS LB Controller in IP mode)
ExternalName	Inside the cluster	DNS CNAME to an external name, no proxying
Headless (`clusterIP: None`)	Inside the cluster	DNS returns pod IPs directly; used by StatefulSets and client-side load balancing (gRPC)

8.4.2 kube-proxy: how a ClusterIP actually works

The ClusterIP is not assigned to any interface. It exists only as rules on every node. kube-proxy watches Services and EndpointSlices and programs the data plane.

```
Pod A (10.244.1.5) connects to web.shop (10.96.120.15:80)
   |
   v  netfilter on Pod A's node (OUTPUT/PREROUTING, nat table)
KUBE-SERVICES: dst 10.96.120.15:80 -> KUBE-SVC-WEB
KUBE-SVC-WEB:  random 1/3 -> KUBE-SEP-1 ; random 1/2 -> KUBE-SEP-2 ; else KUBE-SEP-3
KUBE-SEP-2:    DNAT to 10.244.2.7:8080
   |
   v  routed via CNI to node 2, conntrack remembers the mapping so replies are un-DNATed
Pod B (10.244.2.7:8080)
```

```
sudo iptables -t nat -S KUBE-SERVICES | grep 'shop/web'
sudo iptables -t nat -S KUBE-SVC-ABCDEF1234567890
```

```
-A KUBE-SERVICES -d 10.96.120.15/32 -p tcp -m comment --comment "shop/web:http cluster IP" -m
tcp --dport 80 -j KUBE-SVC-ABCDEF1234567890
-A KUBE-SVC-ABCDEF1234567890 -m comment --comment "shop/web:http -> 10.244.1.9:8080" -m
statistic --mode random --probability 0.33333333349 -j KUBE-SEP-AAAA
-A KUBE-SVC-ABCDEF1234567890 -m comment --comment "shop/web:http -> 10.244.2.7:8080" -m
statistic --mode random --probability 0.50000000000 -j KUBE-SEP-BBBB
-A KUBE-SVC-ABCDEF1234567890 -m comment --comment "shop/web:http -> 10.244.3.4:8080" -j KUBE-
SEP-CCCC
```

kube-proxy mode	Notes
iptables	Default for years; linear rule chains; random backend selection per connection
IPVS	Kernel L4 load balancer with hash tables, more algorithms (rr, lc, sh)
nftables	Newer default-candidate backend (GA in Kubernetes 1.33), more efficient updates
None (eBPF replacement)	Cilium or Calico eBPF handle Services directly

> **Interview:** “Why can’t I ping a ClusterIP?” Because the ClusterIP has no interface and kube-proxy only installs rules for the Service’s TCP/UDP ports. ICMP to the VIP matches nothing (with IPVS mode, the VIP is bound to a dummy interface so ping may answer, which confuses people further). Test Services with `curl` or `nc` on the port, never with ping.

Important consequence: **balancing is per connection**. HTTP/2 and gRPC clients keep one long-lived connection, so all their requests go to one pod. Use a headless Service with client-side balancing, or a mesh/L7 proxy that balances per request.

8.4.3 externalTrafficPolicy and source IPs

With the default `externalTrafficPolicy: Cluster`, a packet arriving at any node’s NodePort may be forwarded to a pod on another node, and is SNATed so the reply returns through the same node: the pod sees the node IP, not the client. With `externalTrafficPolicy: Local`, nodes only send to local pods (no

extra hop, **client IP preserved**), and nodes without a ready pod fail the LB health check so the LB avoids them.

8.5 Ingress and Gateway API

8.5.1 Ingress

An **Ingress** resource declares L7 HTTP routing rules (host, path, TLS); an **Ingress controller** (a reverse proxy such as Nginx, Traefik, HAProxy, or a cloud LB controller) implements them.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
spec:
  ingressClassName: alb
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend: { service: { name: api, port: { number: 80 } } }
          - path: /
            pathType: Prefix
            backend: { service: { name: web, port: { number: 80 } } }
```

Ingress's limitations (HTTP only, features hidden in controller-specific annotations, one resource mixing infrastructure and app concerns) led to Gateway API. The widely used community `ingress-nginx` controller was retired in 2026, which makes this a good time to standardize on Gateway API for new work.

8.5.2 Gateway API

Gateway API (GA since v1.0) splits responsibilities by role:

```
GatewayClass   (infrastructure provider)   "which implementation": Envoy Gateway, Cilium,
Istio, AWS, GKE...
     |
Gateway        (platform/cluster operator) listeners: ports, protocols, hostnames, TLS certs
     |
HTTPRoute / GRPCRoute / TLSRoute / TCPRoute   (application teams) routing rules, attached to
a Gateway
```

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public
  namespace: infra
spec:
  gatewayClassName: envoy
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      hostname: "*.example.com"
      tls:
        certificateRefs: [{ name: wildcard-example-com }]
      allowedRoutes:
        namespaces: { from: All }
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop
  namespace: shop
spec:
  parentRefs: [{ name: public, namespace: infra }]
  hostnames: ["shop.example.com"]
  rules:
    - matches: [{ path: { type: PathPrefix, value: /api } }]
      backendRefs:
        - { name: api-v1, port: 80, weight: 90 }
        - { name: api-v2, port: 80, weight: 10 }   # canary, portable across implementations
    - backendRefs: [{ name: web, port: 80 }]
```

Traffic splitting, header matching, redirects, and header modification are part of the standard API rather than annotations.

8.6 NetworkPolicy

By default, **all pods can talk to all pods** in every namespace. A **NetworkPolicy** selects pods and restricts their ingress and/or egress. It is enforced by the CNI (Flannel alone does nothing). Policies are additive allow-lists: once any policy selects a pod for a direction, everything not explicitly allowed in that direction is denied.

```yaml
# 1) Default deny all ingress and egress in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

```yaml
---
# 2) Allow DNS egress for every pod (otherwise nothing resolves)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-dns, namespace: shop }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
          podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports:
        - { protocol: UDP, port: 53 }
        - { protocol: TCP, port: 53 }
---
# 3) web may receive from the ingress namespace; api may receive from web; api may reach the
DB CIDR
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api, namespace: shop }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from: [{ podSelector: { matchLabels: { app: web } } }]
      ports: [{ protocol: TCP, port: 8080 }]
  egress:
    - to: [{ ipBlock: { cidr: 10.0.64.0/22 } }]
      ports: [{ protocol: TCP, port: 5432 }]
```

> **Warning:** In a single `from` entry, `namespaceSelector` and `podSelector` in the **same** list item are ANDed (pods with that label in those namespaces). As **separate** list items (each starting with `-`) they are ORed (any pod in those namespaces, or matching pods in this namespace). One dash changes the security meaning. Also note that NetworkPolicy matches pod IPs *after* Service DNAT, so policies refer to pod labels and ports, not Service ports.

Standard NetworkPolicy is L3/L4 and namespace-scoped. Cilium (`CiliumNetworkPolicy`, including DNS/FQDN and HTTP-aware rules) and Calico (`GlobalNetworkPolicy`) extend it; the upstream AdminNetworkPolicy API adds cluster-wide policies.

8.7 CoreDNS

CoreDNS runs as a Deployment in `kube-system`, behind the `kube-dns` Service. Its configuration is a Corefile in a ConfigMap:

```
.:53 {
    errors
```

```
    health { lameduck 5s }
    ready
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
    }
    prometheus :9153
    forward . /etc/resolv.conf { max_concurrent 1000 }
    cache 30
    loop
    reload
    loadbalance
}
corp.example.com:53 {
    forward . 10.50.0.53 10.50.0.54
}
```

Cluster names are answered by the `kubernetes` plugin; everything else is forwarded to the node's resolver (on EKS, the VPC resolver). The second block is a **stub domain**: on-prem names go to on-prem DNS. See Part 4 for `ndots:5` and NodeLocal DNSCache.

Debugging DNS from inside the cluster:

```
kubectl run -it --rm dnsdebug --image=nicolaka/netshoot --restart=Never -- bash
# inside:
cat /etc/resolv.conf
dig web.shop.svc.cluster.local +short
dig +search web +short
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50
```

8.8 Pod Lifecycle and Networking Gotchas

- **Readiness probes** control whether a pod's IP is in EndpointSlices. A failing readiness probe removes the pod from Services; a failing liveness probe restarts it. Never make liveness depend on downstream services.
- **Graceful shutdown race**: when a pod is deleted, endpoint removal (propagated to kube-proxy on every node and to external LBs) happens in parallel with `SIGTERM`. If the app exits immediately, some nodes still send it traffic and clients see connection refused or 502s. Add `preStop: { exec: { command: ["sleep", "10"] } }` (or the native `sleep` action) and handle SIGTERM by draining.
- **Hairpin traffic**: a pod calling its own Service may be load-balanced back to itself; the CNI must support hairpin mode.
- **MTU**: overlay CNIs reduce pod MTU; mismatches cause hangs on large responses (Part 9).
- **Conntrack table exhaustion**: busy nodes can fill `nf_conntrack_max`, producing `nf_conntrack: table full, dropping packet` in `dmesg`.

8.9 Service Mesh

A **service mesh** moves service-to-service networking concerns out of application code into infrastructure: mTLS, retries, timeouts, circuit breaking, traffic splitting, and uniform telemetry.

8.9.1 Architecture

```
Sidecar model (Istio classic, Linkerd)

  +----------- Pod A -----------+                +----------- Pod B -----------+
  | [app] -> localhost -> [proxy]| ==== mTLS ===>|[proxy] -> localhost -> [app]|
  +------------------------------+                +------------------------------+
          ^ iptables redirect captures all traffic into the sidecar
                       \                            /
                        +---- Control plane -----+
                          (istiod / linkerd-destination): config, service discovery,
                          certificate authority issuing workload identities

Ambient / sidecarless model (Istio ambient, Cilium)
  Per-node L4 proxy (ztunnel) handles mTLS and L4 policy;
  optional per-namespace L7 "waypoint" proxies only where L7 features are needed.
```

- **Data plane**: proxies (Envoy for Istio; a Rust micro-proxy for Linkerd) that handle the actual traffic.
- **Control plane**: distributes routing config, endpoints, and issues short-lived certificates.
- **Identity**: each workload gets a certificate encoding its identity (SPIFFE ID, e.g., `spiffe://cluster.local/ns/shop/sa/api`), derived from its Kubernetes ServiceAccount. Authorization policies use identities, not IPs.

8.9.2 Example: Istio traffic management and security

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: { name: default, namespace: shop }
spec:
  mtls: { mode: STRICT }          # reject plaintext between workloads
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: api-allow-web, namespace: shop }
spec:
  selector: { matchLabels: { app: api } }
  action: ALLOW
  rules:
    - from: [{ source: { principals: ["cluster.local/ns/shop/sa/web"] } }]
      to: [{ operation: { methods: ["GET", "POST"], paths: ["/v1/*"] } }]
---
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: api, namespace: shop }
```

```
spec:
  hosts: [api]
  http:
    - timeout: 3s
      retries: { attempts: 2, perTryTimeout: 1s, retryOn: "5xx,reset,connect-failure" }
      route:
        - { destination: { host: api, subset: v1 }, weight: 95 }
        - { destination: { host: api, subset: v2 }, weight: 5 }
```

8.9.3 Istio vs Linkerd, and when not to use a mesh

	Istio	Linkerd
Proxy	Envoy (sidecar) or ztunnel + waypoints (ambient)	linkerd2-proxy (Rust, sidecar)
Scope	Very feature-rich: gateways, multi-cluster, extensive traffic policy, VM integration	Focused on simplicity: mTLS, metrics, retries, traffic splitting
Operational cost	Higher	Lower
Config	VirtualService/DestinationRule plus Gateway API	Gateway API (HTTPRoute) and its own policy CRDs

Do not adopt a mesh just because it is fashionable. It adds latency (roughly a millisecond or less per hop for sidecars), resource overhead, upgrade complexity, and a new class of failures. Adopt it when you need zero-trust mTLS between many services, consistent L7 telemetry, or advanced traffic management across many teams. Retries configured in both the app and the mesh multiply (2 x 3 = 6 attempts) and can turn a slow dependency into a self-inflicted DDoS.

8.10 Lab: Kubernetes Networking Hands-On with kind

You need Docker, `kind`, and `kubectl`. We disable kind's default CNI and install Calico so NetworkPolicy is enforced.

Step 1. Create a cluster:

```
cat > kind.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
  disableDefaultCNI: true
  podSubnet: 192.168.0.0/16
  serviceSubnet: 10.96.0.0/12
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF
```

```
kind create cluster --name netlab --config kind.yaml
kubectl get nodes        # NotReady until a CNI is installed
```

Step 2. Install Calico and wait for nodes to become Ready:

```
kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/calico.yaml
kubectl -n kube-system rollout status ds/calico-node --timeout=180s
kubectl get nodes -o wide
```

Step 3. Deploy an app and a Service:

```
kubectl create namespace shop
kubectl -n shop create deployment web --image=hashicorp/http-echo:0.2.3 --replicas=3 --
/http-echo -listen=:8080 -text=hello
kubectl -n shop expose deployment web --port=80 --target-port=8080
kubectl -n shop get pods -o wide
kubectl -n shop get svc,endpointslices
```

Note each pod's IP from a different node's slice of 192.168.0.0/16, and the ClusterIP from 10.96.0.0/12.

Step 4. Test from a debug pod, by Service name, FQDN, and pod IP:

```
kubectl -n shop run client --image=nicolaka/netshoot --restart=Never -- sleep 3600
kubectl -n shop exec client -- curl -s web
kubectl -n shop exec client -- curl -s web.shop.svc.cluster.local
kubectl -n shop exec client -- cat /etc/resolv.conf
kubectl -n shop exec client -- ping -c1 -W1 $(kubectl -n shop get svc web -o
jsonpath='{.spec.clusterIP}')   # fails: expected
```

Step 5. Look at kube-proxy's rules on a node:

```
docker exec netlab-worker iptables -t nat -S KUBE-SERVICES | grep shop/web
```

Step 6. Apply default deny, then observe that the client times out (and that DNS also breaks):

```
kubectl apply -n shop -f - <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF
kubectl -n shop exec client -- curl -s -m 3 web || echo "blocked"
```

Step 7. Allow DNS egress and client-to-web traffic, then retest:

```
kubectl apply -n shop -f - <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-dns }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
          podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: client-to-web }
spec:
  podSelector: { matchLabels: { run: client } }
  policyTypes: [Egress]
  egress:
    - to: [{ podSelector: { matchLabels: { app: web } } }]
      ports: [{ protocol: TCP, port: 8080 }]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: web-from-client }
spec:
  podSelector: { matchLabels: { app: web } }
  policyTypes: [Ingress]
  ingress:
    - from: [{ podSelector: { matchLabels: { run: client } } }]
      ports: [{ protocol: TCP, port: 8080 }]
EOF
kubectl -n shop exec client -- curl -s -m 3 web
```

Notice the egress rule uses the **pod port 8080**, not the Service port 80, because policy is evaluated after DNAT.

Step 8. Clean up: `kind delete cluster --name netlab`.

Key takeaways

- Docker networking is namespaces + veth + a bridge + iptables MASQUERADE (outbound) and DNAT (published ports); published ports bypass INPUT-chain firewalls.
- Kubernetes requires every pod to have a routable IP without NAT; the CNI implements it with overlays or native routing.
- Keep node, pod, and service CIDRs distinct and non-overlapping with everything the cluster must reach.

- A ClusterIP is a virtual IP implemented by kube-proxy (iptables/IPVS/nftables) or eBPF; it balances per connection and does not answer ping.
- `externalTrafficPolicy: Local` preserves client IPs and avoids extra hops.
- Gateway API is the role-oriented successor to Ingress, with portable traffic splitting and matching.
- NetworkPolicy is default-allow until a policy selects a pod; always allow DNS when you default-deny egress, and mind AND vs OR selector semantics.
- Service meshes provide identity-based mTLS, L7 traffic control, and telemetry, at a real operational cost.

Part 9 — Troubleshooting Toolkit, Methodology, and Network Security

Knowing protocols is half the job. The other half is finding out, under pressure, which hop is dropping, delaying, or mangling traffic. This Part covers the essential tools with the output you should expect, a systematic methodology, the MTU problems that defeat most engineers, and the security architecture (zero trust, DDoS protection, WAFs) that production networks need.

9.1 The Toolkit

Tool	Layer	Answers
`ip`, `ethtool`	L1–L3	Is the interface up? What is my IP, route, neighbor? Errors on the NIC?
`ping`	L3	Is the host reachable? What is the RTT and loss?
`traceroute`, `tracepath`, `mtr`	L3	Which hop is losing or delaying packets? What is the path MTU?
`dig`, `getent`	L7 (DNS)	What does the name resolve to, and from whose perspective?
`nc`, `curl`, `openssl s_client`	L4–L7	Is the port open? Does TLS work? What does the app return?
`ss`, `nstat`, `conntrack`	L4	What sockets exist? Retransmissions? NAT state?
`tcpdump`, Wireshark, `tshark`	All	What is actually on the wire?
`nmap`	L3–L4	Which ports are open/filtered across a host or range?
`iperf3`	L4	What throughput can this path sustain?
`tc` (netem)	L2–L3	Simulate latency, loss, and jitter for testing

> **Tip:** Keep a debug image handy. `nicolaka/netshoot` contains nearly every tool in this Part and can be attached to any container's or pod's network namespace: `docker run -it --net container:<name> nicolaka/netshoot` or `kubectl debug -it <pod> --image=nicolaka/netshoot --target=<container>`.

9.1.1 ping

```
ping -c 5 10.0.64.10
```

```
PING 10.0.64.10 (10.0.64.10) 56(84) bytes of data.
64 bytes from 10.0.64.10: icmp_seq=1 ttl=64 time=0.512 ms
64 bytes from 10.0.64.10: icmp_seq=2 ttl=64 time=0.431 ms
...
--- 10.0.64.10 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 0.431/0.470/0.512/0.029 ms
```

Read: loss percentage, average RTT, and `mdev` (jitter). TTL hints at the OS and hop count: Linux starts at 64, Windows at 128, many network devices at 255.

> **Warning:** A failed ping does **not** prove a host is down. Security groups, Azure NSGs, and many firewalls block ICMP by default. A successful ping also does not prove your service works. Always follow with a TCP test on the real port.

9.1.2 traceroute and mtr

`traceroute` sends probes with increasing TTL; each router that decrements TTL to zero returns an ICMP Time Exceeded, revealing itself.

```
traceroute -n -T -p 443 api.example.com    # TCP SYN probes to 443 (gets through firewalls that block UDP/ICMP)
tracepath -n api.example.com               # also discovers path MTU, no root needed
```

`mtr` combines ping and traceroute continuously and is the best tool for intermittent loss:

```
mtr -n -r -c 100 -T -P 443 api.example.com
```

```
HOST: bastion                    Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.0.48.1                 0.0%   100    0.3   0.3   0.2   0.9   0.1
  2.|-- 100.65.12.33              0.0%   100    0.8   1.1   0.6   4.2   0.5
  3.|-- 52.93.28.144             38.0%   100    1.2   1.4   1.0   3.1   0.4
  4.|-- 99.83.89.10                0.0%   100    2.1   2.3   1.9   5.0   0.4
  5.|-- 203.0.113.10               0.0%   100   11.8  12.0  11.5  14.9   0.6
```

> **Note:** Loss at an intermediate hop (hop 3 above) that does **not** continue to later hops is not real loss. Routers rate-limit the ICMP replies they generate for probes because it is a low-priority control-plane task. Only loss that persists through to the final hop matters. Likewise, latency that jumps at one hop and stays high afterward is real (often a long-distance link); a spike at one hop that disappears at the next is not.

Lines of `* * *` mean that hop does not reply to probes; it is common in cloud networks and not a failure by itself.

9.1.3 nc (netcat): the fastest port test

```
nc -zv -w 3 10.0.64.10 5432
```

```
Connection to 10.0.64.10 5432 port [tcp/postgresql] succeeded!
nc: connect to 10.0.64.10 port 5433 (tcp) failed: Connection refused
nc: connect to 10.0.64.11 port 5432 (tcp) timed out: Operation now in progress
```

Succeeded, refused, and timed out are three different diagnoses (Part 3). If `nc` is unavailable, Bash can test a port itself: `timeout 3 bash -c '

Part 10 — Real-World Scenarios, Interview Q&A, and Cheat Sheet

This final Part ties the course together. The scenarios are composites of real incidents; each walkthrough shows the questions to ask, the commands to run, and how the evidence narrows the cause. Then comes an interview question bank with answers and a dense reference sheet to keep next to your terminal.

10.1 Scenario: “The Site Is Slow”

Report: Users say the web app has been slow since this morning. Dashboards show p99 latency up from 300 ms to 2.5 s. No obvious errors.

Step 1 — Define and scope. Slow for whom? All users or one region? All endpoints or some? Since when, and what changed? Suppose: all regions, mostly API calls, started around 09:10, and a deploy went out at 09:05.

Step 2 — Break the request into phases from a client close to users:

```
for i in $(seq 5); do
  curl -o /dev/null -s -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n' \
  https://shop.example.com/api/products
done
```

```
dns=0.004 tcp=0.021 tls=0.058 ttfb=2.310 total=2.312
dns=0.003 tcp=0.020 tls=0.055 ttfb=0.290 total=0.291
dns=0.004 tcp=0.022 tls=0.057 ttfb=2.402 total=2.405
```

DNS, TCP, and TLS are fast and stable. Time-to-first-byte is bimodal: some requests are normal, some take more than 2 s. The network to the edge is fine; the delay is behind the load balancer.

Step 3 — Follow the hops. Load balancer metrics show `TargetResponseTime` high, so the targets are slow. Are all targets slow or some? Per-target metrics or access logs (`upstream_addr` and `upstream_response_time` in Nginx) show requests to two of six pods are slow.

Step 4 — Look at the slow pods. What do they depend on? Their traces show a 2 s gap on a call to an external pricing API. From inside a slow pod:

```
kubectl exec -it api-7f9c-abcde -- sh -c 'time getent ahosts pricing.partner.com'
```

```
real    0m2.012s
```

DNS resolution inside the pod takes 2 s. On the fast pods it takes 3 ms. The two slow pods are on the same node. On that node, the NodeLocal DNSCache pod was evicted during the 09:05 deploy (memory pressure), and the glibc resolver is timing out on the dead local address before retrying the next server.

Root cause and fix: DNS cache eviction on one node. Restore the DNS cache DaemonSet with proper resource requests and a priority class; reduce lookup pressure (lower `ndots`, use FQDNs); add a DNS latency SLO and alert.

Lessons: Measure phases before guessing. Bimodal latency usually means some instances, connections, or paths differ. Slowness is often a dependency, and DNS timeouts manifest as round numbers (1 s, 2 s, 5 s).

10.2 Scenario: “Can’t Reach the Database from a Pod”

Report: A new service in namespace `orders` cannot connect to the PostgreSQL RDS instance. The error: `connection to server at "orders-db.abc123.us-east-1.rds.amazonaws.com" (10.0.64.25), port 5432 failed: Connection timed out`.

Step 1 — Read the error. It resolved the name (DNS works) and got an IP; the connection **timed out**, not refused. Something is dropping packets: security group, NACL, NetworkPolicy, routing.

Step 2 — Reproduce from the pod’s network namespace:

```
kubectl -n orders debug -it orders-api-6d4b-xyz --image=nicolaka/netshoot --target=app --
bash
nc -zv -w 3 10.0.64.25 5432          # timed out
ip route get 10.0.64.25              # via the node/CNI, as expected
```

Step 3 — Compare with a working path. Another service in namespace `payments` reaches the same database. Test from the node hosting the failing pod:

```
nc -zv -w 3 10.0.64.25 5432          # from the node: succeeded
```

The node can reach the DB; the pod cannot. That points to something pod-specific: NetworkPolicy, security groups for pods, or SNAT behavior.

Step 4 — Check NetworkPolicy:

```
kubectl -n orders get networkpolicy
```

```
NAME           POD-SELECTOR   AGE
default-deny   <none>         12d
allow-dns      <none>         12d
```

There is a default-deny egress policy in `orders` and only DNS is allowed. The `payments` namespace has an explicit egress rule to `10.0.64.0/22:5432`; `orders` does not.

Fix: add an egress policy allowing TCP 5432 to the database subnet CIDRs for the `orders-api` pods. Verify with `nc`, then the application.

Other causes to check in this scenario (in order of likelihood):

Check	How
DB security group allows the source	With the AWS VPC CNI, traffic to destinations inside the VPC keeps the **pod IP** as source, and the ENI carries the node's security groups (or the pod's own, with security groups for pods); allow that SG or the pod subnet CIDRs
NACLs on DB subnets allow return traffic	Ephemeral ports outbound from the DB subnet
Route from pod CIDR to DB	Overlay CNIs SNAT pod traffic to the node IP; with custom networking (100.64.0.0/10 pods) the DB must allow that CIDR or traffic must be SNATed
DB listening/accepting	RDS status, `max_connections`, `pg_hba.conf` (gives an auth error, not a timeout)
Wrong endpoint	Reader vs writer endpoint, a different region or VPC

Lesson: "Works from the node, fails from the pod" localizes the problem to pod-level controls in one test.

10.3 Scenario: "Intermittent 502s"

Report: About 0.5% of requests through the ALB return 502 Bad Gateway. No pattern by endpoint. Application logs show no errors for those requests; it is as if they never arrived.

Step 1 — Who generates the 502? The ALB access logs show `elb_status_code=502` and `target_status_code=-`. The target never returned a status: the ALB's connection to the target failed or was reset.

Step 2 — Look for timing patterns. The errors cluster on low-traffic paths and after idle periods. Correlate with connection reuse: the ALB keeps idle keep-alive connections to targets and reuses them.

Step 3 — Check timeouts at each hop:

```
ALB idle timeout:                 60 s
Target: Node.js server.keepAliveTimeout   5 s   (default)
```

The target closes idle connections after 5 s. If the ALB sends a new request on a connection at the instant the target closes it, the target responds with a FIN or RST and the ALB returns 502. Capture on a target to confirm:

```
sudo tcpdump -ni eth0 'tcp port 8080 and (tcp[tcpflags] & (tcp-fin|tcp-rst) != 0)'
```

```
11:02:15.501 IP 10.0.1.40.8080 > 10.0.48.12.31544: Flags [F.], seq 9381, ack 1202
11:02:15.502 IP 10.0.48.12.31544 > 10.0.1.40.8080: Flags [P.], seq 1202:1789, ack 9382  <-
```

```
request on closing conn
11:02:15.502 IP 10.0.1.40.8080 > 10.0.48.12.31544: Flags [R], seq 9382
```

Fix: set the target's keep-alive timeout **greater** than the ALB idle timeout (e.g., `server.keepAliveTimeout = 65000` and `headersTimeout = 66000` in Node.js; `keepalive_timeout 75s` in Nginx; `--timeout-keep-alive` in Uvicorn).

Other common causes of intermittent 502s:

Cause	Evidence
Pods terminated before LB deregistration	502s spike during deploys; fix with `preStop` sleep, readiness gates, graceful SIGTERM handling
Target crashes or OOM kills	Restart counts, `OOMKilled`
Response headers too large for proxy buffers	Nginx `upstream sent too big header`; raise `proxy_buffer_size`
Protocol mismatch	LB speaks HTTPS to an HTTP target, or HTTP/2 to an HTTP/1-only target
PROXY protocol enabled on one side only	Every request fails, not 0.5%
Conntrack table full on nodes	`dmesg` shows `nf_conntrack: table full`

Lesson: a 502 means "the proxy's upstream misbehaved". Find which proxy, read its upstream error, and check timeout ordering first.

10.4 Scenario: "DNS Works on One Host but Not Another"

Report: `curl https://internal-api.corp.example.com` works on `build-01` but fails on `build-02` with `Could not resolve host`. Both are in the same subnet.

Step 1 — Compare what each host's application stack sees:

```
# on both hosts
getent ahosts internal-api.corp.example.com
cat /etc/resolv.conf
resolvectl status | sed -n '1,25p'
grep hosts /etc/nsswitch.conf
grep -i corp /etc/hosts
```

Findings:

```
build-01:  /etc/resolv.conf -> nameserver 127.0.0.53, search corp.example.com
           resolvectl: DNS Servers 10.0.0.2, plus per-link route ~corp.example.com ->
10.50.0.53 (VPN)
```

```
            /etc/hosts: (no entry)
build-02:   /etc/resolv.conf -> nameserver 10.0.0.2  (static file, not systemd-resolved)
```

Step 2 — Query each resolver directly:

```
dig @10.0.0.2 internal-api.corp.example.com +short      # empty, NXDOMAIN
dig @10.50.0.53 internal-api.corp.example.com +short    # 10.50.12.7
```

The VPC resolver does not know the corporate zone. `build-01` works because systemd-resolved has a split-DNS route sending `corp.example.com` to the on-prem DNS server over the VPN interface. `build-02` was configured by a provisioning script that overwrote `/etc/resolv.conf` with a static file, bypassing systemd-resolved.

Fix options:

1. Restore `/etc/resolv.conf` as a symlink to `/run/systemd/resolve/stub-resolv.conf` on `build-02`.
2. Better, make it host-independent: create a Route 53 Resolver **outbound endpoint** with a forwarding rule for `corp.example.com` to `10.50.0.53`, associated with the VPC. Now every host using the VPC resolver resolves corporate names.

Other causes for "one host works, one does not": an `/etc/hosts` override on one host, different search domains (short names), IPv6 preference (AAAA answered but no IPv6 route), a local caching daemon with a stale entry (`resolvectl flush-caches`), a container with a different resolv.conf than its host, or DNS traffic blocked from one host's security group or NetworkPolicy (UDP and TCP 53).

Lesson: `dig` asks a server; `getent` shows what the app sees. Compare both on the working and failing hosts, and diff the resolver configuration.

10.5 Scenario: "New VPC Peering, Nothing Flows"

Report: Peering `pcx-0a1` between `vpc-app` (10.0.0.0/16) and `vpc-data` (10.20.0.0/16) is `Active` , but the app cannot reach `10.20.5.10:6379` .

Walkthrough:

```
1. Active peering only means the link exists. Check routes on BOTH sides:
   vpc-app route tables for the app subnets:   10.20.0.0/16 -> pcx-0a1 ?   (present)
   vpc-data route table for the Redis subnet:  10.0.0.0/16  -> pcx-0a1 ?   (MISSING)
2. Requests arrive at Redis; replies have no route back -> timeout.
3. Also verify: Redis SG allows 6379 from 10.0.0.0/16 (or the peer SG id),
   NACLs, and that the app uses the private DNS name with
   "allow DNS resolution from accepter VPC" enabled if it resolves via public names.
```

Fix: add the return route in `vpc-data` . Reachability Analyzer would have reported "no route to destination" at the `vpc-data` route table. **Lesson:** routing is per direction (Part 3), and cloud consoles showing "Active" or "Available" prove configuration exists, not that traffic flows.

10.6 Scenario: “Outbound Calls Sporadically Time Out from Private Subnets”

Report: Workers calling a third-party API from private subnets see sporadic connect timeouts at peak hours. Other egress is fine.

Walkthrough: NAT gateway CloudWatch metric `ErrorPortAllocation` is non-zero at peak and `ActiveConnectionCount` is high. All workers hit one destination IP:port through one NAT gateway public IP, exceeding about 55,000 simultaneous connections to that destination. Workers create a new HTTPS connection per request and close it.

Fix: enable connection pooling and keep-alive in the HTTP client (biggest win), add secondary public IPs to the NAT gateway, and spread egress across per-AZ NAT gateways. **Lesson:** NAT is stateful and finite. TIME_WAIT and ephemeral port math from Part 3 apply to every middlebox.

10.7 Capstone Lab: Run an Incident

Step 1. Rebuild the Part 8 kind cluster with Calico and the `shop` app, and the Part 6 HAProxy lab.

Step 2. Ask a colleague to inject one fault without telling you which: a default-deny NetworkPolicy without DNS egress; a CoreDNS deployment scaled to 0; a Service selector typo (`app: wbe`); a readiness probe on the wrong port; HAProxy `timeout server 1s` with a backend that sleeps 2 s; or `tc netem loss 10%` on a kind node (`docker exec <node> tc qdisc add dev eth0 root netem loss 10%`).

Step 3. Diagnose using Section 9.2’s methodology. Record each command and what it ruled in or out.

Step 4. Write a short postmortem: impact, timeline, root cause, evidence, fix, and one prevention action (an alert, a test, or a policy).

Expected signals: empty EndpointSlices for the selector typo and readiness failures; `SERVFAIL` or timeouts for DNS faults; HAProxy `sH--` termination state (server timeout) with 504s for the timeout fault; mtr loss at the final hop for netem.

10.8 Interview Questions and Answers

10.8.1 Beginner

1. What is the difference between a switch and a router? A switch forwards frames within a LAN/VLAN using MAC addresses (L2). A router forwards packets between networks using IP addresses and a route table (L3), rewriting the L2 header and decrementing TTL at each hop.

2. What is the difference between TCP and UDP? TCP is connection-oriented, reliable, ordered, with flow and congestion control (HTTP/1.1, HTTP/2, SSH, databases). UDP is connectionless best-effort datagrams with an 8-byte header (DNS, DHCP, VoIP, QUIC). Choose UDP when latency matters more than reliability or when the application implements its own reliability.

3. Describe the TCP three-way handshake. Client sends SYN with its initial sequence number; server replies SYN-ACK with its own ISN and acknowledges the client's; client sends ACK. Both sides confirm sequence numbers and negotiate options (MSS, window scale, SACK). It costs one RTT.

4. What is a subnet mask and what does /24 mean? The mask marks which bits of an address are the network portion. /24 means 24 network bits (255.255.255.0), leaving 8 host bits: 256 addresses, 254 usable.

5. What are the RFC 1918 private ranges? 10.0.0.0/8, 172.16.0.0/12 (172.16.0.0–172.31.255.255), and 192.168.0.0/16.

6. What does ARP do? It maps an IPv4 address to a MAC address on the local link by broadcasting a request and receiving a unicast reply. Hosts ARP for the destination if it is on-link, otherwise for their default gateway.

7. What is DNS and which port does it use? The system that resolves names to records (such as IP addresses). UDP 53 primarily, TCP 53 for large responses and zone transfers; DoT uses 853 and DoH uses 443.

8. What is the difference between an A record and a CNAME? An A record maps a name directly to an IPv4 address. A CNAME aliases one name to another name, which must then be resolved. CNAMEs cannot exist at the zone apex or alongside other records at the same name.

9. What is a default gateway? The next-hop router for destinations not matched by a more specific route (0.0.0.0/0). It must be on a directly connected subnet.

10. What is NAT and why is it used? Network Address Translation rewrites IP addresses (and ports, for PAT) as packets pass through a device. It lets many private hosts share public IPs and can connect overlapping networks.

11. What are well-known ports for HTTP, HTTPS, SSH, and DNS? 80, 443, 22, and 53.

12. What does ping test, and why might it fail even if a server is up? ICMP echo reachability and RTT at L3. Firewalls and security groups often block ICMP, so failure does not prove the host is down; test the real TCP port.

13. What is DHCP and what is DORA? DHCP automatically assigns IP configuration. DORA: Discover (client broadcast), Offer (server), Request (client), Acknowledge (server). UDP 67/68.

14. What is the difference between HTTP and HTTPS? HTTPS is HTTP inside TLS, providing encryption, integrity, and server authentication via certificates. Default port 443 instead of 80.

15. What is a VLAN? A logical L2 broadcast domain on shared switches, identified by a 12-bit 802.1Q tag. Traffic between VLANs must be routed.

10.8.2 Intermediate

16. Walk through what happens when you type `https://example.com` in a browser. Browser and OS caches, then a stub query to the recursive resolver, which iterates root, TLD, and authoritative servers. The OS routes to the IP (ARP for the gateway), TCP handshake, TLS handshake (SNI, certificate validation, key

exchange, ALPN), HTTP request with Host header, possibly through CDN and load balancer to a backend, response, then rendering with additional requests over the same connection(s).

17. How many usable hosts are in a /27, and what is the network and broadcast of 10.1.1.100/27? 30 usable. Block size 32: 100 falls in 96–127, so network 10.1.1.96, broadcast 10.1.1.127, hosts .97–.126.

18. Security group vs NACL? Security groups are stateful, allow-only, attached to ENIs, evaluate all rules, and can reference other security groups. NACLs are stateless, subnet-level, ordered allow/deny rules, and require explicit return rules for ephemeral ports.

19. What is the difference between L4 and L7 load balancing? L4 balances connections using IP/port (any TCP/UDP protocol, very fast, TLS passthrough possible). L7 understands HTTP and routes each request by host, path, or headers, terminates TLS, and supports retries, rewrites, and auth.

20. What is TIME_WAIT and when is it a problem? The state on the side that closes first, lasting 2 x MSL (60 s on Linux), to absorb delayed packets and retransmitted FINs. It becomes a problem when a client opens many short connections to one destination and runs out of ephemeral ports. Fix with connection reuse; `tcp_tw_reuse` helps outbound.

21. What does a large number of CLOSE_WAIT sockets indicate? The peer closed the connection but the local application never called `close()`: a socket leak in the application.

22. Connection refused vs connection timed out? Refused: an RST came back; the host is reachable but nothing listens on the port (or a firewall actively rejects). Timed out: no reply; packets are being dropped (security group, firewall, routing, host down).

23. Explain a public vs private subnet in AWS. A public subnet's route table has a 0.0.0.0/0 route to an Internet Gateway; instances with public IPs are reachable. A private subnet does not; outbound IPv4 goes via a NAT gateway, and inbound from the Internet is impossible without a load balancer.

24. Why is VPC peering not transitive, and what do you use instead? Peering is a point-to-point relationship and AWS does not route through a VPC to another peer (no edge-to-edge routing). For many VPCs use Transit Gateway (or Cloud WAN), which acts as a routed hub with segmented route tables.

25. What is the TLS chain of trust and the most common chain problem? The leaf certificate is signed by an intermediate, which is signed by a root in the client's trust store. Servers must send the leaf plus intermediates. The most common problem is a missing intermediate, which breaks non-browser clients.

26. What is SNI and why does it matter? Server Name Indication: the client sends the hostname in the ClientHello so a server hosting many sites can present the right certificate. Testing without SNI (`openssl s_client` without `-servername`) often returns the wrong default certificate.

27. What are the Kubernetes Service types? ClusterIP (internal virtual IP), NodePort (port 30000–32767 on every node), LoadBalancer (external LB via cloud or MetalLB), ExternalName (DNS CNAME), plus headless (`clusterIP: None`) returning pod IPs directly.

28. How does a DNS TTL affect a migration? Resolvers cache answers for the TTL, so changes take up to the old TTL to propagate. Lower the TTL well ahead of the change (at least one old-TTL period), migrate, then raise it. Remember negative caching and clients that ignore TTLs or hold connections.

29. What does HTTP/2 improve over HTTP/1.1, and what does HTTP/3 add? HTTP/2 adds binary framing, multiplexed streams on one connection, and header compression, removing application-level head-of-line blocking. HTTP/3 runs over QUIC on UDP, eliminating TCP head-of-line blocking, integrating TLS 1.3 for 1-RTT/0-RTT setup, and supporting connection migration.

30. What is MTU and how do you detect a PMTUD black hole? The largest packet a link carries (1500 for Ethernet). A black hole occurs when ICMP "fragmentation needed" is blocked, so large packets are silently dropped. Symptoms: handshakes and small requests work, large transfers hang. Test with `ping -M do -s <size>` and `tracepath`.

10.8.3 Advanced

31. How does kube-proxy implement a ClusterIP in iptables mode, and what are its limitations? It programs nat-table rules on every node: a KUBE-SERVICES match on VIP:port jumps to a per-Service chain that randomly selects a per-endpoint chain which DNATs to a pod IP:port; conntrack handles replies. Limitations: linear rule evaluation and slow updates at very large scale, per-connection (not per-request) balancing, and no ICMP to the VIP. IPVS, nftables, or eBPF (Cilium) address scale.

32. Why does gRPC load-balance poorly behind a Kubernetes Service, and how do you fix it? gRPC uses long-lived HTTP/2 connections that multiplex all requests; Services balance per connection, so each client pins to one pod. Fix with client-side balancing over a headless Service (gRPC `round_robin` with DNS or xDS), an L7 proxy or service mesh that balances per request, or periodic connection recycling (max connection age on the server).

33. Explain BGP path selection basics and where you would use BGP as a DevOps engineer. Routes are chosen by policy: highest LOCAL_PREF, shortest AS_PATH, lowest origin, lowest MED, eBGP over iBGP, lowest IGP metric to next hop, then tie-breakers. DevOps uses: Direct Connect and VPN dynamic routing (prefer one path with AS-path prepending or local preference), Calico peering with ToR switches, MetalLB/Cilium advertising LoadBalancer IPs, and anycast services.

34. A service has p99 latency spikes of about 200 ms and exactly 1 s. What do you suspect? TCP retransmissions: ~200 ms is the Linux minimum RTO for retransmitted data, 1 s is the initial SYN retransmission timeout (lost SYN or SYN-ACK, often from SYN backlog overflow or packet loss). Check `nstat` for retransmits and ListenOverflows, `ss -ti`, mtr for loss, and accept queue saturation on the server.

35. How would you design multi-region active-active ingress? Global entry via anycast (Global Accelerator, cloud global LB, or CDN) or latency-based DNS with health checks; regional L7 load balancers; stateless services; data replication strategy (multi-region database or region-pinned users); health checks that represent regional health; failover tested regularly; and awareness that DNS failover is bounded by TTL and client caching.

36. What is the bandwidth-delay product and why does it matter for cross-region transfers? BDP = bandwidth x RTT, the amount of data in flight needed to fill a path. A single TCP connection's throughput is limited to window/RTT; with 100 ms RTT, a 1 Gbit/s path needs 12.5 MB of window. Small buffers or a lossy path (CUBIC backs off) cap throughput; fixes include buffer tuning, BBR, and parallel streams.

37. How do you preserve client source IPs through an AWS NLB into Kubernetes? Use NLB with client IP preservation (instance or IP targets), `externalTrafficPolicy: Local` to avoid SNAT between nodes (or IP-mode targets that hit pods directly), or PROXY protocol v2 if a proxy is in the path, with the backend configured to parse it. Ensure security groups allow the real client CIDRs.

38. Explain how PrivateLink differs from VPC peering. PrivateLink exposes a single service (behind an NLB/GWLB) via interface endpoint ENIs in consumer VPCs: unidirectional, consumer-initiated, works with overlapping CIDRs, and scales to many consumers. Peering connects entire networks bidirectionally, requires non-overlapping CIDRs and routes on both sides, and is non-transitive.

39. What is conntrack and how can it cause outages? The netfilter connection tracking table used for stateful filtering and NAT (Docker, kube-proxy, NAT gateways). When it fills (`nf_conntrack_max`), new connections are dropped (`table full, dropping packet`). UDP-heavy workloads (DNS) and short-lived connections are common culprits; fixes include raising limits, reducing timeouts, NodeLocal DNSCache, or eBPF data planes that bypass conntrack for some flows.

40. What does zero trust mean in a Kubernetes environment? No implicit trust based on network location: workload identity (ServiceAccount-based SPIFFE IDs), mutual TLS for all service-to-service traffic, identity-based authorization policies (mesh AuthorizationPolicy), default-deny NetworkPolicies, egress controls, short-lived credentials, and comprehensive auditing.

41. How do you debug a pod that can resolve DNS but intermittently takes 5 s to do so? Classic causes: a conntrack race when A and AAAA queries are sent in parallel over the same UDP socket, one CoreDNS replica unhealthy, or upstream resolver throttling (AWS limit 1,024 packets/s per ENI). Check CoreDNS metrics and logs, capture UDP 53 on the node, try `single-request-reopen`, deploy NodeLocal DNSCache, lower ndots, and scale CoreDNS.

42. What happens during a TLS 1.3 handshake, and what is forward secrecy? ClientHello with key shares, SNI, and ALPN; ServerHello with its key share; then encrypted extensions, certificate, CertificateVerify, and Finished; client Finished. Keys come from ephemeral ECDHE, so compromising the server's long-term private key later cannot decrypt past sessions: that is forward secrecy.

43. How do you prevent connection errors during rolling deployments behind a load balancer? Readiness probes so new pods receive traffic only when ready; `preStop` delay so endpoints are removed from kube-proxy and the LB before shutdown; graceful SIGTERM handling that drains in-flight requests; LB deregistration delay aligned with request durations; `maxUnavailable: 0` and surge; keep-alive timeouts ordered correctly.

44. Explain the AND vs OR semantics in NetworkPolicy selectors. In a `from`/`to` entry, a `namespaceSelector` and `podSelector` in the same list element are ANDed (matching pods in matching namespaces). As separate list elements they are ORed. A misplaced dash can open access to all pods in a namespace or to every namespace.

45. You see 38% loss at hop 3 in mtr but 0% at the destination. Is there a problem? No. Intermediate routers deprioritize or rate-limit ICMP generation for probes addressed to them. Only loss that continues to the final hop affects traffic.

46. How would you reduce NAT gateway costs in a large AWS environment? Gateway endpoints for S3 and DynamoDB, interface endpoints for heavily used services (ECR, STS, CloudWatch), pull-through caches for container images, keeping traffic within an AZ (per-AZ NAT), IPv6 with egress-only Internet gateways where possible, and centralized egress via Transit Gateway only if the processing charges net out.

10.9 Cheat Sheet

10.9.1 Common ports

Port	Proto	Service	Port	Proto	Service
20/21	TCP	FTP data/control	1194	UDP	OpenVPN
22	TCP	SSH, SFTP, Git over SSH	1433	TCP	Microsoft SQL Server
23	TCP	Telnet (insecure)	1521	TCP	Oracle DB
25	TCP	SMTP	2049	TCP/UDP	NFS
53	UDP/TCP	DNS	2181	TCP	ZooKeeper
67/68	UDP	DHCP server/client	2379/2380	TCP	etcd client/peer
80	TCP	HTTP	3000	TCP	Grafana (common default)
110	TCP	POP3	3306	TCP	MySQL / MariaDB
123	UDP	NTP	3389	TCP	RDP
143	TCP	IMAP	4789	UDP	VXLAN
161/162	UDP	SNMP / traps	5000	TCP	Docker registry (common)
179	TCP	BGP	5201	TCP	iperf3
389 / 636	TCP	LDAP / LDAPS	5432	TCP	PostgreSQL
443	TCP	HTTPS	5672	TCP	AMQP (RabbitMQ)
443	UDP	HTTP/3 (QUIC)	6379	TCP	Redis
465 / 587	TCP	SMTP over TLS / submission	6443	TCP	Kubernetes API server
500 / 4500	UDP	IPsec IKE / NAT-T	8080	TCP	Alternate HTTP
514	UDP	Syslog	8443	TCP	Alternate HTTPS
853	TCP	DNS over TLS	9090	TCP	Prometheus
993 / 995	TCP	IMAPS / POP3S	9092	TCP	Kafka

Port	Proto	Service	Port	Proto	Service
10250	TCP	kubelet API	9100	TCP	node_exporter
30000–32767	TCP/UDP	Kubernetes NodePort range	9200/9300	TCP	Elasticsearch HTTP/transport
51820	UDP	WireGuard	27017	TCP	MongoDB
15000–15090	TCP	Istio sidecar ports (15001 outbound, 15006 inbound, 15090 metrics)	8500	TCP	Consul HTTP

10.9.2 Subnet quick reference

Prefix	Mask	Addresses	Usable	AWS usable
/16	255.255.0.0	65,536	65,534	65,531
/20	255.255.240.0	4,096	4,094	4,091
/22	255.255.252.0	1,024	1,022	1,019
/24	255.255.255.0	256	254	251
/26	255.255.255.192	64	62	59
/27	255.255.255.224	32	30	27
/28	255.255.255.240	16	14	11
/30	255.255.255.252	4	2	n/a (min /28)

Block size = 256 - mask octet. Network = largest multiple of block size not above the address. Broadcast = next network - 1.

10.9.3 Commands by task

```
# Interfaces, addresses, routes, neighbors
ip -br addr                    ; ip -br link
ip route                       ; ip route get 10.0.64.10
ip -6 route                    ; ip neigh
ip -s link show eth0           ; ethtool -S eth0 | grep -iE 'err|drop'
ip netns list                  ; ip netns exec <ns> <cmd>
nsenter -t <pid> -n ip addr    # enter a container's network namespace by PID

# DNS
dig +short name                ; dig @server name TYPE
dig +trace name                ; dig -x 203.0.113.10
getent ahosts name             ; resolvectl status ; resolvectl flush-caches

# Reachability and path
ping -c 4 host                 ; ping -M do -s 1472 host
mtr -n -r -c 100 -T -P 443 host
```

```
traceroute -n -T -p 443 host   ; tracepath -n host
nc -zv -w 3 host port          ; nc -u -zv host 53

# Sockets and TCP health
ss -tlnp                       ; ss -tunap
ss -tan state time-wait | wc -l
ss -ti dst 10.0.2.40           ; ss -s
nstat -az | grep -E 'Retrans|ListenOverflows|ListenDrops'
conntrack -L | head            ; conntrack -C
cat /proc/sys/net/netfilter/nf_conntrack_max

# HTTP and TLS
curl -v https://host/
curl -o /dev/null -s -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect}
ttfb=%{time_starttransfer} total=%{time_total}\n' https://host/
curl --resolve host:443:10.0.0.5 https://host/
openssl s_client -connect host:443 -servername host -showcerts </dev/null
openssl x509 -in cert.pem -noout -subject -issuer -dates -ext subjectAltName

# Capture
tcpdump -ni any host 10.0.0.5 and port 443
tcpdump -ni eth0 'tcp[tcpflags] & (tcp-syn|tcp-rst) != 0'
tcpdump -ni eth0 -w cap.pcap -s 0 -c 10000 port 5432

# Firewall
iptables -L -n -v --line-numbers ; iptables -t nat -S
nft list ruleset

# Throughput and impairment
iperf3 -s                      ; iperf3 -c host -P 4 -t 10
tc qdisc add dev eth0 root netem delay 100ms loss 1% ; tc qdisc del dev eth0 root

# Kubernetes
kubectl get svc,endpointslices -n ns
kubectl get networkpolicy -A
kubectl debug -it pod/<p> --image=nicolaka/netshoot --target=<container>
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=100
kubectl get pods -o wide       # pod IPs and nodes
```

10.9.4 Useful sysctls

Sysctl	Default	Purpose
`net.ipv4.ip_forward`	0	Route between interfaces (1 on routers, Docker and Kubernetes nodes)
`net.ipv4.ip_local_port_range`	32768 60999	Ephemeral source ports
`net.ipv4.tcp_tw_reuse`	2 (loopback only)	Reuse TIME_WAIT for outbound connections (set 1)

Sysctl	Default	Purpose
`net.core.somaxconn`	4096 (kernel 5.4+)	Max accept backlog
`net.ipv4.tcp_max_syn_backlog`	varies	SYN queue size
`net.ipv4.tcp_syncookies`	1	SYN flood protection
`net.ipv4.tcp_keepalive_time`	7200	Seconds before first keepalive probe
`net.ipv4.tcp_congestion_control`	cubic	Set `bbr` for long, lossy paths
`net.ipv4.tcp_mtu_probing`	0	1 = probe when black hole detected
`net.netfilter.nf_conntrack_max`	memory-based	Conntrack table size
`net.ipv4.conf.all.rp_filter`	0–2 by distro	Reverse path filtering (strict = 1)

10.9.5 Error message decoder

Message	Layer	First suspects
`Name or service not known` / `Could not resolve host`	DNS	resolv.conf, search domains, resolver reachability, split-horizon
`Network is unreachable`	L3 (local)	No route on this host
`No route to host`	L3	ICMP unreachable from a router or firewall reject; ARP failure on local subnet
`Connection timed out`	L3/L4	Firewall, SG, NACL, NetworkPolicy drop; host down; return route
`Connection refused`	L4	Nothing listening; wrong port; bound to 127.0.0.1
`Connection reset by peer`	L4	App crash, idle timeout on a middlebox, keep-alive race, proxy protocol mismatch
`certificate verify failed`	TLS	Expired, missing intermediate, untrusted CA, hostname not in SANs
`wrong version number`	TLS	TLS spoken to a non-TLS port
`502 Bad Gateway`	Proxy	Upstream refused/reset/invalid response; keep-alive mismatch
`503 Service Unavailable`	Proxy	No healthy targets; overload
`504 Gateway Timeout`	Proxy	Upstream too slow; timeout too short

Key takeaways

- Break every problem into phases and hops; measure before you guess.
- “Works from X, not from Y” is the most powerful localization technique: node vs pod, host A vs host B, inside vs outside.
- Read the exact error; it usually names the layer.
- Many intermittent failures are timeout mismatches, finite state (NAT ports, conntrack), or DNS latency.
- Cloud “Active” or “Available” status proves configuration, not traffic flow; verify routes both ways and use flow logs and reachability tools.
- Interviewers reward layered reasoning: explain why, then name the command that proves it.
- Keep the cheat sheet handy, but understand every line of it: that is what makes you an advanced DevOps engineer.